

Verifying Agent Repairs: Evidence Quality, Deployment Gains, and Cost

Anonymous ACL submission

Abstract

Self-improving agent loops change an agent’s harness (its prompts, tool descriptions and control code) and keep a change only when verification evidence says it helps. We evaluate that evidence by deploying every candidate change, accepted or rejected, on held-out tasks, and count its cost. Inside RRSI, a regularised self-improvement loop that edits harness code, run with DeepSeek models on τ^2 -bench and AppWorld, most of the 183 measured candidates did not help held out, and evidence that never runs the changed agent on solved tasks accepted changes that crashed the harness or blocked legitimate actions. A cheaper rule that stops a candidate’s evaluation once RRSI’s rule is predicted to reject it was non-inferior to full re-evaluation on the loop’s recorded decisions at 57–62% of the episodes, and on the loops’ domain and first ten rounds its recorded decisions predicted a loss of only 1.0 point of held-out gain. Yet in a registered comparison of twelve independent airline loops run for ten rounds each, loops driven by it gained 4.4 points less (90% interval -9.4 to -0.1 ; -6.9 among the eight run concurrently), saving 39% of the candidate-evaluation episodes but 10% of the loop’s cost. They accepted fewer changes, missing six that full evaluation would have chosen in their states; we cannot separate this from the early-stopped scores their proposer saw, and adjusting for a chance imbalance in the loops’ calibrated noise bands halves the pooled difference. Offline, a stopping rule that models the uncertainty of the cost change keeps 88% of full evaluation’s acceptances instead of 74%. A study of one round of attribution-driven repair found no verified patch set reliably better than the unverified one. Scoring a verification rule on a loop’s recorded decisions can miss what it does to the loop it drives.

1 Introduction

A growing family of systems improves LLM agents from their own failures. The agent runs, an ana-

lyst model reads the failed trajectories, a proposer changes the agent’s *harness*, the prompts, tool descriptions and code around a frozen model, and the loop keeps the change or discards it (Shinn et al., 2023; Zhao et al., 2024; Hu et al., 2025; Zhang et al., 2025a; Agrawal et al., 2026; Chen et al., 2026; Zhang et al., 2026b; Xia et al., 2026). The check that decides is where these systems differ most: some replay the failure with the change applied (Ma et al., 2025; Lin et al., 2026; Shah, 2026), some ask an LLM judge, some re-run tasks that used to pass (Chen et al., 2026), and RRSI re-evaluates every candidate twice on its whole training set (Xia et al., 2026). What every check is meant to estimate is a change’s *deployment gain*, how much it raises success on tasks the loop has not seen, and that gain can differ from what the loop observes on its own tasks: RRSI reports in-distribution gains that shrink out of distribution (Xia et al., 2026).

This paper asks two questions about verification evidence. *Which evidence predicts a change’s deployment gain?* And *is the evidence worth what it costs?* Answering the first needs ground truth that verification does not provide: deploying the changes, from the first step, on independent tasks. Answering the second needs an account of what verification costs and what the change it selects is worth over a deployment horizon, which we write as $\text{Net}(N)$ (§3). We price every piece of evidence in agent episodes and answer both questions in two studies.

Study 1 (§4) is one round of attribution-driven repair of a DeepSeek-V4-Flash agent on the retail and airline domains of τ^2 -bench (Barres et al., 2025), mostly on injected faults. No patch set chosen by ten verification policies showed a reliable improvement over applying the attributed patches unverified, which raised held-out success by 11.7 (retail) and 6.7 (airline) points, and replays could not resolve the differences between sets.

Study 2 (§5, registered) tests evidence with the power and the modification space Study 1 lacked. We build on RRSI’s open-source loop (Xia et al., 2026), whose proposer edits harness code over many rounds of natural failures, port it to DeepSeek models and τ^2 -bench, make targeted replays work for code-level changes, and replace RRSI’s selection step with each kind of evidence. We deploy every candidate and incumbent on held-out tasks, with endpoints, decision rule and analysis code fixed before any held-out episode was run. Evidence that does not run the changed agent on solved tasks accepted a change that crashed the harness and one that blocked legitimate actions, and no rule was shown to beat keeping the incumbent. Sequential evaluation, which stops a candidate’s evaluation once RRSI’s rule is predicted to reject it, was non-inferior to full re-evaluation on the loop’s recorded decisions at 57% of its episodes (62% on App-World), but against a reference that itself gained almost nothing. On airline’s first ten rounds, where full re-evaluation did gain, the recorded decisions predicted that sequential evaluation would end 1.0 point lower. The loops it drove ended 4.4 points lower: in a registered comparison of six independent airline loops per arm, run for the same ten rounds, its loops gained +4.2 points held out on average and full re-evaluation’s +8.7, saving 39% of the candidate-evaluation episodes but only 10% of the loop’s cost, which the proposer dominates. The sequential loops accepted fewer changes, and the rule’s stopping probabilities were overconfident where RRSI admits a change for being cheaper or new; but the stopped candidates also reached the proposer with pessimistic scores, and part of the pooled difference reflects wider noise bands in the sequential loops, so we do not attribute the loss to one mechanism.

Our contributions are an evaluation protocol that scores verification decisions by deploying every candidate, accepted or rejected, with an explicit cost account; a registered comparison of independent loops in which a rule non-inferior on a loop’s recorded decisions loses ground in the loops it drives, with a matched offline comparison and a calibration analysis of where the rule errs, including a cost-aware correction tested offline; and the measurements of both studies, released with the environments, banks and evidence. The loop, its regularised edits and its selection rule are RRSI’s (Xia et al., 2026); we change only the evidence its selection step uses.

2 Related Work

Self-improving agents and how they verify. Reflexion, Self-Refine and ExpeL turn feedback into verbal lessons (Shinn et al., 2023; Madaan et al., 2023; Zhao et al., 2024); ProTeGi, OPRO, TextGrad and DSPy rewrite prompts (Pryzant et al., 2023; Yang et al., 2024; Yuksekgonul et al., 2025; Khattab et al., 2024); ADAS, the Darwin Gödel Machine and GEPA search over designs (Hu et al., 2025; Zhang et al., 2025a; Agrawal et al., 2026); ACE evolves a context playbook through incremental updates (Zhang et al., 2026b). Harness-Fix repairs harness code and promotes a bundle of repairs only if it solves more validation tasks while breaking at most two (Chen et al., 2026); Bayesian-Agent applies threshold rules to posteriors over skills (Wu et al., 2026). RRSI regularises harness self-improvement: it bounds and anneals the number of edits per candidate, screens benchmark-specific edits with a critic, prunes stale edits, and selects candidates by full re-evaluation against a calibrated noise floor (Xia et al., 2026). These systems report the success of the final agent; we evaluate the evidence behind each decision, and Study 2 uses RRSI’s code as its loop.

Failure attribution. Who&When and its extension show that locating the responsible agent and step is far from solved (Zhang et al., 2025b; Liu et al., 2026). AgenTracer trains an attribution model on counterfactual replays and injected faults (Zhang et al., 2026a); AgentDebug and ErrorProbe pair taxonomies or dependency tracing with corrective feedback (Zhu et al., 2025; Li et al., 2026); MAST and AdaMAST give attribution a vocabulary (Cemri et al., 2025, 2026).

Verification by intervention. DoVer validates hypotheses by editing a message or plan and re-running (Ma et al., 2025); REFLECT tests patches by targeted replay (Lin et al., 2026); CausalFlow and Causal Agent Replay score step interventions causally (Bonagiri et al., 2026; Shah, 2026); Agent-DebugX gates reruns by policy (Zhu et al., 2026). Their evaluations report whether an intervention repairs the failure it targets, how accurately steps are attributed, or how the final agent performs; we deploy every candidate change, accepted or rejected, on independent tasks to ask how well each kind of evidence predicts that gain and what it costs relative to it. Program repair has long warned that a patch passing the failing test may not fix the fault

(Smith et al., 2015).

Sequential evaluation and racing. Stopping the evaluation of candidates that are unlikely to win is an old idea. Hoeffding races and F-race discard configurations once a bound or a Friedman test separates them from the best (Maron and Moore, 1993; Birattari et al., 2002); successive halving and Hyperband spread a budget over rounds of elimination (Jamieson and Talwalkar, 2016; Li et al., 2018); fast cross-validation drops parameter settings by sequential tests across growing subsamples (Krueger et al., 2015); and clinical trials stop for futility when the predictive probability of a conclusive result is small (Spiegelhalter et al., 1986). Our sequential rule is a futility stop of this kind. Its target differs: not the best candidate or a significant effect, but the admission rule of a self-improvement loop, with a calibrated noise band, a cost budget and a branch for new components, judged by what the admitted changes do held out and by whether loops it drives end as well.

3 Verification Evidence and Deployment Gain

Changes and their deployment gain. A loop holds an incumbent harness h and proposes a change c : a rewritten prompt section or tool description, a code edit, or a set of these. Its *deployment gain* $\Delta p(c)$ is the difference in success rate between h with c and h alone, measured from the first step on tasks independent of everything the loop saw. A verification rule spends C_{verify} episodes on evidence, forms an estimate $\hat{\Delta}$ of the change’s effect and accepts the change if the estimate clears a threshold.

Evidence. We study the evidence that self-improvement loops use. *None* accepts the proposed change. An *LLM judge* reads the change and the failures it targets and predicts its effect, at a cost in tokens but no episodes; a *single-step check* regenerates the failing step with the change and asks a judge whether the error is gone. A *targeted replay* resumes a failed trajectory from the failing step with the change and succeeds with probability $b + (1 - b)q$, where q is the share of such failures the change repairs; a *null replay* does the same without the change and measures the *spurious recovery* rate b , without which replay success cannot be interpreted. A *regression run* is a previously solved task, from the first step, with the change; a *sample*

or *full evaluation* runs candidate and incumbent on a random sample of, or all, training tasks. Replays and checks see only the failures a change targets; only regression runs and evaluations see harm to tasks that already succeed.

Is the evidence worth its cost? Compare a rule E with a reference rule, here applying changes without verification. Suppose E ’s choices change the deployed agent’s success by Δp and its running cost by Δc_{run} per episode, that E spends C_{verify} episodes, and that it changes the one-off cost of reviewing and maintaining the applied changes by ΔC_{change} . Over the next N deployed episodes E is worth

$$\text{Net}(N) = N(v\Delta p - \Delta c_{\text{run}}) - C_{\text{verify}} - \Delta C_{\text{change}}, \quad (1)$$

with every cost in episodes of the unmodified agent’s running cost and v the value of one success in the same unit. We count three things: *episodes*, runs of the agent on a task (a replay from a failing step counts as one); *episode equivalents*, which price replays (and, in Study 1, checks) by their tokens relative to an episode and are the unit of C_{verify} ; and *dollars* at list prices, used only for a whole loop’s cost, which also includes the analyst’s and proposer’s model calls. The one-off costs do not grow with N , so once Nv is large compared with C_{verify} the sign of $\text{Net}(N)$ is the sign of $v\Delta p - \Delta c_{\text{run}}$: evidence pays if the changes it selects are better, or cheaper to run. Its worth then rests on its *quality*, which we measure in two ways: how well its estimates correlate with, and are calibrated against, measured deployment gains, and the *decision value* of a rule that uses it, the mean deployment gain of the changes it accepts. Appendix A splits a patch’s net effect into repair, spurious recovery and regressions.

4 Study 1: One Round of Attribution-Driven Repair

Setup. The agent is DeepSeek-V4-Flash, with the provider’s default sampling, on the retail and airline domains of τ^2 -bench (Yao et al., 2024; Barres et al., 2025), which have an LLM user simulator and database-state rewards; attribution, taxonomy induction (Wan et al., 2024), patch generation and judging use DeepSeek-V4-Pro. Patches rewrite one of 24 (retail) or 21 (airline) components: policy sections, tool descriptions and the step limit. Most training episodes carry one injected fault written as

a plausible authoring mistake; clean episodes contribute natural failures, 22 of 137 on retail and 28 of 112 on airline. For 36 (category, component) cells per domain the analyst writes three patches, and for every patch we record 20 targeted replays, each with a single-step check, and 4 to 20 regression runs on solved training tasks, and for every category 20 null replays. Eleven policies spend a budget of episodes from these *intervention banks* and output a patch set: *Apply attributed* uses no evidence and applies the most-blamed components’ first patches; the others accept patches on replays, on regression gates in the style of HarnessFix (Chen et al., 2026), by uncertainty sampling with or without checks, or by multi-fidelity allocation. Each policy’s budget-10 set is deployed on unseen tasks, 40 retail tasks $\times$ 3 episodes and 20 airline tasks $\times$ 3. Retail was the development domain for every method. Appendix B gives the faults, banks and policies, and Appendix M a third environment.

Results. Applying the attributed patches without verification raised held-out success from 75.8% to 87.5% on retail (+11.7 points, 90% interval [4.5, 18.8]) and from 68.3% to 75.0% on airline (+6.7, [−3.3, 16.6]; Table 3). No verified set showed a reliable improvement over it: the differences range from −4.3 to +0.3 points on retail and from −4.0 to −0.3 on airline, with intervals reaching +6.0 and +8.5, so this is not equivalence. Most of retail’s gain comes from fault-injected episodes (+4 [−3, 10] on 158 clean episodes; Appendix K). The sets’ gains predicted from replays net of spurious recovery differed by only 1.0–1.7 points, below 2.6–3.8 points of deployment noise. Judge checks cost 22–72% of a replay’s tokens with at best similar information per token, 20–23% of unpatched replays succeed anyway, and a patch’s harm on solved tasks is the least measurable quantity (Table 4). With a verification bill of at most 10 episodes, whether verification pays turns on whether it selects better changes (Table 5), which Study 1 could not resolve; Study 2 provides candidates that differ by more and more precise deployments.

5 Study 2: Verification Evidence Inside RRSI

Study 1 patches only text, in one round, mostly on injected faults. Study 2 builds on RRSI (Xia et al., 2026), whose proposer edits harness code over many rounds of natural failures. We run RRSI’s

loop, collect every kind of evidence for every candidate it measures, deploy every candidate, accepted or rejected, and ask what each kind of evidence would have decided. Appendix N describes the port, the registrations and every change made after them.

Loop. We vendor RRSI’s code (Apache 2.0) and change only its model backend, its cost accounting and one rule for empty proposals. Analyst, proposer, critic and judge are DeepSeek-V4-Pro with thinking disabled; the agent and user simulator are DeepSeek-V4-Flash at temperature 0, at which our episode loop matches τ^2 ’s own orchestrator; the starting harness is τ^2 ’s reference agent. Each round the proposer drafts $m = 2$ candidates under an edit budget that anneals from 4 to 1, a critic rejects edits that hard-code task data or touch the grader, and RRSI evaluates the candidates that pass a smoke test twice on every evolve task. It accepts a candidate that beats the incumbent by more than a noise band δ , calibrated from three evaluations of the base harness, or that is within the band and cheaper to run or of an untried kind, and it rejects a harness that raises errors in more than 2% of trials (Appendix N). Evolve tasks are τ^2 ’s training split (74 retail, 30 airline) and held-out tasks its test split (40 and 20). The loop ran $T = 20$ rounds per domain on three independent trajectories, r1–r3; r2 and r3 started after r1’s results were known and use corrected component labels (below).

Evidence and decisions. For each measured candidate we collect the evidence of §3 as estimates of its change ΔS in evolve success (Table 20, Appendix N): an LLM judge’s prediction from the diff and the failures it targets (no episodes); *sample@b*, the paired difference on b evolve episodes; *replay@b*, the failure share times the success rate of b targeted replays of failed evolve trials; *replay−null@b*, which subtracts the success rate of null replays with the incumbent; and *net@b*, which splits b between both kinds of replay and regression runs on solved tasks, with or without a Beta prior on the repair rate. Budgets of 10, 40 and 80 episodes per candidate are subsamples of one collection, four replays with each candidate and four null replays of each failed evolve task (at most 20 per round), plus RRSI’s own full evaluation. A replay re-executes the recorded prefix through the new harness and goes live where it would first act differently. Every rule decides with RRSI’s rule, with ΔS replaced by its estimate and

δ by twice the estimate’s standard error; full evaluation keeps RRSI’s calibrated δ . The rules thus differ in their evidence and in the band it implies, which decides when RRSI’s within-band branch applies; Appendix Q decides every rule with one criterion.

Sequential full evaluation. RRSI evaluates a candidate on N paired episodes, two per evolve task. The sequential rule runs them in batches of 10 in random order. After n episodes with mean paired difference m it stops the candidate if its harness errors already exceed 2% of N , or if the predictive probability that RRSI’s rule will admit it after all N episodes is below $\gamma = 0.05$. That probability treats the final ΔS as normal with mean m and variance $(\frac{N-n}{N})^2 s^2 (\frac{1}{n} + \frac{1}{N-n})$, where s^2 is the larger of the running variance of the paired differences and $S_t(1 - S_t)$; it holds the cost change at its running estimate, knows the novelty count, and adds RRSI’s two ways to admit: above the band within the cost budget, and inside the band with a positive score for cost and novelty. A candidate that is not stopped completes its evaluation and is decided as under full evaluation, and RRSI selects the admitted candidate with the highest evolve success. Stopping thus changes only admission: it can lose an acceptance but never add one, and, as the predictive probability is approximate, γ bounds nothing. A stopped candidate enters RRSI’s history with its early-stopped estimate. *Sequential sampling* caps N at 80 sampled episodes and decides with the sample’s own band.

Ground truth and estimand. Every incumbent and measured candidate is deployed from the first step on the held-out tasks, 240 episodes each (retail 40 tasks $\times$ 6, airline 20 $\times$ 12), and the base and final harnesses on 400 and 240. A candidate’s deployment gain $\Delta p(c | H_t)$ is its held-out success minus that of the incumbent H_t it was proposed against, paired by task. A rule E ’s *decision value* on a set $\mathcal{T}$ of rounds of both domains is $D(E) = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \sum_c P_E(c | t) \Delta p(c | H_t)$, the mean expected deployment gain of its choice per round (zero when it keeps H_t), with P_E taken over 400 subsamples of the evidence. For r1 the registered endpoints are (P1) *transfer*, the held-out gain of the final harness over the base, and (P2) D at $b = 40$ with three Holm-adjusted contrasts (Appendix O).

Further trajectories. Because r1’s values rested on one trajectory per domain and its registered analysis gave RRSI’s error check to full evaluation only (§5.1), we registered r2 and r3 before any of their episodes ran, with the primary analysis of this study: D over their 80 rounds with the error check applied to every rule’s own episodes (the *common error check*), and one-sided non-inferiority tests of sample@40, net@40 and replay–null@40 against full evaluation, margin 0.75 points per round, Holm-adjusted at $\alpha = 0.05$. A bootstrap resamples, within domain and trajectory, blocks of rounds that share an incumbent, and held-out tasks ($B = 2000$). Two sequential rules (Appendix N) form a second family. r2 and r3 use corrected component labels and are otherwise as r1. One more registered trajectory runs the same loop and analysis on AppWorld (Trivedi et al., 2024), whose agent writes Python against simulated apps (E3, Appendix P).

5.1 Results

The loop. Over r1–r3 RRSI drafted 240 candidates and measured 154, 106 of which changed harness code. It accepted 5 candidates on r1, all in the first five rounds, and 21 on r2 and r3, as late as round 19; 19 of the 26 were within the noise band. Candidates’ mean deployment gain was -1.5 , -1.25 and $+0.14$ points on r1–r3, and 63 of 154 were positive.

Transfer (P1). Airline’s evolve gains transferred on r1 and r2, by $+8.75$ [$+2.5, +15.8$] and $+7.9$ [$+2.5, +14.2$] points against $+13.3$ on evolve tasks. On r3 its six acceptances, all within the band, left evolve success unchanged ($+1.7$ [$-2.5, +5.8$] held out). Retail’s evolve gains of 4.1, 4.7 and 7.4 points never transferred: -0.25 [$-3.0, +2.75$], 0.0 [$-2.0, +1.75$] and -4.25 [$-7.5, -1.0$].

First trajectory. On r1, most of the losses of no verification, the judge and replay without null replays come from one candidate (retail, round 6), a check that reads an attribute the harness never sets: the harness raised errors in half of its evolve episodes and held-out success fell by 51.7 points, yet the critic passed it and the judge predicted a gain. In its registered analysis, which gave RRSI’s error check to full evaluation only, full evaluation led with $+0.22$ points per round [$-0.06, +0.62$] and net@40 was not shown non-inferior (Holm $p = 0.17$; Appendix O). With the common error

check, accepting whatever passes the check still loses 0.58 points per round.

Primary analysis (r2 and r3). On their 80 rounds, full evaluation’s decision value was +0.04 points per round $[-0.35, +0.43]$ at 141 episodes (Table 19, Appendix N). Sample@40 and net@40, at 54 and 49 episodes, were non-inferior to it (Holm $p = 0.045$ each); replay–null@40 was not ($p = 0.14$), and its own value, $-0.39 [-0.79, -0.03]$, lies below zero, 40% of it from one candidate (r3 retail, round 11) that held every write whose arguments it could not trace to earlier tool results. It raised no harness errors but blocked legitimate writes, lowering held-out success by 12.5 points; full evaluation, sample and net, which run solved tasks, rejected it, while replay–null and the judge accepted it. No rule was shown to beat keeping the incumbent, and because full evaluation gained so little, keeping the incumbent is itself non-inferior to it at the registered margin ($p = 0.004$; Table 23). These tests show that the rules lose little against a rule that gained little, not that they preserve a useful improvement process.

Sequential evaluation on the recorded rounds. Sequential full evaluation was non-inferior to full evaluation, $-0.02 [-0.12, +0.07]$ at 81 episodes per round (57%), as was sequential sampling, $+0.03 [-0.26, +0.35]$ at 53 (both $p < 0.001$). Post hoc, sequential full evaluation stays non-inferior at a margin of 0.25 points ($p < 0.001$), which keeping the incumbent fails, and when any one trajectory and domain is left out (Table 34). It made full evaluation’s choice in 93% of rounds but only 74% [67, 81] of its 21 acceptances; those it would drop gained no less held out ($+0.30$ against $+0.11$ points), so it lost little only because full evaluation’s acceptances were worth little (Table 24). On r1, where it was designed, it gained $+0.15$ at 65 episodes per round against full evaluation’s $+0.22$ at 123.

Is the stopping rule calibrated? No (post hoc; Table 30, Appendix R). Candidates stopped at a mean predicted probability of 0.02 would have been admitted by full evaluation in 8% of cases on r2+r3, 3% on AppWorld and 13% [6, 20] in the six live loops below, so γ does not bound the acceptances lost. Holding the cost change at its running estimate causes this: with the complete evaluation’s cost change in the same formula, looks below γ were followed by admission in at most 2% of cases.

The acceptances lost are those RRSI makes inside its noise band because a candidate is cheaper to run or adds a new component, where admission turns on the sign of a small cost change: the rule stopped about a third of these, in recorded and live rounds alike, against 4% of the candidates admitted above the band. Lowering γ from 0.5 to 0.005 raises the share of full evaluation’s acceptances it keeps on r2+r3 from 26% to 86%, and its episodes from 22% to 74% of full evaluation’s (Table 32). At its episodes, fixed sampling kept as many acceptances (75%) but also accepted in a fifth of the rounds in which full evaluation kept the incumbent, and fully evaluating one random candidate kept 61% at 69% of the episodes; on AppWorld sequential evaluation kept 88%, fixed sampling at most 64% (Table 33). A correction that can be run as is, integrating the predictive probability over the uncertainty of the final cost change instead of holding it fixed, halves the stops inside the band: it keeps 88% of full evaluation’s acceptances on r2+r3 at 64% of the episodes, and 89% instead of 69% on airline at 77% (post hoc, Table 29). It has not been run in a loop.

Independent loops. The analyses above score decisions in the states full evaluation’s loop visited, pooled over two domains and twenty rounds. To compare loops, we registered two airline loops driven by sequential full evaluation (cl1, cl2), then eight more run concurrently from one commit, four with each selection step (Appendix S). With r2 and r3, six loops per arm ran rounds 0–9; their base and final harnesses were deployed on the 20 test tasks, 12 trials each. Transfer after ten rounds averaged $+4.2$ points with sequential and $+8.7$ with full evaluation, a difference of $-4.4 [-9.4, -0.1]$ (bootstrap over loops and tasks; Table 1); non-inferiority at 5 points was not shown ($p = 0.38$). The difference comes from the concurrent loops, $-6.9 [-11.7, -2.6]$; the four historical loops, whose outcomes were known when the comparison was registered, are level. On the same domain and rounds the recorded decisions predict a smaller loss: summed over airline rounds 0–9 of r1–r3, full evaluation’s round values equal those trajectories’ transfer, and sequential evaluation’s are 1.0 point lower $[-3.7, +0.8]$. The loops lost 3.4 points more $[-8.7, +1.3]$, a gap the data do not resolve. Post hoc, the sequential loops’ noise bands, calibrated before round 0, were wider by chance (9.8 against 6.7 points); a wider band leaves more can-

	Loops	Sequential	Full	Difference [90%]	δ -adjusted [90%]	Accepted	Mean δ
Recorded rounds 0–9, offline	3	+5.8	+6.8	−1.0 [−3.7, +0.8]	—	—	10.9
Live, all (registered)	6+6	+4.2	+8.7	−4.4 [−9.4, −0.1]	−2.1 [−6.2, +1.9]	14/23	9.8/6.7
historical	2+2	+6.2	+5.8	+0.4	—	6/7	9.8/7.4
concurrent	4+4	+3.2	+10.1	−6.9 [−11.7, −2.6]	−5.8 [−9.4, −2.2]	8/16	9.7/6.4

Table 1: Transfer after ten rounds on airline (points; held-out success after round 9 minus the base). Recorded: full and sequential evaluation’s round values summed over rounds 0–9 of r1–r3, in full evaluation’s states. Live: the registered comparison (sequential: c11, c12, s1–s4; full: r2, r3, f1–f4); historical loops ran before the eight concurrent ones. 90% intervals: bootstrap of rounds or loops and of tasks; adjusted for the calibrated noise band δ by least squares (post hoc, t intervals). Accepted changes and mean δ (points), sequential/full.

didates to RRSI’s in-band branch, and transfer fell with the band in both arms. Adjusting for it halves the pooled difference, to $-2.1 [-6.2, +1.9]$, but leaves the concurrent one at $-5.8 [-9.4, -2.2]$. The comparison is one counterexample, in one domain, model family and stopping rule, not an estimate of how often recorded decisions mislead.

What changed in the loops. The sequential loops accepted 14 changes and the full-evaluation loops 23. Completing the stopped evaluations, full evaluation would have accepted in 20 of the sequential loops’ 60 rounds, 14 of which the loops did (70%, as offline), so 6 of the 9 missing acceptances were stopped and 3 reflect fewer rounds with an admissible candidate. Scored against their own incumbents, the candidates of both arms were alike (27% and 32% admissible), but the proposer saw the 62 stopped ones as worse than they were: their early-stopped estimates averaged -4.7 points, their complete evaluations -2.5 . Lost acceptances that compound, a history shaped by pessimistic scores and the band imbalance can each contribute; the shadow evaluations do not separate them, and no loop restored a lost acceptance (Appendix S).

Cost. What sequential evaluation saves is candidate-evaluation episodes, not the loop’s cost. Over ten rounds its loops ran 538 such episodes against 880 (-39%), but cost \$3.19 against \$3.53 at list prices ($-\$0.35 [-0.57, -0.10]$, -10%), because the proposal side’s model calls dominate. Both arms ran ten rounds: at equal dollars the sequential arm could have run about one more, which we did not test. On the recorded trajectories it would have cut the loop’s cost by 19% on τ^2 , 17% on AppWorld and 1% with a thinking proposer (Table 28). Per round, full evaluation cost 123 (r1) and 141 (r2, r3) episodes and the @40 rules about a third of that in episode equivalents; on r2 and r3, Net(10^4) at $v = 10$ against keeping the incumbent,

in episodes of running cost, was -108 for full evaluation and positive only for sequential sampling, $+10 [-315, +350]$.

All three trajectories. Pooled over 120 rounds, full evaluation gained $+0.10 [-0.18, +0.41]$, with the same non-inferiority results (Holm $p \leq 0.018$; replay–null@40 $p = 0.058$). Full evaluation’s value was $+0.22$, $+0.16$ and -0.07 on r1–r3, and accepting every candidate’s -1.87 , -1.08 and $+0.14$: trajectories differed more than rules.

A stronger proposer (r4). With thinking enabled, one airline trajectory of ten rounds measured six candidates, all helpful (descriptive, Appendix N); full evaluation gained $+0.92$ points per round, and sequential full evaluation made its choice in 95% of rounds.

A code agent (E3). On AppWorld the 29 measured candidates barely changed held-out success (mean -0.2 points), and the final harness’s evolve gain of 4.2 points did not transfer ($0.0 [-2.1, +2.1]$). Full evaluation’s value was $+0.03$ points per round $[-0.30, +0.71]$ at 174 episodes (Table 22). The @40 rules missed non-inferiority after Holm adjustment (Holm $p = 0.053$); sequential full evaluation was non-inferior at 62% of its episodes ($p < 0.001$) and made full evaluation’s choice in 98% of rounds.

6 Discussion

What the evidence shows. Our results identify failure modes of verification, not a ranking of evidence by predictive quality. Most changes did not help held out, so verification earned its place by rejecting harmful ones, and only evidence that runs the changed agent on solved tasks rejected both kinds we saw.

Recorded decisions are not loops. Sequential evaluation passed every offline test we set, and

on the matched airline rounds the recorded decisions predicted a loss of one point; its loops lost four, seven among those run concurrently. Scoring each decision once, in the states full evaluation chose, ignores two things a loop does with a decision: an accepted change becomes the base of every later round, and every evaluated candidate, stopped or not, enters the history the proposer reads. Our data cannot apportion the loss between them, and calibration noise matters as well: each loop’s noise band, set from three base evaluations, ranged from 2.7 to 12.5 points across the twelve loops and moved transfer in both arms. A cheaper rule should be judged by the loops it drives, with the whole loop’s cost, and loops compared with each other should share or balance their calibration. With the between-loop standard deviation we saw, about 4.8 points of transfer, showing non-inferiority at a margin of 5 points with 80% power needs about 12 loops per arm, and at 3 points about 31.

Resolution limited these studies more than price.

A round of full evaluation costs about 25 cents of τ^2 compute, but comparing evidence needs per-round differences of about 0.7 points or candidate gains above 5.3–5.9 points, which 4–11% of candidates reached, and the evidence correlated with deployment gain at most 0.33 (Appendix O). The independent evidence is small: the primary analysis rests on 23 incumbent blocks and held-out sets of 40 and 20 tasks (Table 35), so more trajectories and tasks would help more than more trials.

Implications for self-improvement loops.

Loops that report only the final agent’s held-out success (Chen et al., 2026; Zhang et al., 2026b; Xia et al., 2026) measure all their decisions at once; on retail and AppWorld, evolve gains of 4–7 points never transferred. Replays need null replays and regression runs. A stopping rule must be calibrated on each branch of the acceptance rule it predicts, here the cost condition, whose uncertainty it should model, and tested in the loops it drives.

7 Conclusion

Inside RRSI’s loop most changes did not help held out, and evidence that never runs the changed agent on solved tasks accepted harmful ones. Sequential evaluation reproduced most of full re-evaluation’s decisions on recorded rounds at about half its episodes, and on the loops’ domain and horizon the

recorded decisions predicted a loss of one point; loops it drove for ten rounds gained 4.4 points less held out, for a 10% saving. Why they lost more, through lost acceptances, the proposer’s history or calibration noise, remains open, as does which evidence predicts deployment gain. Cheaper verification should be judged by the loops it drives.

Limitations

Both studies use one model family, with DeepSeek-V4-Flash as agent and DeepSeek-V4-Pro as analyst, proposer and judge, and two customer-service domains of τ^2 -bench; Study 1 adds ShopDesk and Study 2 one AppWorld trajectory. Study 1’s patches rewrite only policy sections, tool descriptions and a step limit; it runs a single round, mostly on injected faults (natural failures are 16–25% of training failures), and deploys patch sets only at budget 10, on 120 (retail) and 60 (airline) held-out episodes, where most of retail’s gain comes from fault-injected episodes. Its bank analyses rest on a harm model fitted to at most 20 regression runs per patch, and its edge priors use an analyst accuracy of 0.7 unless stated. Study 1’s agent used the provider’s default sampling and Study 2’s uses temperature 0. Study 2’s proposer is likely weaker than the model RRSI was developed with; a fourth trajectory with a thinking proposer was cut short because its drafts took over an hour and mostly ended without a proposal; on airline all six of its measured candidates helped, which suggests that a stronger proposer leaves less for verification to decide, but rests on one trajectory (Appendix N). Its held-out sets are τ^2 ’s test splits of 40 and 20 tasks, on which most candidates’ gains were not resolved. Its decision values are conditional on RRSI’s trajectories: they score each rule’s decisions in the states RRSI visited, and the independent loops show that this can mislead. Those loops are six per arm, on one domain and for ten rounds, compared at equal rounds rather than equal cost; the arms mix loops run at different times (cl1, cl2, r2 and r3, whose outcomes were known when the comparison was registered, and the eight concurrent loops), each loop calibrated its own noise band, and the bands came out wider in the sequential arm. They show that such a loss can occur, not how often. Their shadow evaluation scores full evaluation’s decision in the states the sequential loop visited, not the states full evaluation would have reached, and no experiment separates lost acceptances from the proposer’s

history. The cost-aware stopping rule was tested only offline. $\text{Net}(N)$ sets the change cost to zero; charging each accepted change a fixed review cost favours rules that accept less and leaves the order of the sequential rules and full evaluation unchanged (Appendix Q). Three trajectories per domain show that between-trajectory variation is as large as the differences between rules, and the pooled analysis treats trajectories as fixed. The non-inferiority margin of 0.75 points per round was set before any result, but it is large relative to the decision values we observed; the smaller margins, the agreement rates, their split by full evaluation’s decision, the decision-rule ablation, the coverage analyses, the calibration, threshold, matched-cost and leave-one-out analyses, the matched offline comparison, the noise-band adjustment, the accounting of the loops and the cost-aware stopping rule are post hoc. On r1, the registered analysis gave RRSI’s error check to full evaluation only, so the comparison with a common check is post hoc there; its held-out count was raised after six rounds, when some held-out episodes had run and a few results had been seen (Appendix N), and the registered count gives the same conclusions. While r2 and r3 ran, a few of their held-out success rates appeared in progress logs before analysis; the analysis code had been fixed and tested on r1 only. The sequential rules were designed on r1’s rounds and confirmed on r2 and r3. r1’s component labels did not recognise our harness’s files, which may have changed what its proposer was told to explore; r2 and r3 use corrected labels, so the trajectories differ in that respect as well as in sampling. Token prices for the models we used change; we report costs in episodes, and weighting tokens by list prices changes no ranking.

Ethics Statement

Self-improving agents change their own behaviour without a person reviewing each change. In Study 2 most proposed changes did not raise held-out success; one that crashed the harness in half of its episodes, and another that blocked legitimate writes, passed both an LLM critic and an LLM judge. That Study 1 detected no gain from verification does not show that verification is unnecessary; our measurements cannot exclude small harms per patch and should not be read as licence to skip review. On ShopDesk a single patch silently broke a quarter of tasks (Appendix M). Our environments

use synthetic customer data only.

References

- Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations*.
- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. τ^2 -bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*.
- Mauro Birattari, Thomas Stützle, Luís Paquete, and Klaus Varrentrapp. 2002. A racing algorithm for configuring metaheuristics. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO 2002)*, pages 11–18.
- Akash Bonagiri, Devang Borkar, Gerard Janno Anderias, Setareh Rafatirad, and Houman Homayoun. 2026. CausalFlow: Causal attribution and counterfactual repair for LLM agent failures. *arXiv preprint arXiv:2605.25338*.
- Mert Cemri, Andrei Cojocaru, Melissa Pan, Shu Liu, Shubham Agarwal, Alexander Krentsel, Jay Tang, Kannan Ramchandran, Joseph E. Gonzalez, Matei Zaharia, Alexandros G. Dimakis, and Ion Stoica. 2026. Fantastic adaptive taxonomies and how to use them. *arXiv preprint arXiv:2607.16387*.
- Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*.
- Kathryn Chaloner and Isabella Verdinelli. 1995. Bayesian experimental design: A review. *Statistical Science*, 10(3):273–304.
- Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. 2026. From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws. *arXiv preprint arXiv:2606.06324*.
- Peter I. Frazier, Warren B. Powell, and Savas Dayanik. 2008. A knowledge-gradient policy for sequential information collection. *SIAM Journal on Control and Optimization*, 47(5):2410–2439.
- Shengran Hu, Cong Lu, and Jeff Clune. 2025. Automated design of agentic systems. In *International Conference on Learning Representations*.

Kevin Jamieson and Ameet Talwalkar. 2016. Non-stochastic best arm identification and hyperparameter optimization. In <i>Proceedings of the 19th International Conference on Artificial Intelligence and Statistics</i> , volume 51 of <i>PMLR</i> , pages 240–248.	prompt optimization with “gradient descent” and beam search. In <i>Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing</i> .	918 919 920 921
Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling declarative language model calls into self-improving pipelines. In <i>International Conference on Learning Representations</i> .	Jaineet Shah. 2026. Causal agent replay: Counterfactual attribution for LLM-agent failures. <i>arXiv preprint arXiv:2606.08275</i> .	922 923 924
Tammo Krueger, Danny Panknin, and Mikio Braun. 2015. Fast cross-validation via sequential testing. <i>Journal of Machine Learning Research</i> , 16(33):1103–1155.	Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In <i>Advances in Neural Information Processing Systems</i> .	925 926 927 928 929
Jiazheng Li, Emine Yilmaz, Bei Chen, and Dieu-Thu Le. 2026. Towards self-improving error diagnosis in multi-agent systems. <i>arXiv preprint arXiv:2604.17658</i> .	Edward K. Smith, Earl T. Barr, Claire Le Goues, and Yuriy Brun. 2015. Is the cure worse than the disease? overfitting in automated program repair. In <i>Proceedings of the 10th Joint Meeting on Foundations of Software Engineering</i> .	930 931 932 933 934
Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Roshtamizadeh, and Ameet Talwalkar. 2018. Hyperband: A novel bandit-based approach to hyperparameter optimization. <i>Journal of Machine Learning Research</i> , 18(185):1–52.	David J. Spiegelhalter, Laurence S. Freedman, and Patrick R. Blackburn. 1986. Monitoring clinical trials: Conditional or predictive power? <i>Controlled Clinical Trials</i> , 7(1):8–17.	935 936 937 938
Xiaofeng Lin, Yingxu Wang, Tung Sum Thomas Kwok, Daniel Guo, Sahil Arun Nale, Charles Fleming, and Guang Cheng. 2026. REFLECT: Intervention-supported error attribution for silent failures in LLM agent traces. <i>arXiv preprint arXiv:2606.09071</i> .	Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In <i>Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics</i> .	939 940 941 942 943 944 945
Jiale Liu, Huajun Xi, Shaokun Zhang, Yifan Zeng, Tianwei Yue, Chi Wang, Jian Kang, Qingyun Wu, and Huazheng Wang. 2026. Who&when pro: Can LLMs really attribute failures in AI agents? <i>arXiv preprint arXiv:2607.09996</i> .	Rakshith Vasudev, Melisa Russak, Dan Bikel, and Waseem Alshikh. 2026. Accurate failure prediction in agents does not imply effective failure prevention. <i>arXiv preprint arXiv:2602.03338</i> .	946 947 948 949
Ming Ma, Jue Zhang, Fangkai Yang, Yu Kang, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2025. DoVer: Intervention-driven auto debugging for LLM multi-agent systems. <i>arXiv preprint arXiv:2512.06749</i> .	Abraham Wald. 1945. Sequential tests of statistical hypotheses. <i>The Annals of Mathematical Statistics</i> , 16(2):117–186.	950 951 952
Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback. In <i>Advances in Neural Information Processing Systems</i> .	Mengting Wan, Tara Safavi, Sujay Kumar Jauhar, Yujin Kim, Scott Counts, Jennifer Neville, Siddharth Suri, Chirag Shah, Ryen W. White, Longqi Yang, Reid Andersen, Georg Buscher, Dhruv Joshi, and Nagu Rangan. 2024. TnT-LLM: Text mining at scale with large language models. In <i>Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining</i> .	953 954 955 956 957 958 959 960
Oded Maron and Andrew W. Moore. 1993. Hoeffding races: Accelerating model selection search for classification and function approximation. In <i>Advances in Neural Information Processing Systems 6</i> , pages 59–66.	Xiaojun Wu, Cehao Yang, Honghao Liu, Xueyuan Lin, Wenjie Zhang, Zhichao Shi, Xuhui Jiang, Chengjin Xu, Jia Li, and Jian Guo. 2026. Bayesian-Agent: Posterior-guided skill evolution across LLM agent harnesses. <i>arXiv preprint arXiv:2606.08348</i> .	961 962 963 964 965
Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chengguang Zhu, and Michael Zeng. 2023. Automatic	Peng Xia, Rujun Han, Zifeng Wang, Yanfei Chen, Yufan Zhuang, Yoonho Lee, Chengsong Huang, Han Yu, Zhongying CuiZhu, Yifei Ming, Huaxiu Yao, Burak Gokturk, Tomas Pfister, and Chen-Yu Lee. 2026. RRSI: Regularized recursive self-improvement of agent harnesses. <i>arXiv preprint arXiv:2609.24972</i> .	966 967 968 969 970 971

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large language models as optimizers. In *International Conference on Learning Representations*.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. Optimizing generative AI by back-propagating language model feedback. *Nature*, 639.

Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. 2026a. AgentTracer: Who is inducing failure in the LLM agentic systems? In *International Conference on Learning Representations*. ArXiv:2509.03312.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. 2025a. Darwin Gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2026b. Agentic context engineering: Evolving contexts for self-improving language models. In *International Conference on Learning Representations*.

Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. 2025b. Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. In *Proceedings of the 42nd International Conference on Machine Learning*. ArXiv:2505.00212.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, Jiaxun Zhang, Pengrui Han, Qipeng Xie, Fuyang Cui, Weijia Zhang, Xiaoteng Ma, Xiaodong Yu, Gowtham Ramesh, Jialian Wu, Zicheng Liu, Pan Lu, James Zou, and Jiaxuan You. 2025. Where LLM agents fail and how they can learn from failures. *arXiv preprint arXiv:2509.25370*.

Kunlun Zhu, Xuyan Ye, Zhiguang Han, Yuchen Zhao, Bingxuan Li, Weijia Zhang, Muxin Tian, Xiangru Tang, Pan Lu, James Zou, Jiaxuan You, and Heng Ji. 2026. AgentDebugX: An open-source toolkit for failure observability, attribution, and recovery in LLM agents. *arXiv preprint arXiv:2607.18754*.

A An Accounting Framework for Single-Round Repair

Setting. An agent is assembled from *components* j (prompt sections, tool descriptions, configuration). On a task distribution it fails with rate F ; an LLM groups its failures into categories i with shares f_i . For each *cell* (i, j) that the analyst blames, it proposes K candidate patches k that rewrite component j . Let $e_{ij} \in \{0, 1\}$ indicate whether changing j can repair category i at all, and q_k the fraction of category- i failures that patch k repairs if so. Separating e from q matters because a patch that fails to help may be aimed at the wrong component or merely be a bad patch. We call a cell with $e_{ij} = 1$ an *effective edge*. This is an interventional notion: some change to j repairs failures of category i . It need not mean that j was the original cause of those failures.

Net effect. Applying patch k of cell (i, j) changes the success rate by

$$\Delta_k = F f_i e_{ij} q_k - (1 - F) \rho_k - c_a - c_w (1 - e_{ij}), \quad (2)$$

where ρ_k is the patch’s absolute excess failure rate on tasks that used to succeed (regressions, which may hit any category): such a task fails a re-run with probability r_0 unpatched and $r_0 + \rho_k$ patched, so $\rho_k < 0$ is possible; $c_a \geq 0$ is a cost charged for every applied change (review, maintenance) and $c_w \geq 0$ a penalty charged only for a change aimed at a wrong component (risk not captured by ρ_k). Costs are fractions of the failure mass F . The goal is to choose a set of patches, at most one per cell, that maximises $\sum_k \Delta_k$ on held-out tasks; effects of patches to different components are treated as additive, and held-out success of whole patch sets (Table 3) and a leave-one-out ablation (Appendix K) are the only checks. A targeted replay of a category- i failure with patch k succeeds with probability $b_i + (1 - b_i) e_{ij} q_k$, a null replay with probability b_i , and a regression run fails with probability $r_0 + \rho_k$.

Value of verification. Before any evidence, the net effects $\Delta = (\Delta_k)$ of the candidate patches are uncertain, with a prior that reflects the analyst’s accuracy and what is known about patch quality and regressions. The *default* applies, in each cell, the patch with the largest $\mathbb{E}[\Delta_k]$ if that is positive. Verification observes outcomes Y and then chooses; its value is $V(Y) = \mathbb{E}[\max_S \mathbb{E}[\sum_{k \in S} \Delta_k \mid Y]] - \max_S \mathbb{E}[\sum_{k \in S} \Delta_k]$, before the cost of the

experiments. Write $x^+ = \max(x, 0)$ and $x^- = \max(-x, 0)$.

Proposition 1 (Value of verification). *For any experiments Y , $V(Y)$ is at most the value of learning Δ exactly, which is the sum over cells of*

$$V_{ij} = \mathbb{E}[\max_k \Delta_k^+] - \max_k (\mathbb{E}\Delta_k)^+. \quad (3)$$

With k^* the default patch of the cell,

$$V_{ij} \leq \underbrace{\mathbb{E}[\max_k \Delta_k^+ - \Delta_{k^*}^+]}_{\text{choosing among patches}} + \underbrace{\mathbb{E}[\Delta_{k^*}^-]}_{\text{avoiding harm}}, \quad (4)$$

with equality when the default applies k^* , and $\mathbb{E}[\Delta_{k^*}^-] \leq (1 - F)\mathbb{E}[\rho_{k^*}^+] + c_a + c_w P(e_{ij}=0)$.

Corollary 1 (Edge-only verification). *Suppose a cell has a single candidate patch whose net effect depends only on e_{ij} : $\Delta = G$ if $e_{ij} = 1$ and $\Delta = -H$ if $e_{ij} = 0$, with $G, H \geq 0$ known. With $\pi = P(e_{ij}=1)$, $V_{ij} = \min(\pi G, (1 - \pi)H)$.*

What survives deployment constraints. Deployment may use a heuristic default D , applies at most one patch per component, which couples cells, combines repairs of one category as $1 - \prod_k (1 - q_k)$ (overstated by at most $f_i \sum_{k < l} q_k q_l$ when added), and cannot push the failure rate on solved tasks below zero ($-\sum_k \rho_k \leq r_0$). Three statements survive. (i) The first sentence of the proposition holds for any set of admissible actions, so $V(Y)$ is at most the value of perfect information over deployable sets. (ii) Over a heuristic default that value is $V + G_D$, where $G_D = \max_S \mathbb{E}[\sum_{k \in S} \Delta_k] - \mathbb{E}[\sum_{k \in D} \Delta_k] \geq 0$ is the decision-rule gap that better use of the prior alone would close. (iii) The value of dropping harmful default patches is at most their expected harm, $\sum_{k \in D} \mathbb{E}[(1 - F)\rho_k^+ + c_a + c_w(1 - e_{ij})]$, because dropping a patch from an overlapping set loses at most its marginal repair. What does not survive is the separation over cells: with one patch per component, the choice and discovery terms have no closed form, and we compute them in sequence (harm, then choice on the default's components, then any component) with a greedy optimiser, which gives a lower bound on the constrained value of perfect information under a given harm model. The three sources of value are *avoiding harm*, which needs regressions or change costs because replay success alone cannot create it; *choosing among patches* of different quality; and *discovery* of edges the analyst missed, the whole term $\mathbb{E}[\max_k \Delta_k^+]$ in cells

	Default rule	Value	Gap	Harm	Choice	Discov.	Total
Retail	Apply, $c = 0$	1.43	0.06	0.41	0.57	0.20	1.18
	Bayes, $c = 0$	1.49	–	0.80	0.32	0.00	1.12
	Apply, $c_w = 0.02$	1.10	–0.19	0.54	0.56	0.03	1.13
	normal, partial pooling	0.77		0.87	0.98	0.17	2.02
	normal, $\tau=0$	1.12		0.23	0.22	0.42	0.87
	no pooling	–1.82		3.30	1.21	0.13	4.65
	tasks resampled	–0.52		2.28	0.90	0.13	3.31
	per cell	0.64		1.39	0.60	0.16	2.15
Airline	Apply, $c = 0$	0.66	–0.13	0.20	0.36	0.29	0.86
	Bayes, $c = 0$	0.53	–	0.55	0.44	0.00	0.98
	Apply, $c_w = 0.02$	0.54	0.02	0.27	0.36	0.11	0.74
	normal, partial pooling	0.73		0.04	0.25	0.40	0.69
	normal, $\tau=0$	0.73		0.04	0.25	0.40	0.69
	no pooling	0.39		0.91	0.10	0.15	1.15
	tasks resampled	0.63		0.39	0.25	0.23	0.87
	per cell	0.86		0.15	0.11	0.30	0.56

Table 2: Value of perfect verification over two no-evidence defaults, deployable sets, posterior means under the task-level binomial harm model (Appendix J) as fractions of the repair-only oracle. *Gap*: Bayes default minus Apply attributed (G_D with the sign reversed; negative when the heuristic does better). Harm, choice and discovery are added in that order: dropping default patches with negative net effect, then the best cell and patch on the default's components, then any other component. Indented rows give the value over Apply at $c = 0$ under the normal-approximation harm models of an earlier analysis (partial, complete and no pooling, tasks resampled before fitting) and with the additive per-cell form of Proposition 1 instead of deployable sets. A value of 1 is 0.070 (retail) or 0.114 (airline) in absolute success.

the default leaves alone. The corollary fails when quality or regressions are uncertain given the edge ($\pi = 1$ with $\Delta = +0.10$ or -0.05 equally likely gives $V = 0.025$).

Proposition 2 (Cost of evidence). *Under Wald's approximations, a sequential test that decides e_{ij} from targeted replays with error δ needs on average about $\log \frac{1-\delta}{\delta} / \text{KL}(b + (1-b)q \parallel b)$ replays when the edge is real. Detecting a regression of size ρ against a base rate r_0 with a one-sided level- α test and power $1 - \beta$ needs about $(z_\alpha + z_\beta)^2 r_0 (1 - r_0) / \rho^2$ regression runs.*

Two consequences follow. First, targeted replays are uninformative about the harm term, because they only see the target category: a patch can pass every replay and still break other tasks. Second, small regressions are expensive to see. With $r_0 = 0.1$, a regression of $\rho = 0.25$ is detected with 90% power by 20 runs, whereas $\rho = 0.03$ needs about 850. Proposition 1 is the standard value-of-information argument (Chaloner and Verdinelli, 1995); what is new is its decomposition for patches and its evaluation on measured banks.

Where could verification add value? Table 2 evaluates Proposition 1 on deployable sets. Every value in it is conditional on a model of the harm ρ_k that the regression runs cannot pin down per patch, so we read only its qualitative pattern. *Defaults.* The Bayes default, built from the net-effect method’s priors, scores 1.49 oracles on retail against 1.43 for applying the attributed patches, and 0.53 against 0.66 on airline; $G_D \geq 0$ holds only under the prior, so the negative realised gap on airline means the edge prior is too generous there. *Value.* Under the binomial model, perfect information is worth 1.18 oracles over the heuristic on retail (90% interval $[0.10, 2.99]$, about 8 points of absolute success) and 0.86 on airline ($[0.24, 1.55]$, about 10 points), split into 0.41, 0.57 and 0.20 (retail) and 0.20, 0.36 and 0.29 (airline) for harm, choice and discovery. Under the earlier normal models the retail value ranges from 0.87 (complete pooling) to 4.65 (no pooling) and airline’s from 0.69 to 1.15; dropping the support constraint, so that the uncertain benefits of every unblamed patch add up without limit, gives 9.5 on retail (Table 14). Only coarse features are stable: all three terms contribute, and most of the value needs evidence about patches the default applies (harm and choice) rather than discovery of missed edges. Proposition 1 names the quantities that could make verification pay, but it cannot decide whether to verify a new diagnosis: its most influential input, each patch’s harm, takes hundreds of regression runs to measure (Proposition 2), and its values move several-fold with the harm model.

B Study 1: Measurements and Net Value

Setup details. Each training episode runs clean or with one injected fault (retail: a policy section allowing gift-card refunds, a cancellation rule without the reason check, a product-tool description that confuses item and product ids, a tight step limit; airline: a wrong bag allowance, a 7-day instead of 24-hour cancellation window, a missing basic-economy restriction, proactive compensation, a tight step limit). The 36 cells per domain are the blamed cells in order of blame plus random unblamed ones; regression runs were extended from 4 to 20 for the 121 patches that any policy accepted, and an unpatched baseline of 10 runs on each of the 111 regression tasks gives task-matched estimates of harm. Every experiment is priced in measured tokens relative to a full replay. *Apply attributed ap-*

plies, for each category, the first patch of the most-blamed component and of any component with at least 40% of the blame; *Replay-each* accepts a hypothesis on two successes in three replays, as do-then-verify methods do (Ma et al., 2025); two HarnessFix-style gates require regression runs to break nothing, per patch or on the whole bundle; uncertainty sampling runs with and without cheap checks; four variants of CARVE (Appendix U) and a net-effect allocation that charges regressions complete the list (Appendix D). Deployed sets come from five seeds, reduced to one patch per component; the retail held-out episodes are 38 clean and 82 with an injected fault, the airline ones 25 clean, and five policies were deployed on airline. On clean episodes an exploratory comparison puts the bundle gate 4 points ahead, from seed sets identical to the unverified one; on retail’s 38 clean episodes the unverified set gains +5.3 points $[-5.1, 15.6]$, and on 158 clean episodes, 120 of them new, +4 $[-3, 10]$ (Appendix K). Predicted from their replays net of spurious recovery, the deployed sets’ gains correlate weakly with the observed ones ($r = 0.23$ over 47 retail sets, -0.49 over 14 airline sets).

Tables 4 and 5 give the measurements and the $\text{Net}(N)$ values discussed in §4.

Does the evidence predict deployment? Replays measure repair from the failing step, whereas deployed patches act from the first step and on every task. For every distinct deployed set we predict its gain over the unpatched agent from its patches’ replays as $\hat{F} \sum_i f_i [1 - \prod_k (1 - \hat{q}_k)]$, with $\hat{F}$ the held-out failure rate, f_i the category shares and $\hat{q}_k = (p_k - b_i)/(1 - b_i)$ the replay success p_k net of the category’s spurious recovery b_i . Figure 1 compares the predictions with the observed gains. They correlate weakly, at $r = 0.23$ over 47 distinct retail sets and -0.49 over 14 airline sets; predictions from the judge checks, calibrated to replay success, do no better ($r = 0.14$ and -0.05).

The ranking test has little power, for a reason that matters beyond this study. Replay predictions differ between sets by a standard deviation of 1.0 point on retail and 1.7 on airline, whereas a set’s held-out success measured on 120 (60) episodes has a run-to-run standard error of 2.6 (3.8) points; the observed gains spread by 2.8 (3.7), no more than that noise. A perfect predictor of differences this small would be indistinguishable from a useless one. Testing whether evidence predicts deployment needs candidates that differ by more than the

Patch set (budget 10)	Retail (40 tasks $\times$ 3)					Airline (20 tasks $\times$ 3)				
	All	Clean	–Apply [90%]	#p	C	All	Clean	–Apply [90%]	#p	C
Unpatched	75.8	81.6		0.0		68.3	76.0		0.0	
Apply attributed	87.5	86.8	–	7.0	0	75.0	76.0	–	5.0	0
Replay-each	83.5	86.8	–4.0 [–9.8, 1.8]	6.0	10					
HarnessFix-style, per patch	83.2	90.0	–4.3 [–12.8, 4.2]	0.8	9	71.0	76.8	–4.0 [–11.9, 3.9]	1.0	8
HarnessFix-style, bundle	84.8	86.8	–2.7 [–5.0, –0.4]	5.6	6	71.7	76.8	–3.3 [–9.3, 2.6]	3.0	6
Uncertainty sampling	85.8	86.3	–1.7 [–6.9, 3.5]	5.8	10					
Uncertainty + checks	84.7	82.1	–2.8 [–8.4, 2.7]	5.8	10					
CARVE, replay only	87.8	86.8	0.3 [–5.3, 6.0]	6.0	10					
CARVE	83.8	80.5	–3.7 [–9.9, 2.6]	5.2	10	74.3	72.0	–0.7 [–8.8, 7.5]	4.2	10
CARVE, measured check	84.8	86.3	–2.7 [–7.6, 2.3]	6.0	10					
CARVE, robust check	86.3	83.7	–1.2 [–6.3, 4.0]	6.0	9					
Net-effect	83.2	83.2	–4.3 [–9.9, 1.3]	3.4	10	74.7	76.0	–0.3 [–9.1, 8.5]	3.6	10

Table 3: Held-out success (%) of the patch set each policy selects with a budget of 10 episodes, mean over five policy seeds. *All*: every held-out episode (retail: 82 of 120 with an injected fault; airline: 35 of 60). *Clean*: episodes without an injected fault (38 and 25). *–Apply*: paired difference from Apply attributed over all episodes, in points, with a task-clustered 90% interval. *#p*: patches deployed; *C*: verification episodes spent.

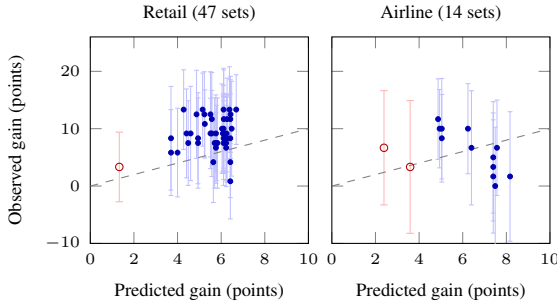


Figure 1: Each distinct deployed patch set’s gain over the unpatched agent as predicted from its bank replays (net of spurious recovery) against its observed held-out gain, with 90% intervals. Filled: sets containing the step-limit patch; open: sets without it. Dashed: $y = x$. Correlations are 0.23 (retail) and -0.49 (airline).

	ShopDesk	Retail	Airline
Analyst accuracy (audited)	0.72	0.41	0.29
Effective edges among blamed cells	–	0.18	0.33
Judge check: cost / replay	0.72	0.28	0.23
sens. / FPR, vs. single replay	0.57 / 0.21	0.37 / 0.11	0.41 / 0.13
sens. / FPR, vs. patch replay rate	–	0.49 / 0.05	0.55 / 0.07
efficiency ratio vs. replays	–	0.88 [0.46, 1.53]	1.25 [0.19, 3.95]
Spurious recovery b (range)	–	0.23 (0–0.50)	0.20 (0–0.50)
Repair q of effective edges (median)	–	0.50	0.41
Replays to decide an edge	–	5	7
Unpatched failure on solved tasks	–	0.15	0.12
Mean harm $\bar{\rho}$ (points)	–	–1.0 [–3.4, 1.5]	–0.6 [–3.3, 1.9]
Spread of ρ_k (points)	–	2.6 [0.2, 5.3]	2.1 [0.2, 4.5]
Patches harmful / helpful (95%)	–	2 / 5	3 / 4

Table 4: What the evidence costs and shows. The efficiency ratio is the judge’s break-even cost ratio divided by its actual cost (Proposition 4); values above 1 favour judge checks (Appendix I). Harm rows: excess failure rate of a patch on previously solved tasks under the task-level binomial model (Appendix J), posterior mean and 90% interval; the last row counts patches whose 20 regression runs differ significantly from the task-matched baseline.

deployment noise and more precise deployment measurements; Study 2 is designed for both.

C Proofs

Proposition 1. For any signal Y and any action set, $\max_S \mathbb{E}[U(S) \mid Y] \leq \mathbb{E}[\max_S U(S) \mid Y]$, so taking expectations gives $V(Y) \leq \mathbb{E}[\max_S U(S)] - \max_S \mathbb{E}[U(S)]$, the value of perfect information; this is the standard argument and needs no assumption on how Y relates to Δ . With $U(S) = \sum_{k \in S} \Delta_k$ and at most one patch per cell, both maxima separate over cells: $\max_S U(S) = \sum_{ij} \max_k \Delta_k^+$ and $\max_S \mathbb{E}[U(S)] = \sum_{ij} \max_k (\mathbb{E}[\Delta_k])^+$, which gives V_{ij} ; no independence between cells is needed. For Eq. 4, let k^* maximise $\mathbb{E}[\Delta_k]$. If $\mathbb{E}[\Delta_{k^*}] \geq 0$, then $V_{ij} = \mathbb{E}[\max_k \Delta_k^+] - \mathbb{E}[\Delta_{k^*}^+] + \mathbb{E}[\Delta_{k^*}^-]$, which is Eq. 4 with equality. Otherwise $V_{ij} =$

$\mathbb{E}[\max_k \Delta_k^+] = \mathbb{E}[\max_k \Delta_k^+ - \Delta_{k^*}^+] + \mathbb{E}[\Delta_{k^*}^+]$, and $\mathbb{E}[\Delta_{k^*}^+] < \mathbb{E}[\Delta_{k^*}^-]$ because $\mathbb{E}[\Delta_{k^*}] < 0$. Finally, the repair term $F f_{ieij} q_k$ is non-negative, so $\Delta_k^- \leq (1 - F) \rho_k^+ + c_a + c_w(1 - e_{ij})$; take expectations.

Heuristic defaults and constraints. For a default D , $\mathbb{E}[\max_S U(S)] - \mathbb{E}[U(D)] = (\mathbb{E}[\max_S U(S)] - \max_S \mathbb{E}[U(S)]) + (\max_S \mathbb{E}[U(S)] - \mathbb{E}[U(D)])$, and the second bracket is $G_D \geq 0$ by definition of the maximum. Restricting S to deployable sets leaves the argument unchanged. For repairs in one category, the Bonferroni inequality gives $1 - \prod_k (1 - q_k) \geq \sum_k q_k - \sum_{k < l} q_k q_l$, which bounds the overstatement of the additive form. The

Verified set	Δp [90%]	Δc_{run}	C	Net(1000) [90%], episodes	
				$v = 1$	$v = 10$
<i>Retail</i>					
Replay-each	-4.0 [-9.8, 1.8]	-0.06	10	12 [-217, 242]	-348 [-966, 271]
HarnessFix-style, per patch	-4.3 [-12.8, 4.2]	-0.03	9	-18 [-204, 167]	-408 [-1273, 456]
HarnessFix-style, bundle	-2.7 [-5.0, -0.4]	-0.00	6	-31 [-244, 183]	-271 [-583, 42]
Uncertainty sampling	-1.7 [-6.9, 3.5]	-0.07	10	41 [-189, 271]	-109 [-676, 458]
Uncertainty + checks	-2.8 [-8.4, 2.7]	0.02	10	-56 [-285, 173]	-311 [-907, 286]
CARVE, replay only	0.3 [-5.3, 6.0]	-0.02	10	10 [-220, 240]	40 [-568, 648]
CARVE	-3.7 [-9.9, 2.6]	-0.04	10	-8 [-227, 210]	-338 [-999, 322]
CARVE, measured check	-2.7 [-7.6, 2.3]	-0.01	10	-24 [-254, 206]	-264 [-810, 282]
CARVE, robust check	-1.2 [-6.3, 4.0]	0.02	9	-37 [-263, 188]	-142 [-704, 420]
Net-effect	-4.3 [-9.9, 1.3]	0.05	10	-100 [-307, 106]	-490 [-1083, 103]
<i>Airline</i>					
HarnessFix-style, per patch	-4.0 [-11.9, 3.9]	-0.04	8	-7 [-226, 212]	-367 [-1185, 451]
HarnessFix-style, bundle	-3.3 [-9.3, 2.6]	-0.06	6	23 [-209, 254]	-277 [-911, 356]
CARVE	-0.7 [-8.8, 7.5]	-0.11	10	96 [-156, 349]	36 [-814, 887]
Net-effect	-0.3 [-9.1, 8.5]	-0.08	10	68 [-194, 330]	38 [-875, 951]

Table 5: Net(N) of each verified set against Apply attributed (Eq. 1), at $N = 1000$ deployed episodes, with no change cost and a success worth v episodes of running cost. Δp : held-out difference in points (Table 3); Δc_{run} : estimated change in running cost per deployed episode, from the tokens of patched agents on solved tasks and of truncated step-limit episodes; C : verification episodes. Intervals combine the uncertainty of Δp and of Δc_{run} (standard error about 0.1), as normal.

bound on $-\sum_k \rho_k$ holds because the patched failure rate $r_0 + \sum_k \rho_k$ on solved tasks is non-negative under additivity. For (iii), removing patch k from a set S changes its value by $-\text{rep}(S) + \text{rep}(S \setminus k) + (1 - F)[\bar{\rho}(S) - \bar{\rho}(S \setminus k)] + c_a + c_w(1 - e_{ij})$, where rep is the product-form repair and $\bar{\rho}$ the capped sum of harms. Repair is monotone in S , so the first bracket is non-negative, and capping $\sum \rho$ from below changes it by at most ρ_k^+ when k is removed; summing over the dropped patches and taking expectations gives the bound. The greedy optimiser keeps, per component, the cell and patch with the largest positive net effect, which is a feasible deployable set, so the value it attains is a lower bound on the constrained optimum.

Corollary 1. With one patch, $V = \mathbb{E}[\Delta^+] - (\mathbb{E}\Delta)^+ = \pi G - (\pi G - (1 - \pi)H)^+ = \min(\pi G, (1 - \pi)H)$. The assumption that Δ is a function of e_{ij} is what makes the edge state sufficient: if q_k or ρ_k is uncertain given e_{ij} , learning only e_{ij} leaves value on the table and the general expression applies.

Proposition 2. The first part is Wald’s approximation for a sequential probability ratio test between $p_1 = b + (1 - b)q$ and $p_0 = b$: with thresholds $\pm \log \frac{1-\delta}{\delta}$, neglecting overshoot, the expected sample size under p_1 is the threshold divided by the per-observation drift $\text{KL}(p_1 \| p_0)$. The second is the normal-approximation sample size of a one-sided test of a binomial proportion, evaluated with the variance at r_0 .

D Verification Policies on the Intervention Banks

Summary. The banks give the same answer as the deployment comparison without new runs. In that comparison the policy seed added no detectable variance: the variance of the seed means of the verified sets’ differences from Apply attributed is 0.0007, against 0.0019 expected from run-to-run noise alone. Under the task-level harm model, applying the attributed patches is worth 1.43 (retail) and 0.66 (airline) of the repair-only oracle, and the verifying policies 0.39–1.56 at budget 40, beating it in 19–57% of posterior draws; scored on bank halves that share no trajectory with the policies’ evidence, they beat it in 22–47%. Policies that use the analyst’s blame as an edge prior used an analyst accuracy of 0.7; with the audited 0.41 and 0.29, or half of these, every such policy is worth less than with 0.7, on both domains (Table 8).

Policies. *Net-effect allocation.* A spike-and-slab model per cell takes its edge prior from the attribution counts (Eq. 7, Appendix U), with grid posteriors for q_k and Beta posteriors for each b_i and each patch’s regression failure rate r_k . It estimates Eq. 2 as $P(e_{ij}=1 \mid \mathcal{D}) F f_i \mathbb{E}[q_k \mid \mathcal{D}] - (1 - F) \max(0, \mathbb{E}[r_k \mid \mathcal{D}] - r_0) / (1 - r_0) - c$, a stricter objective than Eq. 2: it charges c on every applied change, never credits a patch for lowering failures on solved tasks, and scales harm by $1/(1 - r_0)$; Table 12 ablates the last two. It repeatedly runs the targeted replay, null replay or regression run with the largest n -step knowledge gradient per unit cost, $n \in \{1, 2, 4, 8\}$ (Frazier et al., 2008), and applies every patch with positive posterior net effect. *Apply attributed* uses no evidence: for each category it applies the first patch of the most-blamed component and of any component with at least 40% of the blame. *Replay-each* tests hypotheses in order of blame with three replays and accepts on two successes, as in do-then-verify methods (Ma et al., 2025). *HarnessFix-style, per patch* adds two regression runs that must break nothing; these counts are ours. *HarnessFix-style, bundle gate* is closer to the original, which validates a whole bundle of repairs on held-out tasks and promotes it if it solves at least one more task and breaks at most two (Chen et al., 2026); we apply the *Apply attributed* bundle, draw validation episodes from the bank, and allow three attempts. *Uncertainty sampling* replays the cell with the largest frequency-weighted posterior uncertainty, with or without three cheap checks per

patch first. CARVE and its variants are described in Appendix U.

Scoring. Policies run offline by drawing recorded outcomes from the bank, with replacement (with-out, as a check), with costs in measured tokens and a full replay as the unit; resampling seeds measure policy variability given the bank, not new evidence. *Adjudication:* a cell is an effective edge if its best patch’s replay success exceeds the category’s null rate by at least 0.2 (one-sided Fisher $p < 0.05$), giving 4 effective edges on retail and 7 on airline; choosing the best of three patches on the same replays makes this label optimistic, so we also score policies on bank halves they did not see. *Harm:* every patch’s ρ_k is drawn from the task-level binomial model of Appendix J, and edges and repair rates from a bootstrap of the replays. *Deployable sets:* every accepted set is reduced to one patch per component, keeping the cell of the most frequent category, as on held-out tasks. Values are fractions of the *repair-only oracle*, the failure mass repaired by the effective edges’ best patches with no cost, a fixed yardstick worth 0.070 (retail) and 0.114 (airline) in absolute success. The unpatched baseline (10 runs on each of 111 regression tasks) was collected about 12 hours after the regression runs; unpatched held-out success did not change over that time (0.816 vs. 0.817 on clean retail episodes). *Protocol:* retail was the development domain for every method, including the variants in Appendix L; airline was evaluated once for the original policies and once, later, for the net-effect method, which was frozen on retail beforehand.

Results. Table 6 reports each policy’s posterior expected value. On retail, applying the attributed patches is worth 1.43 oracles (10.1 points of success) and is the best policy at budget 10; at 40 and 80, CARVE with a robust check is ahead (1.56 and 1.59) but beats applying everything in only 57% of draws at budget 40; across budgets, the other verifying policies are worth 0.55–1.49. On airline, applying everything scores 0.66 and no verifying policy exceeds 0.73. The 90% intervals span about three oracles on retail (applying everything: $[-0.52, 2.67]$) and one and a half on airline. On source-split halves every value falls, because each half supports fewer effective edges: applying everything scores 0.96 (retail) and 0.23 (airline), and verifying policies beat it in 22–42% (retail) and 35–47% (airline) of draws at budget 40. With $c_w = 0.02$ per wrong change (Table 7) applying

everything falls to 1.10 and 0.54 on the full banks; the leaders change with budget and split, and none leads everywhere. On the banks, as on held-out tasks, no verification policy reliably beats applying the attributed patches.

Priors. Every policy with an edge prior used an analyst accuracy of $\hat{r} = 0.7$, well above the audited 0.41 (retail) and 0.29 (airline). Table 8 reruns their budget-40 decisions with the audited accuracy and with half of it. Every such policy is worth less than with 0.7 on both domains, by 0.11 to 0.65 oracles on retail and 0.04 to 0.25 on airline at the audited accuracy, and halving it again changes values by at most 0.30 more. With the audited or the weaker accuracy, no verifying policy is worth more than applying the attributed patches.

E Significance-Thresholded Scoring

Charging only regressions significant against the paired-trial r_0 reproduces the picture of a scoring that sets uncertain harm to zero: applying everything recovers 84% and 60% of the oracle and no verifying policy reaches it. With $c_w = 0.02$ per wrong change, the per-patch HarnessFix-style rule (retail, budgets 10–40; airline, 40) and the net-effect method (retail, 80; airline, 80, tied with uncertainty sampling with checks) lead, and CARVE declines with budget on airline (0.30→0.19). Appendix G varies c for both cost types; the winner changes with c .

F Robustness at Budget 40

G Change-Cost Sweep

Table 11 varies the change cost. The best policy changes with c : applying the attributed patches is best on retail at $c = 0$ (on airline, CARVE told $c = 0$ edges it, 0.71 against 0.60), replay-each at $c = 0.01$ on retail, the per-patch HarnessFix-style rule from $c = 0.01$ to 0.02 on airline and at 0.02 on retail, and the net-effect method at $c = 0.05$ on retail under both cost types and on airline per applied change. Charging every applied change rather than only wrong ones lowers every policy but changes the winner in only one setting (airline, $c = 0.05$). Told $c = 0$, the policies that use a cost internally accept patches with significant regressions, which is why some of them are negative in the first column.

Budget (episodes)	τ^2 retail (development)						τ^2 airline					
	full bank			source-split halves			full bank			source-split halves		
	10	40	80	10	40	80	10	40	80	10	40	80
Apply attributed	1.43	1.43	1.43	0.96	0.96	0.96	0.66	0.66	0.66	0.23	0.23	0.23
Bayes default (Prop. 1)	1.49	1.49	1.49	—	—	—	0.53	0.53	0.53	—	—	—
Replay-each	1.37	1.26	1.35	0.85	0.81	0.83	0.60	0.53	0.54	0.18	0.15	0.11
HarnessFix-style, per patch	0.55	1.01	1.28	0.55	0.87	0.97	0.19	0.45	0.53	0.09	0.24	0.22
HarnessFix-style, bundle	0.68	1.16	1.31	0.38	0.56	0.67	0.36	0.57	0.60	0.12	0.17	0.17
Uncertainty sampling	1.27	1.39	1.32	0.84	0.94	0.94	0.59	0.65	0.67	0.10	0.22	0.24
Uncertainty + checks	1.36	1.33	1.35	0.73	0.87	0.98	0.54	0.58	0.65	0.04	0.14	0.18
CARVE, replay only	1.23	1.41	1.49	0.87	0.91	0.98	0.71	0.65	0.70	0.24	0.24	0.24
CARVE	1.30	1.12	0.93	0.61	0.54	0.63	0.63	0.39	0.25	0.11	0.08	0.09
CARVE, measured check	1.23	1.41	1.49	0.82	0.93	1.01	0.71	0.65	0.71	0.24	0.23	0.23
CARVE, robust check	1.35	1.56	1.59	0.84	0.96	1.02	0.68	0.64	0.73	0.26	0.24	0.23
Net-effect (ours)	1.02	1.09	1.29	0.82	0.87	0.97	0.54	0.60	0.64	0.14	0.18	0.19

Table 6: Posterior expected value of each policy’s deployable patch set on the τ^2 banks at $c = 0$, under the task-level binomial harm model, as a fraction of the repair-only oracle; bold is the best per column. *Full bank*: policies and scoring use the same bank; 300 posterior draws, each also bootstrapping the replays, averaged over 40 policy seeds. *Source-split halves*: the bank is split into halves that share no source trajectory and no regression task; policies run on one half, with their inputs (experiment costs, spurious recovery, judge calibration) from that half and the audited analyst accuracy, and are scored on the other (5 halvings $\times$ 100 draws, 20 seeds). The Bayes default is Proposition 1’s no-evidence rule under the net-effect method’s priors. Values above 1 arise when patches are estimated to reduce failures on previously solved tasks.

H Policy Inputs and Regression Sampling

Attribution, taxonomy induction, labelling and candidate patches are shared by all policies and not charged to any budget. Every policy that uses an edge prior uses Eq. 7 with analyst accuracy $\hat{r} = 0.7$, not the audited 0.41 and 0.29 of Table 4; Table 8 rescores them with the audited accuracy and half of it. Three further inputs come from the bank itself and are also uncharged: the cost of each kind of experiment (mean tokens), the mean spurious recovery $\hat{b}$, which CARVE and uncertainty sampling plug in and the net-effect method uses only as a prior mean for its per-category b_i , and, for the two CARVE variants that use them, the judge’s measured sensitivity and false-positive rate. The re-run failure rate r_0 comes from paired re-runs of training tasks. *Regression tasks* are drawn uniformly, with replacement, from the clean training episodes the agent solved, so they follow the training-task distribution of the same domain, not the held-out tasks, which also include fault-injected episodes. The extension to 20 runs covered every patch an original policy accepted; 98–100% of the patches that any policy, including the later net-effect method, deploys at budget 40 have 20 runs, and a policy asking for more runs of a 4-run patch resamples those 4.

I Judge Configurations

The three new configurations regenerate the failing step once per pair and judge it with each prompt; the short prompt shows the task, the diagnosed root cause, the tool results or user message just before the failing step, and the old and new actions. Pairs are stratified on the paired replay’s outcome; because a check and its paired replay are separate regenerations, the check is independent of that outcome given the patch, so the stratification does not bias the patch-level fit. The short-prompt judges agree with the bank judge on 64–76% of pairs. The experiment used about 310 episode-equivalents of tokens. *Why the two definitions differ*. Conditioning on a single replay’s outcome attenuates both quantities towards the judge’s overall pass rate, because many failed replays come from patches that do repair some failures and many successful ones from spurious recovery; the patch-level fit removes this noise, which raises the bank judge’s break-even ratio from about 0.1 to 0.25 (retail) and 0.29 (airline). Across categories, using each category’s own spurious recovery and pairs (≥ 20), the single-replay ratio of the bank judge ranges from 0 to 2.0 on retail and 0.04 to 1.5 on airline. *Latency* was not measured; counted in sequential model calls, a check is two (regeneration and judge) and a replay from the failing step a median of 6 (retail) or 7 (airline), interquartile range 4–11. *Money*: we cannot

Budget (episodes)	τ^2 retail (development)						τ^2 airline					
	full bank			source-split halves			full bank			source-split halves		
	10	40	80	10	40	80	10	40	80	10	40	80
Apply attributed	1.10	1.10	1.10	0.61	0.61	0.61	0.54	0.54	0.54	0.06	0.06	0.06
Bayes default (Prop. 1)	0.91	0.91	0.91	–	–	–	0.56	0.56	0.56	–	–	–
Replay-each	1.10	1.03	1.13	0.54	0.52	0.51	0.49	0.45	0.44	0.02	0.01	–0.06
HarnessFix-style, per patch	0.55	0.93	1.10	0.55	0.80	0.78	0.18	0.42	0.48	0.06	0.18	0.10
HarnessFix-style, bundle	0.52	0.89	1.02	0.22	0.30	0.40	0.29	0.46	0.48	0.02	0.01	0.02
Uncertainty sampling	1.00	1.20	1.13	0.79	0.82	0.74	0.53	0.59	0.61	0.03	0.13	0.12
Uncertainty + checks	1.10	1.19	1.21	0.69	0.79	0.85	0.47	0.53	0.60	–0.02	0.06	0.07
CARVE, replay only	0.92	1.14	1.15	0.75	0.71	0.67	0.62	0.56	0.60	0.13	0.11	0.07
CARVE	1.08	1.02	0.89	0.56	0.44	0.49	0.55	0.33	0.21	0.04	–0.02	–0.03
CARVE, measured check	0.92	1.14	1.15	0.70	0.73	0.69	0.63	0.56	0.62	0.14	0.10	0.06
CARVE, robust check	1.04	1.27	1.25	0.71	0.75	0.70	0.61	0.56	0.64	0.16	0.11	0.06
Net-effect (ours)	0.87	0.98	1.15	0.77	0.78	0.83	0.49	0.55	0.58	0.08	0.08	0.06

Table 7: As Table 6, with a cost of $c_w = 0.02$ of the failure mass charged per patch aimed at a wrong edge.

	Retail			Airline		
	0.7	0.41	0.20	0.7	0.29	0.15
Analyst accuracy $\hat{r}$	0.7	0.41	0.20	0.7	0.29	0.15
Apply attributed	1.43	1.43	1.43	0.66	0.66	0.66
Uncertainty sampling	1.39	1.02	0.95	0.65	0.40	0.35
Uncertainty + checks	1.33	0.99	1.00	0.58	0.35	0.26
CARVE, replay only	1.41	1.22	1.10	0.65	0.57	0.49
CARVE	1.12	0.47	0.17	0.39	0.17	0.12
CARVE, measured check	1.41	1.22	1.10	0.65	0.61	0.50
CARVE, robust check	1.56	1.27	1.14	0.64	0.53	0.47
Net-effect	1.09	0.98	0.99	0.60	0.42	0.40

Table 8: Policies that use the edge prior of Eq. 7, with the analyst accuracy used in the main analysis (0.7), the audited accuracy and half of it; posterior value of deployable sets at budget 40 and $c = 0$ on the full bank (binomial harm model). Apply attributed uses no prior.

state prices for the models we used that will remain accurate, so the table reports token cost with the analyst model priced at one, two and four times the agent model.

J Harm Model, Checks and Sensitivity

Model. Regression outcomes are modelled per patch and task, together with the unpatched baseline runs on the same tasks. With n_{0t} unpatched runs of regression task t and f_{0t} failures, and n_{kt} runs of patch k on task t with f_{kt} failures,

$$f_{0t} \sim \text{Bin}(n_{0t}, \sigma(\alpha_t)),$$

$$f_{kt} \sim \text{Bin}(n_{kt}, \sigma(\alpha_t + \beta(\alpha_t - \bar{\alpha}) + \delta_k)),$$

with $\alpha_t \sim N(a, s_a^2)$, $\delta_k \sim N(\mu, s_d^2)$, $a, \mu \sim N(0, 3^2)$, $s_a \sim \text{HalfNormal}(2)$, $s_d \sim \text{HalfNormal}(1)$, $\beta \sim N(0, 1)$, and $\bar{\alpha}$ the logit of the mean unpatched failure rate. Task effects are shared by the baseline and every patch, so tasks that often fail unpatched do not make patches look

harmful, and the common slope β lets patches move every task’s failure rate towards or away from the middle. A patch’s harm is its excess failure probability averaged over the regression tasks, weighted by how often each task was solved among clean training episodes,

$$\rho_k = \sum_t w_t [\sigma(\alpha_t + \beta(\alpha_t - \bar{\alpha}) + \delta_k) - \sigma(\alpha_t)],$$

which respects the support $\rho_k \geq -r_0$ by construction; patches without runs draw δ_k from the population. We sample by Metropolis within Gibbs, four chains, 2,400 retained draws; $\hat{R} \leq 1.006$ for μ , s_d , β and the pooled harm.

Estimates. The unpatched failure rate on solved tasks is 0.146 (retail) and 0.124 (airline). The pooled excess failure rate of the patches is -1.0 points, 90% interval $[-3.4, 1.5]$, on retail and -0.6 $[-3.3, 1.9]$ on airline, and the spread of ρ_k between patches is 2.6 $[0.2, 5.3]$ and 2.1 $[0.2, 4.5]$ points. On retail the slope is negative, $\beta = -0.36$ $[-0.51, -0.17]$: patches tend to raise failures on tasks that rarely fail and lower them on tasks that often fail; on airline it is 0.03 $[-0.21, 0.31]$.

Posterior predictive check. On the patches with 20 runs, the observed standard deviation of failure rates is 0.076 (retail) and 0.066 (airline); data replicated from the fitted model give 0.079 and 0.072, with $P(\text{replicated} \geq \text{observed}) = 0.62$ and 0.79. The number of patches without a failure (4 and 7 observed, 4.2 and 6.2 replicated) and the largest failure rate (0.30 and 0.30 observed, 0.35 and 0.31 replicated) are also reproduced. An earlier analysis used a normal approximation, $y_k = m_k/n_k - r_{0,k} \sim N(\rho_k, v_k)$ with $\rho_k \sim N(\mu, \tau^2)$,

Budget (episodes)	τ^2 retail (development)						τ^2 airline (held out)					
	significant harm			+ $c_w=0.02$			significant harm			+ $c_w=0.02$		
	10	40	80	10	40	80	10	40	80	10	40	80
Apply attributed	0.84	0.84	0.84	0.16	0.16	0.16	0.60	0.60	0.60	0.15	0.15	0.15
Replay-each	0.78	0.74	0.71	0.14	0.33	0.36	0.43	0.09	0.10	0.07	-0.12	-0.08
HarnessFix-style, per patch	0.52	0.51	0.52	0.52	0.42	0.30	0.18	0.40	0.43	0.18	0.37	0.31
HarnessFix-style, bundle	0.11	0.21	0.26	-0.21	-0.31	-0.31	0.34	0.53	0.52	0.07	0.12	0.11
Uncertainty sampling	0.42	0.50	0.31	0.01	0.23	0.01	0.56	0.50	0.51	0.28	0.25	0.25
Uncertainty + checks	0.14	0.47	0.48	-0.25	0.25	0.24	0.55	0.50	0.53	0.27	0.29	0.36
CARVE, replay only	0.01	0.18	0.14	-0.48	-0.26	-0.39	0.22	0.35	0.48	-0.16	-0.02	0.12
CARVE	0.33	0.32	0.52	-0.01	0.19	0.47	0.55	0.34	0.25	0.30	0.23	0.19
CARVE, measured check	0.01	0.18	0.14	-0.48	-0.26	-0.39	0.25	0.43	0.54	-0.09	0.07	0.20
CARVE, robust check	0.34	-0.03	0.02	-0.15	-0.49	-0.48	0.22	0.34	0.48	-0.12	-0.01	0.14
Net-effect (ours)	0.24	0.52	0.66	0.09	0.41	0.52	0.26	0.39	0.46	0.14	0.31	0.36

Table 9: Sensitivity analysis: as Table 6, but accepted patches are charged only their significant measured regressions (one-sided binomial $p < 0.05$ against r_0), mean over 200 resampling seeds of the original bank. This scoring sets uncertain harm to zero and so favours policies that apply many patches.

Policy (budget 40)		Posterior, $c_w=0.02$		Sensitivity: significant harm only						
		Value [90%]	$P(> \text{apply})$	c_w	c_a	Split	Natural	No step limit	One per comp.	No repl.
Retail	Apply attributed	0.44 [-1.68, 2.40]	—	0.16	0.01	-0.46	-1.25	-0.61	0.47	0.16
	HarnessFix-style, per patch	0.70 [0.04, 1.38]	0.57	0.42	0.34	0.12	-0.82	-0.27	0.42	0.54
	HarnessFix-style, bundle	0.39 [-1.11, 1.89]	0.44	-0.31	-0.42	-0.80	-1.99	-0.83	-0.08	-0.45
	Uncertainty sampling	0.87 [-0.50, 2.15]	0.69	0.23	0.09	-0.13	-1.10	-0.64	0.29	0.28
	CARVE	0.75 [-0.09, 1.65]	0.59	0.19	0.07	0.04	-0.89	-0.25	0.20	0.51
	Net-effect (ours)	0.80 [0.06, 1.52]	0.60	0.41	0.32	0.23	-0.86	-0.35	0.41	0.52
Airline	Apply attributed	0.61 [-0.13, 1.38]	—	0.15	-0.08	-0.41	-0.07	-0.03	0.46	0.15
	HarnessFix-style, per patch	0.44 [0.06, 0.86]	0.26	0.37	0.25	0.17	0.28	0.20	0.38	0.38
	HarnessFix-style, bundle	0.55 [-0.12, 1.25]	0.15	0.12	-0.09	-0.30	-0.09	-0.04	0.39	0.10
	Uncertainty sampling	0.66 [0.06, 1.32]	0.66	0.25	0.01	0.04	0.05	0.05	0.41	0.26
	CARVE	0.36 [-0.09, 0.83]	0.16	0.23	0.08	-0.03	0.24	0.22	0.25	0.23
	Net-effect (ours)	0.65 [0.06, 1.26]	0.58	0.31	0.14	0.17	0.23	0.13	0.35	0.26

Table 10: Uncertainty and robustness at budget 40 (fractions of the repair-only oracle). *Posterior*: deployable sets as in Table 6, mean and 90% interval over the posterior of the earlier normal harm model (Appendix J) and the replay bootstrap, and the probability of beating Apply attributed in the same draw. Sensitivity columns charge significant regressions and $c_w = 0.02$ per wrong change unless stated: c_a , 0.02 per applied change instead; *Split*, policies see a random half of every outcome list and adjudication uses the other half (5 splits); *Natural*, values reweighted to the category shares of natural failures; *No step limit*, the step-limit component removed from the bank; *One per comp.*, the accepted set reduced to one patch per component as deployed; *No repl.*, policies draw bank outcomes without replacement.

which predicted more spread between patches than observed (0.092 and 0.081, $P = 0.95$ and 0.97); its posterior columns in Table 10 and its variants in Tables 2 and 14 are kept as sensitivity analyses.

Sensitivity. Table 14 shows why the harm model matters on retail and hardly on airline. The binomial model values the attributed set at 1.43 oracles on retail; the normal model, which over-predicts the spread between patches and therefore the harm a seven-patch set may carry, values it at 0.77, and its variants range from -1.82 (no pooling) to 1.12 (complete pooling). On airline all models but no pooling give 0.63–0.86. Across models, the best of the five verifying policies beats applying everything in 33–82% of draws on retail and 43–47% on

airline.

K Held-Out Ablations

With a margin of 5 points, chosen after the fact, applying the attributed patches is non-inferior to 9 of the 10 verified sets on retail (all but CARVE with replays only) and to the two HarnessFix-style sets on airline. Single patches show the same problem: leaving the step-limit patch out of the attributed set lowers retail success by 3.3 points (s.e. 4.2) where its replays predict 4.9, and leaving out each policy patch lowers it by 0.0 to 3.3 points where replays predict 0.0 to 0.5, each within about one standard error. The shortfall in retail’s predicted level is therefore spread over the policy patches rather than

Policy (budget 40)		per wrong change c_w				per applied change c_a			
		0	0.01	0.02	0.05	0	0.01	0.02	0.05
Retail	Apply attributed	0.84	0.50	0.16	-0.85	0.84	0.43	0.01	-1.23
	Replay-each	0.74	0.53	0.33	-0.30	0.74	0.48	0.22	-0.57
	HarnessFix-style, per patch	0.51	0.46	0.42	0.28	0.51	0.42	0.34	0.09
	HarnessFix-style, bundle	0.21	-0.05	-0.31	-1.09	0.21	-0.11	-0.42	-1.38
	Uncertainty sampling	-1.90	-2.46	0.23	0.05	-1.90	-2.57	0.09	-0.17
	Uncertainty + checks	-1.57	-1.95	0.25	0.25	-1.57	-2.06	0.12	0.01
	CARVE, replay only	-0.07	-0.49	-0.26	0.00	-0.07	-0.60	-0.44	-0.25
	CARVE	-0.19	-0.19	0.19	0.28	-0.19	-0.27	0.07	0.14
	CARVE, measured check	-0.07	-0.49	-0.26	0.00	-0.07	-0.60	-0.44	-0.25
	CARVE, robust check	-0.05	-0.70	-0.49	0.01	-0.05	-0.81	-0.66	-0.26
	Net-effect (ours)	-1.02	-0.58	0.41	0.63	-1.02	-0.67	0.32	0.44
Airline	Apply attributed	0.60	0.37	0.15	-0.53	0.60	0.26	-0.08	-1.09
	Replay-each	0.09	-0.02	-0.12	-0.43	0.09	-0.08	-0.25	-0.76
	HarnessFix-style, per patch	0.40	0.39	0.37	0.32	0.40	0.33	0.25	0.02
	HarnessFix-style, bundle	0.53	0.32	0.12	-0.50	0.53	0.22	-0.09	-1.02
	Uncertainty sampling	0.49	0.09	0.25	0.19	0.49	-0.04	0.01	-0.25
	Uncertainty + checks	0.44	0.01	0.29	0.21	0.44	-0.12	0.08	-0.13
	CARVE, replay only	0.48	0.08	-0.02	0.06	0.48	-0.07	-0.26	-0.44
	CARVE	0.71	0.33	0.23	0.10	0.71	0.22	0.08	-0.07
	CARVE, measured check	0.56	0.10	0.07	0.11	0.56	-0.04	-0.17	-0.38
	CARVE, robust check	0.63	0.21	-0.01	0.08	0.63	0.06	-0.26	-0.42
	Net-effect (ours)	0.60	0.10	0.31	0.29	0.60	-0.04	0.14	0.05

Table 11: Value at budget 40 as a fraction of the repair-only oracle, with significant regressions charged, as the change cost varies. Policies that use a cost internally are told the same c (the net-effect method always charges it per applied change, the others per wrong change). At $c = 0$ the uncertainty-based policies accept almost every cell with a positive posterior and pick up patches with significant regressions.

Regression term	Retail			Airline		
	10	40	80	10	40	80
As used (clip, rescale)	0.70	0.90	1.08	0.63	0.70	0.77
No clip	0.65	0.38	0.52	0.78	0.82	0.81
No rescale	0.70	0.90	1.08	0.63	0.70	0.77
Neither (Eq. 2)	0.63	0.40	0.56	0.79	0.82	0.81

Table 12: Net-effect method with its regression term as used (clipped at zero, scaled by $1/(1 - r_0)$) and without either, posterior value of deployable sets at $c = 0$ over 40 seeds.

located in one, and no single patch’s deployed effect is resolved. These runs were made on retail, on the same held-out episodes as Table 3 (Tables 15 and 16) or on new clean episodes (Table 17). Differences are paired and task-clustered, with 90% intervals. Leaving any one of the seven attributed patches out lowers success by 0.00 to 0.03, and the sum of these contributions is not distinguishable from the whole set’s effect, so the runs neither show nor exclude interactions. On clean episodes, applying everything gains $+0.04 [-0.03, 0.10]$ over the unpatched agent. The bundle gate is ahead of it by $+0.04 [0.01, 0.08]$, but three of its five seed sets are identical to the attributed set and one is empty, so the difference is not an effect of verification; on the 72 slots where it ran the identical set the differ-

ence is $+0.03 [-0.04, 0.09]$, which indicates the run-to-run noise of these comparisons. Unpatched clean success was 0.82 both before and after the new runs.

L Development Variants

On the retail bank we tried, before any airline evaluation of them:

- CARVE with two null replays per category before trusting its replays (per-category b);
- CARVE with a regression gate of two runs per accepted patch;
- both combined;
- an evidence rule that requires a positive likelihood ratio from data;
- tempered and flat priors;
- a Bayesian regression gate.

With control replays and the gate combined, CARVE gained 0.68 and 0.60 of the oracle at budgets 10 and 20, above plain CARVE (0.42, 0.48), but fell below it at 40 and 80 (0.47 vs. 0.57 and 0.56). It never beat the per-patch HarnessFix-style rule (0.72, 0.73). Post hoc on airline it was worse than plain CARVE (0.19 vs. 0.31 at budget 10).

Judge	Cost per check		vs. single replay	
	Tokens	Pro $2\times / 4\times$	Break-even	Ratio [95% CI]
<i>Retail</i>				
Bank judge (analyst, full trajectory)	0.29	0.43 / 0.70	0.14	0.47 [0.25, 0.70]
Analyst, failing step only	0.30	0.45 / 0.76	0.10	0.32 [0.13, 0.51]
Agent model, full trajectory [†]	0.67	0.67 / 0.67	0.11	0.16 [0.01, 0.31]
Agent model, failing step only	0.28	0.28 / 0.28	0.10	0.35 [0.13, 0.57]
<i>Airline</i>				
Bank judge (analyst, full trajectory)	0.22	0.33 / 0.54	0.14	0.62 [0.33, 0.91]
Analyst, failing step only	0.23	0.34 / 0.57	0.10	0.43 [0.17, 0.69]
Agent model, full trajectory [†]	0.58	0.58 / 0.58	0.24	0.40 [0.10, 0.70]
Agent model, failing step only	0.22	0.22 / 0.22	0.09	0.39 [0.16, 0.62]

Table 13: Single-step checks with four judges. Cost is per check as a fraction of a full replay in uncached input plus output tokens, including regenerating the failing step (0.15 of a replay on retail, 0.12 on airline); “Pro $2\times / 4\times$ ” prices the analyst model’s tokens at twice or four times the agent model’s, which affects only analyst-model judges (replays use the agent model). Break-even is the cost ratio below which checks decide an edge at lower expected cost than replays (Proposition 4, with the domain’s mean spurious recovery and median repair rate of effective edges, averaged over true and false edges); Ratio is break-even divided by cost, so values above 1 favour checks. *vs. single replay*: sensitivity and false-positive rate conditional on the paired replay’s outcome (360 stratified pairs; Table 4 gives the bank judge’s on all pairs). *vs. patch replay rate*: $z = \epsilon + (s - \epsilon)p_k$ fitted by moments with the binomial noise of each patch’s 20-replay rate removed, which is the observation model of Proposition 4; the bank judge row uses all 2,160 pairs, the others their 360 pairs. Intervals bootstrap source trajectories (single replay) or patches (patch rate) together with null replays. [†]40 random pairs per domain.

Harm model	Retail					Airline				
	μ	Apply	Best	P	VPI	μ	Apply	Best	P	VPI
Task-level binomial (primary)	−0.01	1.43	1.39	0.33	1.18	−0.01	0.66	0.65	0.45	0.86
Hierarchical normal	−0.00	0.77	1.05	0.63	2.02	−0.01	0.73	0.71	0.46	0.69
Complete pooling ($\tau = 0$)	−0.00	1.12	1.02	0.36	0.87	−0.01	0.73	0.71	0.46	0.69
Spread doubled (2τ)	−0.00	0.27	0.96	0.70	2.51	−0.01	0.73	0.71	0.46	0.69
No pooling	0.04	−1.82	0.20	0.82	4.65	0.04	0.39	0.37	0.43	1.15
Tasks resampled	−0.00	−0.52	0.58	0.65	3.31	−0.00	0.63	0.62	0.45	0.87
No support cap	−0.00	0.82	1.09	0.64	9.48	−0.01	0.74	0.71	0.45	1.06
Not deployable	−0.00	0.64	1.14	0.61	2.15	−0.01	0.86	0.86	0.47	0.56

Table 14: Sensitivity of the posterior to the harm model, $c = 0$, budget 40: mean harm μ (for the binomial model, the pooled excess failure rate), value of Apply attributed and of the best of five verifying policies (both HarnessFix-style gates, uncertainty sampling, CARVE, net-effect), the probability that it beats Apply attributed, and the total value of perfect information over Apply attributed, as fractions of the oracle. Rows below the first use the earlier normal approximation; airline has $\tau = 0$ there, so pooling and doubling τ change nothing.

Set (seed 0)	With – without step patch			Without – unpatched	
	All	No step fault	Step fault	No step fault	
Apply attributed	0.03 [−0.04, 0.10]	−0.03 [−0.10, 0.04]	0.58 [0.37, 0.80]	0.09 [0.01, 0.18]	
HarnessFix-style, bundle	0.03 [−0.04, 0.10]	−0.03 [−0.10, 0.04]	0.58 [0.37, 0.80]	0.09 [0.01, 0.18]	
Uncertainty sampling	0.01 [−0.07, 0.08]	−0.02 [−0.09, 0.06]	0.25 [−0.07, 0.57]	0.06 [−0.04, 0.15]	
CARVE	0.03 [−0.04, 0.11]	0.00 [−0.07, 0.07]	0.33 [0.07, 0.59]	0.05 [−0.03, 0.13]	
Net-effect (ours)	0.05 [−0.02, 0.12]	−0.01 [−0.08, 0.06]	0.58 [0.34, 0.83]	0.02 [−0.07, 0.10]	

Table 15: Removing the step-limit patch from each policy’s seed-0 set (the bundle gate’s seed-0 set equals the attributed set). 108 episodes without and 12 with the step-limit fault.

Patch left out of Apply attributed	All	No step fault
CFG.max_steps	0.03 [−0.04, 0.10]	−0.03 [−0.10, 0.04]
POL.cancel	0.03 [−0.04, 0.10]	0.04 [−0.04, 0.12]
POL.exchange	0.03 [−0.04, 0.09]	0.03 [−0.05, 0.11]
POL.generic	0.03 [−0.04, 0.09]	0.01 [−0.05, 0.07]
POL.intro	0.02 [−0.05, 0.08]	0.02 [−0.05, 0.08]
POL.modify	0.03 [−0.05, 0.10]	0.04 [−0.05, 0.12]
POL.return	0.00 [−0.06, 0.06]	−0.01 [−0.08, 0.06]
Whole set vs unpatched	0.12 [0.05, 0.19]	0.06 [−0.01, 0.14]
Whole set – sum of parts	−0.04 [−0.35, 0.27]	−0.03 [−0.38, 0.33]

Table 16: Leave-one-out on the attributed set: success of the whole set minus the set without the named patch.

The other variants were worse everywhere; the evidence rule and the Bayesian gate accepted almost nothing (gain ≤ 0.12). These numbers use the fixed change cost c without regressions and the bank before the regression runs were extended.

The net-effect method of Appendix D was developed afterwards, on retail, from the diagnosis that CARVE lacked per-category spurious recovery and evidence for regressions.

	Old	New	Pooled	– Unpatched	– Apply
Unpatched	0.82	0.82	0.82	–	–0.04 [–0.10, 0.03]
Apply attributed	0.87	0.85	0.85	0.04 [–0.03, 0.10]	–
Net-effect (ours)	0.83	0.87	0.86	0.04 [–0.01, 0.09]	0.00 [–0.05, 0.06]
HarnessFix-style, bundle	0.87	0.91	0.90	0.08 [0.03, 0.14]	0.04 [0.01, 0.08]
HarnessFix-style, per patch	0.90	0.85	0.86	0.05 [–0.01, 0.10]	0.01 [–0.06, 0.08]

Table 17: Clean held-out episodes: 38 old (Table 3) and 120 new (40 tasks $\times$ 3), where new episode e uses the policy’s seed- $(e \bmod 5)$ set. Differences pool both, paired by episode or slot and clustered by task.

M ShopDesk Details

ShopDesk has six deterministic tools (orders, products, policies, refunds), answers checked exactly (refund amounts, counts, totals), and five injected faults: a tool description misstating price units, a list tool whose description omits pagination, a format instruction that rounds to whole dollars, a planning instruction that skips the policy lookup, and a tight step limit. Over three seeds, the analyst’s attribution accuracy on injected failures was 0.75, 0.64 and 0.77. ShopDesk was evaluated online, without a bank: each policy ran against the live agent with fault-based ground truth (an edge is real when at least 30% of a category’s failures come from that component’s fault). Table 18 reports edge F1 and held-out success. Large harms do occur: a formatting patch that passed its target-category replays made 26% of held-out tasks answer N/A (success 0.89 $\rightarrow$ 0.73).

N Study 2: RRSI Port and Registered Analysis

Code. RRSI’s loop is vendored from its public repository under the Apache 2.0 licence, with the upstream copyright headers kept. We changed the model backend (calls go to DeepSeek’s Anthropic-compatible endpoint, with thinking disabled and token usage recorded per role), and one rule: a proposal that ends with no file change and no declared edit is bounced back to the proposer like an aborted one, at most three times, instead of ending the round with no candidate. RRSI’s selection rule is used unchanged: a candidate is admissible if its evolve success is at most δ below the best so far and either exceeds the incumbent’s by more than δ at an increase in running cost within $\beta_0 + \beta_1 \Delta S$, or lies within the band with a positive weighted sum of gain, cost saving and novelty (with RRSI’s weights, gain counts 0 there, so only a cheaper or untried kind of change passes); a harness with errors in more than 2% of trials is rejected; the admissible candidate with the highest success wins. RRSI’s

own domains (coding, workspace, engineering design) are not used. Our τ^2 adapter implements RRSI’s domain interface: a frozen episode driver that uses τ^2 ’s databases, tools, policies, tasks and user-simulation guidelines, a starting harness with four hooks (choose the next action, check a tool call before it runs, render a tool result, rebuild per-episode state from a recorded prefix), and optional mechanisms the proposer may use: bounded extra model calls, harness-side tools and skill files. Memory across episodes is not allowed.

Harness check. At temperature 0 our episode loop matches τ^2 ’s own orchestrator on 60 test tasks $\times$ 3: 90.0% against 90.8% success on retail (paired difference -0.8 points, $[-5.8, 3.3]$) and 71.7% against 71.7% on airline. Scoring τ^2 ’s natural-language assertions as well changed the reward of 2 of about 1,000 episodes, so we keep Study 1’s reward (database state and required information).

Leakage screening. Besides RRSI’s LLM critic, a pattern check rejects any added line that contains a task’s user id, order or reservation id, item or flight number or customer e-mail, refers to the grader (gold actions, database hashes, evaluation criteria) or to τ^2 ’s task data, imports the benchmark or our experiment code, or writes files.

Evidence collection per round. *References:* for every evolve task the incumbent failed in RRSI’s full evaluation, its worst trial, at most 20 tasks (a seeded random subset beyond that). The start step is the earliest step the round’s failure digest cites; without a digest, the first state-changing call that is not a gold action; otherwise the first step. Each reference is replayed 4 times with each measured candidate and 4 times with the incumbent (null replays). *Judge:* DeepSeek-V4-Pro reads the candidate’s diff and declared edits, up to 14 of the incumbent’s failed trials of the round with their grading block, and one line per solved task, and returns a probability of repair for each failed task and an expected number of broken trials; its estimate is (expected repairs $-$ expected breaks) / evolve trials. *Deployment:* every incumbent and measured candidate on the held-out tasks, retail 6 and airline 12 trials per task (4 and 8 as registered; see below), the base and final harnesses on 10 (retail) and 12 (airline) trials per task.

Estimators. With f the incumbent’s evolve failure share, $sample@b$ is the mean paired difference

Method	Edge F1 at budget				Held-out success @60	Single checks
	10	20	30	60		
LLM attribution only	0.61 $\pm$ 0.04	0.61 $\pm$ 0.04	0.61 $\pm$ 0.04	0.61 $\pm$ 0.04	0.98 $\pm$ 0.00	0
Replay-each (do-then-verify)	0.62 $\pm$ 0.03	0.63 $\pm$ 0.03	0.67 $\pm$ 0.05	0.75 $\pm$ 0.03	0.99 $\pm$ 0.00	0
Uncertainty sampling	0.55 $\pm$ 0.04	0.57 $\pm$ 0.04	0.58 $\pm$ 0.05	0.66 $\pm$ 0.04	0.99 $\pm$ 0.00	0
Uncertainty + cheap checks	0.52 $\pm$ 0.07	0.56 $\pm$ 0.07	0.53 $\pm$ 0.09	0.53 $\pm$ 0.08	0.96 $\pm$ 0.03	57
CARVE, full replay only	0.61 $\pm$ 0.07	0.61 $\pm$ 0.06	0.59 $\pm$ 0.04	0.64 $\pm$ 0.03	0.91 $\pm$ 0.09	0
CARVE (default check model)	0.59 $\pm$ 0.06	0.57 $\pm$ 0.05	0.61 $\pm$ 0.05	0.68 $\pm$ 0.03	0.98 $\pm$ 0.01	4
CARVE, calibrated check	0.59 $\pm$ 0.06	0.58 $\pm$ 0.05	0.60 $\pm$ 0.05	0.66 $\pm$ 0.04	0.89 $\pm$ 0.08	0
CARVE, robust check model	0.58 $\pm$ 0.07	0.61 $\pm$ 0.06	0.61 $\pm$ 0.04	0.69 $\pm$ 0.01	0.99 $\pm$ 0.01	0
Unpatched agent	0.59 $\pm$ 0.04					

Table 18: ShopDesk (3 seeds, mean $\pm$ s.e.). Held-out success after applying the patches accepted at budget 60 to 120 held-out episodes.

on b random (task, trial) pairs of the full evaluations; $\text{replay}@b$ is $f \cdot \widehat{\text{fix}}_c$ from b candidate replays spread evenly over the references; $\text{replay-null}@b$ is $f(\widehat{\text{fix}}_c - \widehat{\text{fix}}_{\text{null}})$ from $b/2$ replays of each kind; $\text{net}@b$ uses $b/4$ candidate and $b/4$ null replays and $b/2$ regression pairs, on tasks the incumbent solved, and adds $(1-f)$ times their mean paired difference; $\text{net}@b$ with prior shrinks the candidate’s repair rate to a Beta prior with mean 0.41 or 0.29, 0.7 or 0.2 and two pseudo-trials; full is RRSI’s own decision. Every estimate goes through RRSI’s selection rule with the evidence’s own noise band, twice its standard error, and the evidence’s measured token cost; the judge has no noise band. Each rule’s per-round score is the deployment gain of its choice, averaged over 400 random subsamples of the evidence. The registered analysis applies RRSI’s error guard, which rejects a harness that raises errors in more than 2% of trials, only through RRSI’s own decision, not to the other rules’ evidence; §5 reports the alternative post hoc.

Registration of r1 and changes after it. The pre-registration (verify/PREREGISTRATION.md), the analysis code and the run scripts were committed before the first held-out episode; domains, splits, models, temperatures, T , m , k , budgets, evidence definitions, the decision rule, the margin and the endpoints did not change. The registration file logs four changes. (i) Before any held-out episode, a bug fix: the failure share f was computed from the replay references, which are capped at 20 tasks, instead of from all failed trials of the incumbent’s evaluation. (ii) Also before any held-out episode, $\text{Net}(N)$, a registered secondary outcome, was implemented as the mean over rounds of $N(v\Delta p - \Delta c_{\text{run}}) - C_{\text{verify}}$ against keeping the incumbent, with $\Delta C_{\text{change}} = 0$ because every rule

sees the same candidates. (iii) After six rounds, $\text{net}@40$ and full evaluation chose differently in 58% of them, above the 40% of the power analysis, and held-out trials per harness were raised from 160 to 240 (retail 4 to 6, airline 8 to 12 per task, airline endpoints 10 to 12); harnesses already deployed were topped up. The registration assumed this rate would be known before any held-out episode ran; in fact the first rounds had been deployed, and a progress log had shown a few held-out success rates while the run was monitored, although no held-out analysis was run. (iv) Over all 40 rounds the rate was 26.5%, for which the registered 160 episodes would have sufficed; we report 240 as primary and 160 as a sensitivity analysis, with the same conclusions. The analyses without the broken candidate, with the error check applied to every rule, with the execution-check baselines, with blocks of rounds that share an incumbent and of detectable differences, and the rules that verify one candidate or verify sequentially, are post hoc.

Registration of r2 and r3. After r1’s results, we registered r2 and r3 (verify/PREREGISTRATION_E1R.md) before any of their episodes ran, with the analysis of §5. The analysis code (verify/e1r.py) was committed while they ran and tested on r1 only. The sequential rules and their tests were added as an addendum before any decision value of r2 or r3 was computed. The operations log records the following. In r2’s third retail round, RRSI’s digester received a string where it expected an object and the driver stopped; the fix returns such a reply to the digester as an error and changes nothing else. The container restarted twice, and every run resumed from its last settled step, reusing the reports, drafts and evaluations it had written,

Evidence	b	Decision value D [90%], common error check			r2+r3			
		r1 (40 rounds)	r2+r3 (80, primary)	r1-r3 (120)	Accepts	Agree	Ep.	Net(10^4)
Keep incumbent	0	+0.00 [+0.00, +0.00]	+0.00 [+0.00, +0.00]	+0.00 [+0.00, +0.00]	0.00	0.74	0	0
None	0	-1.87 [-3.52, +1.53]	-0.47 [-1.52, +0.62]	-0.94 [-1.94, +0.22]	0.91	0.25	0	-818
LLM judge	0	-1.72 [-3.30, +1.37]	-0.40 [-1.41, +0.67]	-0.84 [-1.75, +0.27]	0.81	0.30	0	-731
None + check	10	-0.58 [-1.88, +1.56]	-0.47 [-1.52, +0.62]	-0.51 [-1.39, +0.46]	0.91	0.25	13	-831
Judge + check	10	-0.43 [-1.53, +1.41]	-0.40 [-1.41, +0.67]	-0.41 [-1.20, +0.47]	0.81	0.30	13	-744
Full (RRSI)	all	+0.22 [-0.04, +1.11]	+0.04 [-0.35, +0.43]	+0.10 [-0.18, +0.41]	0.26	1.00	141	-108
Sequential full	$\leq$ all	+0.15 [-0.03, +0.79]	+0.02 [-0.34, +0.35]	+0.06 [-0.18, +0.32]	0.20	0.93	81	-71
Sequential sample	$\leq$ 80	+0.12 [-0.15, +0.98]	+0.07 [-0.26, +0.41]	+0.09 [-0.15, +0.38]	0.23	0.78	53	10
Sample	10	-0.04 [-0.42, +0.77]	-0.08 [-0.54, +0.37]	-0.07 [-0.39, +0.30]	0.39	0.60	13	-107
Replay	10	-0.37 [-1.26, +0.88]	-0.67 [-1.56, +0.17]	-0.57 [-1.20, +0.13]	0.63	0.44	13	-1,000
Replay-null	10	-0.05 [-0.25, +0.16]	-0.36 [-0.74, -0.01]	-0.25 [-0.53, -0.00]	0.18	0.69	11	-594
Net	10	-0.07 [-0.54, +0.93]	-0.17 [-0.73, +0.35]	-0.13 [-0.52, +0.30]	0.48	0.56	12	-217
Sample	40	+0.04 [-0.35, +0.93]	-0.06 [-0.58, +0.45]	-0.02 [-0.37, +0.38]	0.40	0.67	54	-119
Replay	40	-0.40 [-1.56, +1.34]	-0.74 [-1.83, +0.36]	-0.63 [-1.46, +0.30]	0.86	0.33	50	-1,126
Replay-null	40	-0.07 [-0.34, +0.21]	-0.39 [-0.79, -0.03]	-0.29 [-0.58, -0.01]	0.20	0.68	45	-674
Net	40	-0.04 [-0.53, +0.76]	-0.10 [-0.62, +0.39]	-0.08 [-0.45, +0.31]	0.44	0.63	49	-152
Sample	80	+0.08 [-0.30, +1.06]	+0.04 [-0.46, +0.57]	+0.06 [-0.30, +0.47]	0.38	0.78	94	-57
Replay	80	-0.40 [-1.55, +1.34]	-0.71 [-1.80, +0.42]	-0.60 [-1.45, +0.33]	0.88	0.31	56	-1,101
Replay-null	80	-0.12 [-0.60, +0.45]	-0.51 [-1.03, -0.05]	-0.38 [-0.74, -0.02]	0.25	0.65	84	-848
Net	80	-0.02 [-0.53, +0.81]	-0.04 [-0.55, +0.45]	-0.03 [-0.42, +0.36]	0.41	0.69	98	-123

Table 19: Study 2: decision value D of each rule with the error check applied to every rule’s own episodes, the held-out gain of its choice over the incumbent in points per round (keeping the incumbent scores 0), with 90% intervals from a bootstrap of blocks of rounds that share an incumbent and of held-out tasks, within domain and trajectory. r1: the first trajectory per domain (its registered analysis is in Table 21); r2+r3: the registered primary analysis; r1-r3: pooled. + *check* rules run 10 episodes of each candidate only for the check; sequential rules stop a candidate’s episodes early. Accepts (mean probability of accepting a candidate in a round), Agree (share of rounds in which the rule makes full evaluation’s choice), evidence episodes per round, and Net(10^4) (Eq. 1 against keeping the incumbent, $N = 10^4$ deployed episodes, a success worth $v = 10$ episodes, in episodes) are for r2+r3. Full: RRSI’s own decision. Best value per column in bold.

Evidence	Episodes	Estimate of ΔS
None	0	accept a candidate (at random if two)
LLM judge	0	predicted fixes – predicted breaks
Sample@ b	b	paired difference on b evolve episodes
Replay@ b	b	failure share $\times$ repair rate
Replay-null@ b	b	failure share $\times$ (repair – null rate)
Net@ b	b	replay-null – success share $\times$ regressions
Net@ b , prior	b	net@ b with a Beta prior on the repair rate
Full (RRSI)	148 / 60	paired difference on all evolve episodes

Table 20: Evidence compared in Study 2, per candidate, with $b \in \{10, 40, 80\}$; full evaluation costs 148 (retail) or 60 (airline) episodes. The priors have means 0.41 or 0.29 (audited analyst accuracy), 0.7 and 0.2. Every rule decides with RRSI’s selection rule.

so no completed step was sampled again. While the second restart was checked, progress logs displayed the held-out success rates of a few rounds’ harnesses (r2 retail round 10, r2 airline 12, r3 retail 9, r3 airline 15). Nothing was decided from them, and the analysis was fixed before.

Thinking proposer (r4). A fourth trajectory per domain, registered as descriptive, runs the proposer,

analyst, digester and critic with thinking enabled (8,000 tokens). With thinking, a draft took 1–1.6 hours and 4 of the first 5 drafts after round 0 ended at the proposer’s turn limit without a proposal. Retail’s noise band was calibrated at 17.7 points from base evaluations of 89.2%, 78.4% and 89.2% (r1–r3: 1.1–4.0), so its acceptance rule was not comparable; it was stopped after two rounds. Airline ran ten rounds, after which the base and final harnesses were deployed; we compare it with the first ten airline rounds of r1–r3. Of its 20 drafts, 14 ended without a proposal (r1–r3: 2–6), so it measured 6 candidates (13–15) and accepted 2 (3–4). All six measured candidates had a positive deployment gain, on average +3.5 points (r1–r3: +0.2 to +1.0, with 40–54% positive), although with a single-candidate MDE of 8.5 points none is resolved on its own. Because nearly every candidate helped, the rules hardly differed: decision values per round lay between +0.85 (sequential full) and +1.17 (no verification), with full evaluation at +0.92 (r1–r3:

Evidence	b	Registered analysis		Common error check		Accepts	Cost per round episodes / equiv.	Net(10^4) $v = 10$
		D	[90%]	D	[90%]			
None	0	-1.87	[-4.61, +0.25]	-1.87	[-4.61, +0.25]	0.90	0 / 0	-1,904
LLM judge	0	-1.72	[-4.42, +0.28]	-1.72	[-4.42, +0.28]	0.72	0 / 0	-1,763
None + check	10	-	-	-0.58	[-1.94, +0.79]	0.88	12 / 12	-652
Judge + check	10	-	-	-0.43	[-1.61, +0.72]	0.70	12 / 12	-512
Full (RRSI)	all	+0.22	[-0.06, +0.62]	+0.22	[-0.06, +0.62]	0.12	123 / 123	78
Sequential full	$\leq$ all	-	-	+0.15	[-0.05, +0.42]	0.10	65 / 65	75
Sequential sample	$\leq$ 80	-	-	+0.12	[-0.19, +0.51]	0.17	43 / 43	77
Sample	10	-0.34	[-0.99, +0.21]	-0.04	[-0.44, +0.35]	0.30	12 / 12	-69
Replay	10	-0.43	[-1.27, +0.37]	-0.37	[-1.20, +0.45]	0.54	12 / 9	-423
Replay-null	10	-0.05	[-0.26, +0.13]	-0.05	[-0.26, +0.13]	0.08	10 / 8	-69
Net	10	-0.50	[-1.43, +0.23]	-0.07	[-0.57, +0.41]	0.37	11 / 10	-101
Sample	40	+0.04	[-0.38, +0.45]	+0.04	[-0.38, +0.45]	0.28	47 / 47	-6
Replay	40	-1.67	[-4.35, +0.26]	-0.40	[-1.60, +0.78]	0.80	42 / 33	-499
Replay-null	40	-0.07	[-0.35, +0.15]	-0.07	[-0.35, +0.15]	0.10	42 / 34	-125
Net	40	-0.07	[-0.52, +0.36]	-0.04	[-0.50, +0.38]	0.30	44 / 40	-101
Sample	80	+0.08	[-0.35, +0.54]	+0.08	[-0.35, +0.54]	0.26	82 / 82	3
Replay	80	-1.69	[-4.41, +0.27]	-0.40	[-1.59, +0.79]	0.80	47 / 37	-496
Replay-null	80	-0.12	[-0.59, +0.27]	-0.12	[-0.59, +0.27]	0.14	74 / 59	-210
Net	80	-0.02	[-0.49, +0.46]	-0.02	[-0.49, +0.46]	0.27	88 / 81	-119

Table 21: Study 2, first trajectory (r1): decision value D of each rule, the held-out gain of its choice over the incumbent in points per round (keeping the incumbent scores 0), over 40 rounds of both domains, with a 90% bootstrap interval over rounds and tasks (Table 19 gives intervals from blocks of rounds). Registered: RRSI’s harness-error check (reject above 2% errors) only inside full evaluation. Common error check (post hoc): the check applied to every rule’s own episodes; + *check* rules run 10 episodes of each candidate only for the check; sequential rules stop a candidate’s episodes early (post hoc, Appendix N). Accepts, cost and Net(10^4) (Eq. 1 against keeping the incumbent, $N = 10^4$ deployed episodes, a success worth $v = 10$ episodes, in episodes) are for the common check. No candidate was measured in 4 of the 40 rounds. Full: RRSI’s own decision. Best value in bold.

+0.29 to +0.87) and every 90% interval reaching from about 0 to +6 or more. After ten rounds the incumbent gained +9.2 [+3.8, +15.4] points held out over the base harness (r1-r3 after ten rounds: +8.8, +8.8, +2.9) and +20.0 on the evolve set. Thinking raised the loop’s model cost per round by a factor of 3.4–3.9 over r2 and r3. On this one trajectory a stronger proposer drafted fewer but better candidates, which left little for verification to decide; one trajectory cannot separate that from the variation between trajectories.

Sequential rules. Designed post hoc on r1 and registered for r2 and r3, a sequential rule plans N paired episodes per candidate (RRSI’s full evaluation, or 80) and runs them in batches of 10 in random order. After n episodes with mean paired difference m it drops the candidate if its harness errors exceed 2% of N , or if the predictive probability that RRSI’s rule admits it after N episodes is below $\gamma = 0.05$. For that probability the final ΔS is normal with mean m and variance $\left(\frac{N-n}{N}\right)^2 s^2 \left(\frac{1}{n} + \frac{1}{N-n}\right)$, where s^2 is the larger of

the running variance of the paired differences and $S_t(1 - S_t)$; the cost change is held at its running estimate and the novelty count is known. Candidates that are not dropped complete the N episodes and are decided as without stopping; without the predictive stop, sequential full evaluation makes every decision full evaluation makes. With $\gamma = 0.01$ and 0.10, sequential full evaluation gains +0.17 at 81 and +0.13 at 55 episodes per round on r1, and +0.03 at 100 and +0.02 at 68 on r2 and r3 (+0.02 at 81 with $\gamma = 0.05$). Fully evaluating only the candidate the judge prefers gains +0.00 on r2 and r3 at 97 episodes, and a random one +0.03.

Classifying candidates. We call a candidate a code change if it edits the harness’s agent, tool or check modules, and a text change if it edits only the system prompt or skill files. Code changes rarely lengthened replays: 88 of 1,222 replays of code changes went live before the requested step, all for 4 of 31 candidates, and none of 640 replays of text changes did. RRSI’s own component labels rely on file-name patterns that do not match our harness,

and called most code changes prompt edits. The labels also feed RRSI’s record of which kinds of change have been tried, which it uses to point the proposer at untried kinds and in the novelty term of its selection rule, so the loop explored as RRSI would on a harness whose files its patterns do not recognise; one candidate declared as a harness-side tool was labelled a configuration change. For r2 and r3 the domains’ signals see git’s python diff driver and label code in the agent module or in a new harness module as control flow; on r1’s 47 candidates this gives 24 control-flow and 18 prompt labels instead of 2 and 38, and no code change is labelled prompt only. With these labels the 107 candidates of r2 and r3 were labelled control flow 44 times, prompt 32, rendering of tool results 26, skill 10, harness-side tool 4, extra model call 4, context 3 and configuration 2 (a candidate can carry several labels). As on r1, code changes rarely lengthened replays (316 of 3,236, for 12 of 75 candidates), and replays of text changes never did.

Planned and actual cost. The registered estimate was, per domain, the RRSI trajectory (three base evaluations and up to 40 candidate evaluations), about 2,000–2,700 targeted replays and about 48 harnesses deployed on 160 held-out episodes each: about 30,000 episodes in all, plus the search and judge models, about \$70. The study used 21,151 episodes (retail 12,428, airline 8,723): 5,556 in RRSI’s evaluations, 3,515 replays and 12,080 held-out episodes, and about \$37 at third-party list prices, \$10 of it for DeepSeek-V4-Pro. Only 47 of 80 drafts were measured, so fewer replays and deployments were needed than planned. r2 and r3 were quoted at about 42,000 episodes and \$75, with a stop at \$110. They used 46,584 episodes (11,480 in RRSI’s evaluations, 7,824 replays and 27,280 held-out episodes) and about \$83, \$20 of it for DeepSeek-V4-Pro.

O Study 2: Further Results on the First Trajectory

Registered endpoints. For r1 the registered primary endpoints are (P1) transfer, the held-out gain of the final harness over the base, and (P2) D at $b = 40$, with three one-sided contrasts, Holm-adjusted at $\alpha = 0.05$: net@40 against full evaluation (non-inferiority, margin 0.75 points per round), net@40 against no verification, and full evaluation against no verification. Its bootstrap resamples rounds and held-out tasks within domain. The regis-

tered 160 held-out episodes per harness were raised to 240 after six rounds, as the registration allowed (Appendix N); 160 is a sensitivity analysis.

P2, as registered. Full evaluation had the highest decision value, +0.22 points per round $[-0.06, +0.62]$ (Table 21, Appendix N); sample@40 and sample@80 also had small positive point estimates. Net@40, at a third of the cost, had -0.07 ; the non-inferiority difference was $-0.28 [-0.77, +0.17]$, one-sided $p = 0.058$ (Holm 0.17), so neither non-inferiority nor inferiority was shown. Neither rule was shown better than no verification (full +2.09, net@40 +1.80; Holm $p = 0.17$). With the registered 160 held-out episodes the non-inferiority difference was $-0.48 [-1.03, +0.01]$, $p = 0.185$. The priors moved net@40 by at most 0.01.

The broken candidate and a common error check. Most of the losses of no verification, the judge and replay without null replays come from one candidate (retail, round 6): a check before item-changing calls that reads an attribute the harness never sets, so the harness raised an error in half of its evolve episodes and held-out success fell by 51.7 points. RRSI’s critic accepted it and the judge predicted a gain of 1.5 points; 6 of its 48 replays still succeeded, which replay@ b counts as repairs, while 15 of 48 null replays succeeded. The registered analysis gave RRSI’s error check to full evaluation only. To compare evidence rather than error handling, we apply the check to every rule’s own episodes and add two baselines that run ten episodes of each candidate only for the check, then accept a passing candidate at random or by the judge (post hoc; Tables 21 and 19). Every rule that runs episodes then rejects the broken candidate. Full evaluation still has the highest value; its differences from sample@40, net@40 and replay–null@40 are +0.18 $[-0.22, +0.58]$, +0.26 $[-0.20, +0.75]$ (non-inferiority $p = 0.049$, Holm 0.15) and +0.29 $[-0.06, +0.72]$, and these three differ from each other by at most 0.11. Accepting what passes the execution check loses 0.58 points per round $[-1.94, +0.79]$ (0.43 with the judge), 0.80 $[-0.67, +2.19]$ less than full evaluation: beyond crashes, most candidates still hurt.

Dependence and resolution. Retail’s 20 rounds share 3 incumbents and airline’s 4. Resampling blocks of rounds that share an incumbent widens the standard errors of these contrasts by a factor of

1.3–1.9 (full minus net@40: $[-0.27, +1.00]$), and leaving out any one incumbent moves them by at most 0.20 points. With 80% power at a one-sided 5% level, the design detects per-round differences of 0.60–0.74 points between full evaluation and the @40 rules (0.78–1.11 by blocks), 0.29–0.52 among the @40 rules and about 2 against the execution-only baseline; a single candidate’s gain is detected only above about 5.3 points (median standard error 2.1). Apart from the broken candidate, gains were too small to replicate: the split-half correlation of candidates’ held-out gains was 0.76 with it and -0.30 without it. Correlations with deployment gain were 0.85 for RRSI’s evolve difference (0.33 without that candidate), 0.03 (0.08) for the judge and 0.25 (0.26) for replays.

Spending fewer episodes (post hoc, exploratory on r1). Fully evaluating only the candidate the judge prefers gains $+0.14$ points per round at 94 episodes, as does a random one. Sequential rules run a candidate’s evaluation in batches of ten in random order and drop it once its errors exceed the check or the predictive probability that RRSI’s rule will admit it falls below 0.05 (Appendix N). Sequential full evaluation gains $+0.15$ $[-0.05, +0.42]$ at 65 episodes per round, 0.07 $[-0.03, +0.24]$ below full evaluation at about half its cost. Sequential sampling capped at 80 episodes gains $+0.12$ $[-0.19, +0.51]$ at 43, against $+0.04$ for fixed sample@40 at 47.

Cost. On r1, full evaluation cost 123 episodes per round and the @40 rules 33–47 episode equivalents, because a replay used about 0.8 of an episode’s tokens. Weighting tokens by list prices changes no ranking; it adds the judge’s calls, on the larger model, at about 7 episodes per round. Against keeping the incumbent, full evaluation’s Net(10^4) at $v = 10$ is $+78$ episodes $[-183, +466]$, positive in 58% of resamples; its break-even horizon of about 6,000 episodes is a point estimate, and at $v = 1$ no rule repays its evidence within 10^4 episodes. r1 used about 21,200 episodes and \$37 (Appendix N).

P Study 2 on AppWorld (E3)

Design. Registered before any episode of its loop, E3 runs the loop, evidence and analysis of r2 and r3 on AppWorld 0.1.3 (Trivedi et al., 2024). The agent is DeepSeek-V4-Flash at temperature 0 with AppWorld’s ReAct prompt; each step is one assistant turn whose first Python block runs in a persistent

Evidence	D [90%]	Ep.	— full [90%]	p (Holm)
Full (RRSI)	$+0.03$ $[-0.30, +0.71]$	174	—	—
Sequential full	$+0.08$ $[-0.19, +0.72]$	108	$+0.05$ $[-0.00, +0.14]$	<0.001 (<0.001)
Sequential sample@80	-0.01 $[-0.22, +0.31]$	58	-0.04 $[-0.47, +0.14]$	0.019 (0.019)
Sample@40	-0.04 $[-0.38, +0.41]$	58	-0.07 $[-0.49, +0.15]$	0.018 (0.053)
Net@40	-0.08 $[-0.46, +0.37]$	53	-0.11 $[-0.59, +0.18]$	0.025 (0.053)
Replay—null@40	$+0.22$ $[-0.20, +0.79]$	48	$+0.19$ $[-0.47, +0.67]$	0.023 (0.053)
Sample@80	-0.05 $[-0.38, +0.44]$	116	—	—
None + check	-0.02 $[-1.10, +1.35]$	14	—	—
Judge + check	$+0.18$ $[-0.86, +1.51]$	14	—	—

Table 22: E3 (AppWorld, 20 rounds): decision value D under the common error check in points per round, episodes per round, and the registered non-inferiority contrasts with full evaluation (margin 0.75, one-sided p , Holm within each family), with 90% intervals from the block bootstrap.

interpreter that calls the apps’ APIs. Evolve tasks are the first 20 training scenarios (60 tasks, two trials each) and held-out tasks are test_normal (168 tasks, scenarios disjoint from training); every candidate and incumbent is deployed on two trials per held-out task and the base and final harnesses on four. A task succeeds when all of its unit tests pass. The judge’s prompt calls the agent a code agent; nothing else differs. Base evaluations gave 88.3%, 90.0% and 89.2%, a noise band of 2.4 points. The loop ran $T = 20$ rounds, drafted 40 candidates and measured 29 (16 changed code), and accepted three, two within the band. No candidate’s harness errors exceeded the check. The study used 15,472 episodes and about \$48.

Results. Table 22. Accepting every candidate gained -0.02 points per round and the judge’s choice $+0.18$; candidates’ gains exceeded the single-candidate detectable effect of 3.2 points for 1 of 29, and their split-half correlation was 0.36. Under r1’s registered analysis, with its bootstrap over rounds, net@40 was non-inferior to full evaluation (Holm $p = 0.036$). Net(10^4) at $v = 10$ was negative for every rule (-160 episodes for full evaluation, -39 for sequential full evaluation).

Q Margins, Decision Rules, Calibration and Loop Cost

The analyses of this appendix are post hoc and use no new episodes: they recompute the registered decision values from the same evidence and deployments (verify/posthoc_r7.py, verify/posthoc_r8.py).

Smaller margins and agreement. Table 23 repeats the non-inferiority tests against full evaluation at margins of 0.1, 0.25 and 0.5 points per round,

adds keeping the incumbent as a rule with zero cost and zero value, and reports how often each rule makes full evaluation’s choice (for keeping the incumbent, how often full evaluation keeps it). At the registered margin every rule that runs solved tasks passes, and so does keeping the incumbent; at 0.25 points only sequential full evaluation passes on r2+r3 (sequential sampling: $p = 0.056$), and on r1, the design set, it passes from 0.5. On r4, where full evaluation gained, keeping the incumbent fails at every margin and the sequential rules pass at 0.25. r4’s tests rest on ten rounds, two acceptances and three incumbent blocks and are not adjusted for the number of margins and rules; failing non-inferiority does not show that full evaluation beats keeping the incumbent there, and we treat r4 as descriptive.

Agreement on acceptances. Agreement counts every round, and full evaluation keeps the incumbent in 74% of the rounds of r2+r3, where sequential full evaluation, which only stops candidates early, cannot differ from it. Table 24 conditions on full evaluation’s decision. Sequential full evaluation made 74% [67, 81] of full evaluation’s 21 acceptances on r2+r3 (73% of 26 on r1–r3, 88% of 3 on E3) and never accepted when full evaluation kept the incumbent. It did not select out the unhelpful acceptances: the candidates it would drop gained +0.30 points held out on average, those it keeps +0.11. It lost little because full evaluation’s acceptances were themselves worth little (+0.16 points on average), so the gains it forgoes and the harm it avoids nearly cancel. On r4, whose two acceptances gained +4.6 points, it would drop a quarter of them in expectation and forgo 0.09 points per round; where accepted changes help more, the gain it forgoes grows with them. The fixed-budget rules differ from full evaluation in both directions: they make about half of its acceptances (sample@80: 76%) and accept in 21–34% of the rounds in which it keeps the incumbent, which with what they accept instead costs 0.12–0.14 points per round (replay–null@40: 0.37).

Evidence or decision rule. In the registered analysis full evaluation is RRSI’s own decision, with the noise band calibrated from three base evaluations, while every other rule uses twice its estimate’s standard error, and RRSI admits a candidate within its band if it is cheaper or of an untried kind. Table 25 decides full evaluation (and sequential full evaluation) like the other rules, with its

own band, and then decides every rule by one criterion: admit only if the estimate exceeds twice its standard error and the cost condition holds, with no within-band branch. The judge and no verification have no standard error and are unchanged. With their own band, full evaluation’s value hardly changes. With one criterion every rule that runs episodes accepts a candidate in at most 3% of rounds (replay–null@40: 7%), and its value moves towards zero; replay–null@40 remains below full evaluation decided the same way (−0.21 [−0.47, −0.01]). The evidence’s standard errors are wide relative to candidates’ effects, so whether a rule gains or loses is decided mostly by its within-band acceptances.

Calibration of the bands. Table 26 gives, for each evidence type at 40 episodes, the share of evidence draws whose interval, the estimate $\pm$ twice its standard error, covers the candidate’s full-evaluation ΔS on the evolve set, which the evidence estimates, and its held-out deployment gain. Neither is a nominal coverage of the true effect: full evaluation’s ΔS is itself an estimate, with a standard error of 3–5 points on τ^2 ; the paired samples are subsamples of that same evaluation; and the deployment gain adds held-out noise and the shift from evolve to held-out tasks. We read the table as a consistency check against full evaluation. Paired samples and net evidence are close to 95% on τ^2 and below it on AppWorld (0.81 and 0.70); replay–null covers in 39–62% of draws. Table 27 separates two causes of a shortfall. Replay–null’s draws are always covered around their own mean, and its standard error is about twice their spread; its misses come from an offset: its mean exceeds full evaluation’s ΔS by +3.1 points [+2.5, +3.8] on τ^2 and +1.9 [+1.1, +2.8] on AppWorld, for 78–79% of candidates. Sampling noise in either estimate averages out over candidates, so a mean offset means that replay–null is optimistic relative to full evaluation, as expected of evidence that cannot see harm to solved tasks; the draws resample one collection of replays, which understates that collection’s own variance but does not create the offset. On AppWorld the shortfall of paired samples has the other cause: they have no offset, but their standard error understates the spread of their draws (coverage of their own mean 0.80). Full evaluation’s own band covers the deployment gain in 90–100% of candidates.

Rule	r2+r3: non-inferiority to full evaluation					Agree with full evaluation				p at margin 0.25		
	– full [90%]	0.1	0.25	0.5	0.75	r2+r3	r1	r4	E3	r1	r4	E3
Sequential full	–0.02 [–0.12, +0.07]	0.086	<0.001	<0.001	<0.001	0.93	0.96	0.95	0.98	0.111	0.023	<0.001
Sequential sample@80	+0.03 [–0.26, +0.35]	0.233	0.056	0.002	<0.001	0.78	0.88	0.96	0.89	0.166	0.005	0.136
Sample@80	+0.00 [–0.44, +0.44]	0.330	0.165	0.029	0.004	0.78	0.84	1.00	0.87	0.269	<0.001	0.131
Sample@40	–0.10 [–0.57, +0.35]	0.477	0.280	0.083	0.018	0.67	0.78	0.86	0.78	0.355	0.135	0.170
Net@40	–0.14 [–0.58, +0.27]	0.536	0.309	0.080	0.015	0.63	0.74	0.82	0.68	0.485	0.316	0.264
Replay–null@40	–0.43 [–0.94, +0.05]	0.868	0.728	0.392	0.141	0.68	0.81	0.77	0.69	0.565	0.182	0.113
Keep incumbent	–0.04 [–0.43, +0.35]	0.390	0.170	0.028	0.004	0.74	0.88	0.80	0.85	0.433	0.778	0.202

Table 23: Non-inferiority to full evaluation at smaller margins (one-sided bootstrap p , block bootstrap as in Table 19, not adjusted; 0.75 is the registered margin), and the expected share of rounds in which a rule makes full evaluation’s choice. For keeping the incumbent, Agree is the share of rounds in which full evaluation keeps it. r1 is the exploratory set of the sequential rules; r4 has ten airline rounds and three blocks.

Rule	Recall [90%]	Acc.	Full evaluation’s acceptances			$D(\text{rule}) - D(\text{full})$, points per round			
			Gain kept	Gain dropped	Dropped	Forgone	Avoided	Other	Total
Sequential full	0.74 [0.67, 0.81]	0.00	+0.11	+0.30	5.4	–0.06	+0.04	–0.00	–0.02
Sequential sample@80	0.48 [0.39, 0.56]	0.11	+1.06	–0.68	10.9	–0.10	+0.19	–0.07	+0.03
Sample@80	0.76 [0.66, 0.85]	0.21	+0.81	–1.94	5.0	–0.03	+0.15	–0.12	+0.00
Sample@40	0.55 [0.48, 0.62]	0.28	+0.60	–0.38	9.4	–0.11	+0.16	–0.14	–0.10
Net@40	0.54 [0.48, 0.61]	0.34	+0.27	+0.03	9.6	–0.12	+0.12	–0.13	–0.14
Replay–null@40	0.23 [0.10, 0.35]	0.15	–0.32	+0.30	16.3	–0.28	+0.21	–0.37	–0.43
LLM judge	0.48 [0.30, 0.65]	0.76	–0.12	+0.42	11.0	–0.21	+0.16	–0.38	–0.44
None	0.62 [0.55, 0.69]	0.88	+0.24	+0.03	8.0	–0.11	+0.11	–0.51	–0.51
Keep incumbent	0.00 [0.00, 0.00]	0.00	–	+0.16	21.0	–0.32	+0.28	+0.00	–0.04
Sequential full, r1 (5)	0.69 [0.49, 0.90]	0.00	+1.52	+2.27	1.5	–0.09	+0.00	+0.02	–0.07
Sequential full, r1–r3 (26)	0.73 [0.67, 0.80]	0.00	+0.37	+0.73	7.0	–0.07	+0.03	+0.01	–0.04
Sequential full, r4 (2)	0.74 [0.49, 1.00]	0.00	+5.57	+1.70	0.5	–0.09	–0.00	+0.02	–0.06
Sequential full, E3 (3)	0.88 [0.64, 1.00]	0.00	+0.59	–2.60	0.4	–0.00	+0.05	–0.00	+0.05

Table 24: Agreement with full evaluation’s acceptances on r2+r3 (21 acceptances in 80 rounds; post hoc). Recall: expected share of full evaluation’s acceptances the rule also makes, with a 90% block-bootstrap interval. Acc.: share of the rounds in which full evaluation keeps the incumbent where the rule accepts a candidate. Gain kept and dropped: mean held-out gain (points) of full evaluation’s accepted candidates, weighted by the probability that the rule makes or does not make the same acceptance; Dropped: expected number it does not make. The last four columns split the difference in decision value: gains forgone by dropping helpful acceptances, harm avoided by dropping harmful ones, the value of what the rule accepts instead (another candidate, or one in a round where full evaluation keeps the incumbent), and the total. Lower block: sequential full evaluation on the other trajectories, with full evaluation’s number of acceptances in parentheses.

The loop’s whole cost. Table 28 prices the RRSI loop itself, not this study’s extra evidence or deployments: the proposal side’s model calls (analyst, digester, proposer, critic), the candidates’ full evaluations, smoke tests and the base-harness calibration, at list prices. Candidate evaluation is 24–54% of it on τ^2 and 45% on AppWorld; the episodes sequential full evaluation would have saved are 19% of the loop’s cost on r1–r3 and 17% on AppWorld. With a thinking proposer (r4) the model calls are 93% of the cost and sequential evaluation saves 1%. In the independent loops, where the proposer also saw early-stopped estimates and made different proposals, sequential evaluation ran 39% fewer candidate-evaluation episodes and the loops cost 10% less (Appendix S).

What each rule buys. Every rule uses what the loop already has: the incumbent’s full evaluation, which it ran when it accepted the incumbent, and its failed trials. Sample and sequential rules buy only candidate episodes, paired with the incumbent’s existing trials. Replay rules buy candidate and null replays from recorded failures; null replays are shared by a round’s candidates. Net evidence buys replays and candidate runs on solved tasks. Under sequential evaluation accepted candidates complete their evaluation, which becomes the next incumbent’s, so the next round analyses the same kind of traces as under full evaluation. The proposer’s inputs still change: RRSI shows it the scores of rejected candidates, which become early-stopped estimates, so its later proposals may differ even

Rule	Registered rule		Full evaluation with own band		One criterion, no within-band branch	
	D [90%]	Accepts	D [90%]	Accepts	D [90%]	Accepts
Full (RRSI) / own band	+0.04 [−0.35, +0.43]	0.26	+0.03 [−0.47, +0.54]	0.36	+0.09 [−0.06, +0.36]	0.03
Sequential full / own band	+0.02 [−0.34, +0.35]	0.20	+0.08 [−0.21, +0.41]	0.20	+0.09 [−0.05, +0.35]	0.02
Sequential sample@80	+0.07 [−0.26, +0.41]	0.23	+0.07 [−0.26, +0.41]	0.23	+0.10 [−0.02, +0.35]	0.02
Sample@80	+0.04 [−0.46, +0.57]	0.38	+0.04 [−0.46, +0.57]	0.38	+0.10 [−0.02, +0.36]	0.02
Sample@40	−0.06 [−0.58, +0.45]	0.40	−0.06 [−0.58, +0.45]	0.40	+0.04 [−0.04, +0.17]	0.02
Net@40	−0.10 [−0.62, +0.39]	0.44	−0.10 [−0.62, +0.39]	0.44	−0.00 [−0.04, +0.04]	0.02
Replay—null@40	−0.39 [−0.79, −0.03]	0.20	−0.39 [−0.79, −0.03]	0.20	−0.12 [−0.30, +0.03]	0.07
LLM judge	−0.40 [−1.41, +0.67]	0.81	−0.40 [−1.41, +0.67]	0.81	−0.40 [−1.41, +0.67]	0.81
None	−0.47 [−1.52, +0.62]	0.91	−0.47 [−1.52, +0.62]	0.91	−0.47 [−1.52, +0.62]	0.91

Table 25: Decision value on r2+r3 (common error check, points per round, block bootstrap) under three decision rules. “Own band”: full and sequential full evaluation decided with twice their own standard error, as the other rules already are, so only their rows change. “One criterion”: every rule admits a candidate only if its estimate exceeds twice its standard error and the cost condition holds.

Evidence (@40)	r1	r2+r3	r4	E3
Sample@40	0.98 / 0.94	0.98 / 0.92	1.00 / 0.96	0.81 / 0.80
Net@40	0.95 / 0.94	0.92 / 0.94	0.99 / 0.95	0.70 / 0.83
Replay—null@40	0.54 / 0.76	0.51 / 0.66	0.62 / 0.88	0.39 / 0.67
Replay@40	0.12 / 0.15	0.16 / 0.15	0.32 / 0.00	0.24 / 0.45
Full evaluation	1.00 / 0.94	1.00 / 0.90	1.00 / 1.00	1.00 / 0.97

Table 26: Coverage of ± 2 standard-error intervals: evolve ΔS / deployment gain, averaged over candidates (200 evidence draws each). Full evaluation covers its own ΔS by construction.

Evidence (@40)	Offset [90%]	> 0	SD / SE	Own
τ^2 , $r1-r4$ (159–160 candidates)				
Sample@40	−0.0 [−0.1, +0.0]	0.46	4.1 / 5.8	0.98
Net@40	+1.2 [+0.8, +1.7]	0.70	5.6 / 6.7	0.97
Replay—null@40	+3.1 [+2.5, +3.8]	0.78	1.1 / 2.1	1.00
Replay@40	+6.6 [+6.0, +7.3]	0.96	0.2 / 1.2	1.00
<i>AppWorld</i> (29 candidates)				
Sample@40	−0.0 [−0.0, +0.0]	0.48	2.5 / 2.8	0.80
Net@40	+0.8 [+0.4, +1.1]	0.72	3.0 / 2.8	0.81
Replay—null@40	+1.9 [+1.1, +2.8]	0.79	0.1 / 0.6	1.00
Replay@40	+2.4 [+1.7, +3.3]	0.93	0.0 / 0.5	1.00

after rounds whose decisions agree; we have not measured this. The fixed-budget rules would need an extra full evaluation of each accepted candidate to give the next round the same traces; our counts omit it, which favours them.

Change cost. Net(N) sets $\Delta C_{\text{change}} = 0$ because every rule sees the same candidates, a simplification: reviewing and maintaining an accepted change costs something, and rules accept different numbers of changes. Charging κ episodes per accepted change lowers a rule’s Net(10^4) by κ times its acceptance rate. On r2+r3 at $v = 10$, sequential sampling, sequential full evaluation and full evaluation have +10 [−315, +350], −71 [−438, +254] and −108 [−500, +274] episodes at $\kappa = 0$ and −215, −267 and −370 at $\kappa = 1,000$; the fixed-budget rules, which accept 38–44% of rounds, fall further.

Registration timeline. All times UTC, 2026, from the repository history. 7 Oct 11:26: r1’s registration, analysis code and launch script committed; the loop and its held-out deployments started after it. 15:34–15:54: the registered power rule raised held-out trials (Appendix N). 20:22: r1 finished; 20:23 its final disagreement rate logged. 8 Oct

Table 27: Offset of each evidence type at 40 episodes from full evaluation’s evolve ΔS : the mean over candidates of the mean of 200 draws minus ΔS , in points, with a 90% bootstrap interval over candidates; the share of candidates with a positive offset; the draws’ standard deviation against their mean reported standard error (points); and the share of draws whose ± 2 SE interval covers the draws’ own mean (post hoc).

00:24: r2 and r3 registered, before any of their episodes; 00:45: addendum (thinking roles for r4, verify-one-candidate rules). 01:00: E3 registered, before any episode of its loop. 02:06: the sequential rules and their tests registered for r2, r3 and E3 (addendum 2); no decision value of r2, r3 or E3 had been computed. 08:01: r2 and r3’s analysis code committed, tested on r1 only. 09:55: every round of r2 and r3 deployed; 10:00–10:15 their registered analyses run as committed. 16:32 and 18:02: E3’s loop and deployments finished; 18:14 its registered analysis run. 9 Oct 08:32: the closed loops cl1 and cl2 registered, before any of their episodes; 08:38: their analysis code; 14:07: their deployments finished; 14:08 their registered analysis run. 14:43: IL registered with its analysis code, before any of its episodes, and its eight loops launched; 17:28: its last deployment and shadow evaluation finished;

Trajectory	Loop \$	Model calls %	Cand. eval. %	Smoke, calib. %	Sequential episodes	full saves % of loop
r1 airline	6.52	69	27	4	793	15
r1 retail	10.73	47	46	6	1,536	20
r2 airline	6.92	64	32	4	437	8
r2 retail	11.92	41	53	6	2,361	32
r3 airline	7.10	72	24	3	392	7
r3 retail	11.83	40	54	6	1,604	21
r4 airline	9.39	93	5	2	93	1
E3 AppWorld	18.58	52	45	3	1,318	17

Table 28: Dollar cost of the RRSI loop per trajectory and domain (r4 retail, stopped after two rounds, omitted), its shares, and the evolve episodes sequential full evaluation would have saved, priced at the mean price of a candidate-evaluation episode.

18:01: its registered analysis run. The operations logs in the registration files record the restarts, the r4 truncation and the held-out rates seen in progress logs.

R Validating the Stopping Rule

The analyses of this appendix are post hoc and use no new episodes (verify/posthoc_r9.py).

Calibration. For every measured candidate of r1–r4 and E3 we replay sequential full evaluation ($\gamma = 0.05$, batches of 10) in 400 random orders of its full evaluation’s episodes and record, at every interim look, the predicted probability p that RRSI’s rule admits the candidate after all N episodes, and whether it stops there; the outcome is full evaluation’s decision. In the six live loops (cl1, cl2, s1–s4) each candidate has the one order it ran, and the outcome is the loop’s decision for candidates that completed and the shadow evaluation’s for those it stopped. Table 30 bins the looks by p . The probabilities are too extreme at both ends: on r2+r3, looks with $p \geq 0.95$ were followed by admission in 78% of cases and looks with $p < 0.01$ in 8%. Candidates stopped at a mean p of 0.02 were admitted by full evaluation in 8% of cases on r2+r3, 12% on airline, 3% on AppWorld and 13% in the live loops, so γ does not bound the admissible candidates lost: the rule stopped 24% of the candidates full evaluation admitted on r2+r3 and 35% in the live loops. The second pair of columns recomputes p with the cost change of the complete evaluation in place of its running estimate, a diagnostic rather than a usable rule: looks below 0.05 are then followed by admission in at most 2% of cases, so most of the error at the low end comes from the cost term. The error at the high end remains.

A stopping rule that models the cost change (post hoc). The diagnostic above uses the complete evaluation’s cost change and cannot be run. A rule that can treat the final cost change as normal around its running estimate, with variance $(\frac{N-n}{N})^2 s_c^2 (\frac{1}{n} + \frac{1}{N-n})$ from the per-pair variance s_c^2 of the token difference relative to the incumbent’s mean, independent of the final ΔS , and integrates RRSI’s admission regions over both (verify/posthoc_r10.py). We replay it in the same 400 orders per candidate as the registered rule (Table 29). At $\gamma = 0.05$ it keeps 88% of the candidates full evaluation admits on r2+r3 instead of 75%, 91% instead of 71% on airline and 94% instead of 86% on AppWorld, for 64%, 77% and 67% of full evaluation’s episodes instead of 57%, 65% and 62%. Its stopped candidates are admitted by full evaluation in 4%, 5% and 1% of cases, against 8%, 12% and 3%, and on r2+r3 it stops 14% and 15% of the candidates admitted inside the band for cost and for novelty, against 32% and 35%. Per round it makes 88% of full evaluation’s acceptances on r2+r3 (registered rule 74%), 89% on airline (69%) and 96% on AppWorld (88%), and its decision value differs from full evaluation’s by +0.00, −0.02 and +0.02 points per round. At $\gamma = 0.10$ it keeps 79% at 54% of the episodes on r2+r3, more than the registered rule at fewer episodes. Never stopping for cost, that is stopping only when the final ΔS is unlikely to reach RRSI’s floor, keeps 96–100% but saves little: 77% of the episodes on r2+r3 and 97% on airline. The correction removes about half of the lost acceptances offline; it has not been run in a loop.

By branch. Table 31 splits candidates by the branch of RRSI’s rule that decided their complete evaluation. Sequential evaluation stops most rejected candidates early and rarely those admitted above the band, but about a third of those admitted inside it. Inside the band RRSI’s shaped score is $w_s \Delta S - 15 \Delta C + 0.5 \nu$ with $w_s = 0$ in RRSI’s configuration, so a candidate without a new component is admitted exactly when its token cost falls; the rule holds that cost change at its running estimate, whose sign after 10–30 episodes is often wrong.

Thresholds. Table 32 varies γ from 0.005 to 0.5. On r2+r3 the episodes rise from 22% to 74% of full evaluation’s and the share of its acceptances kept from 26% to 86%; keeping 85% needs $\gamma =$

Stopping rule	τ^2 r2+r3			τ^2 airline r1-r4			AppWorld		
	Stop adm.	Kept	Ep.	Stop adm.	Kept	Ep.	Stop adm.	Kept	Ep.
Registered (cost held)	8%	75%	57%	12%	71%	65%	3%	86%	62%
Cost-aware, $\gamma = 0.05$	4%	88%	64%	5%	91%	77%	1%	94%	67%
Cost-aware, $\gamma = 0.10$	7%	79%	54%	8%	84%	69%	2%	89%	59%
Cost-aware, $\gamma = 0.20$	11%	62%	41%	12%	71%	58%	6%	71%	45%
Never stop for cost	2%	96%	77%	6%	99%	97%	0%	100%	94%

Table 29: Sequential full evaluation with the cost change held at its running estimate (registered) or with its uncertainty modelled (cost-aware), replayed in 400 random orders of each candidate’s episodes (post hoc). Stop adm.: share of stopped candidates that full evaluation admits; Kept: share of the candidates full evaluation admits that are not stopped; Ep.: episodes as a share of full evaluation’s. Never stop for cost: stop only when the final ΔS is predicted to fall below RRSI’s floor.

Predicted p	r2+r3		r2+r3, final ΔC		Airline r1-r4		AppWorld		Live loops	
	mean p	admitted	mean p	admitted	mean p	admitted	mean p	admitted	mean p	admitted
[0.00, 0.01)	0.003	0.08	0.003	0.00	0.003	0.10	0.004	0.03	0.004	0.22
[0.01, 0.05)	0.028	0.08	0.028	0.02	0.026	0.14	0.030	0.03	0.027	0.08
[0.05, 0.20)	0.12	0.12	0.12	0.07	0.11	0.17	0.10	0.05	0.12	0.18
[0.20, 0.50)	0.34	0.21	0.34	0.19	0.33	0.25	0.32	0.10	0.35	0.16
[0.50, 0.80)	0.64	0.36	0.64	0.39	0.65	0.38	0.65	0.38	0.65	0.51
[0.80, 0.95)	0.88	0.52	0.88	0.62	0.88	0.52	0.89	0.66	0.87	0.64
[0.95, 1]	0.98	0.78	0.98	0.89	0.99	0.74	0.98	0.84	0.98	0.74
Stopped: mean p / admitted	0.021	0.08	–	–	0.014	0.12	0.025	0.03	0.018	0.13
Stopped: selected by full		0.07	–	–		0.10		0.02		0.10
Admitted by full: stopped		24% of 24	–	–		28% of 23		15% of 5		35% of 23

Table 30: Calibration of sequential full evaluation’s predicted admission probabilities (post hoc). Rows bin the interim looks by p ; “admitted” is the share of those looks whose candidate full evaluation admitted. Recorded trajectories: 400 orders per candidate. Live loops: cl1, cl2 and s1–s4, outcomes from the loop or its shadow evaluation. “Final ΔC ”: p recomputed with the complete evaluation’s cost change (a diagnostic). Bottom rows: stopped candidates (mean p at the stop, share admitted and selected by full evaluation) and the share of full evaluation’s admitted candidates that were stopped.

Full evaluation’s branch	τ^2 r1-r4		AppWorld		Live loops	
	Cand.	Stopped	Cand.	Stopped	Cand.	Stopped
Above the band	11	0.04	2	0.01	2	0.00
In band, cheaper	20	0.33	2	0.19	16	0.38
In band, new component	5	0.38	1	0.33	5	0.40
In band, rejected	53	0.81	13	0.93	54	0.91
Below the floor	71	0.91	11	0.99	9	0.44

Table 31: Share of candidates sequential full evaluation stops (statistical stops; expected over 400 orders for recorded trajectories), by the branch of RRSI’s rule that decided the complete evaluation.

0.01 and 71% of the episodes. The decision values hardly move, because full evaluation’s own is near zero; on r1 they rise with the share kept, toward full evaluation’s +0.22.

Matched cost. Table 33 sets the rules on r2+r3 against their episodes. At sequential full evaluation’s episodes, fixed sampling (sample@60) keeps as many of full evaluation’s acceptances, 75% against 74%, but also accepts in 21% of the rounds in which full evaluation keeps the incum-

bent, which a rule that only stops evaluations never does. Fully evaluating one candidate drawn at random keeps 61% at 69% of the episodes. The decision values differ by less than 0.05 points per round, because full evaluation’s acceptances were worth little. On AppWorld sequential full evaluation keeps 88% of full evaluation’s acceptances at 62% of its episodes, fixed sampling 55–64% at 50–67%, and one random candidate 47% at 66%.

Leaving out trajectories. Table 34 repeats the non-inferiority tests with one trajectory and domain left out at a time. Sequential full evaluation stays non-inferior at 0.25 points in every case ($p \leq 0.006$). Sample@40 and net@40 keep $p \leq 0.063$ at the registered margin of 0.75 points, below 0.05 except when r3’s airline rounds are left out; replay–null@40 fails in every case on r2+r3.

Independent evidence. Table 35 counts the units behind each analysis. The block bootstrap of the primary analysis resamples 23 incumbent blocks

γ	r2+r3			r1			E3 (AppWorld)			r2+r3 p at 0.25
	Ep. share	Kept	D —full	Ep. share	Kept	D —full	Ep. share	Kept	D —full	
0.005	0.74	0.86	−0.01	0.70	0.79	−0.05	0.79	0.95	+0.02	<0.001
0.01	0.71	0.85	−0.02	0.65	0.78	−0.05	0.74	0.93	+0.03	<0.001
0.02	0.65	0.80	−0.02	0.60	0.75	−0.06	0.68	0.90	+0.04	<0.001
0.05	0.57	0.74	−0.02	0.53	0.69	−0.07	0.62	0.88	+0.05	<0.001
0.1	0.48	0.65	−0.02	0.44	0.62	−0.09	0.49	0.80	+0.07	0.006
0.2	0.38	0.52	−0.02	0.34	0.53	−0.09	0.44	0.73	+0.06	0.022
0.3	0.32	0.43	−0.01	0.30	0.45	−0.10	0.34	0.62	+0.05	0.033
0.5	0.22	0.26	+0.01	0.22	0.33	−0.11	0.20	0.34	+0.01	0.045

Table 32: Sequential full evaluation across thresholds γ (post hoc): candidate-evaluation episodes as a share of full evaluation’s, the expected share of full evaluation’s acceptances it makes (Kept), its decision value minus full evaluation’s (points per round), and the one-sided bootstrap p of non-inferiority at a margin of 0.25 points on r2+r3.

Rule (r2+r3)	Ep.	Kept	Acc. keep	D	—full [90%]	$p_{0.25}$
Full evaluation	141	1.00	0.00	+0.04	—	—
Sequential full ($\gamma = 0.05$)	81	0.74	0.00	+0.02	−0.12, +0.07	<0.001
Full, one random candidate	97	0.61	0.00	+0.03	−0.21, +0.19	0.029
Sample@10	13	0.35	0.32	−0.08	−0.58, +0.34	0.299
Sample@20	27	0.47	0.33	−0.07	−0.61, +0.36	0.303
Sample@30	40	0.51	0.30	−0.06	−0.57, +0.35	0.279
Sample@40	54	0.55	0.28	−0.06	−0.57, +0.35	0.280
Sample@50	67	0.60	0.25	−0.01	−0.51, +0.38	0.225
Sample@60	80	0.75	0.21	+0.05	−0.43, +0.45	0.161
Sample@80	94	0.76	0.21	+0.04	−0.44, +0.44	0.165
Sample@100	108	0.79	0.19	+0.05	−0.40, +0.43	0.143
Sample@120	122	0.81	0.18	+0.05	−0.38, +0.43	0.137
Sample@150	141	0.86	0.17	+0.03	−0.41, +0.42	0.165

Table 33: Rules on r2+r3 by their candidate-evaluation episodes per round (post hoc). Kept: expected share of full evaluation’s acceptances the rule makes; Acc. | keep: how often it accepts when full evaluation keeps the incumbent; D and its difference from full evaluation in points per round with a 90% block-bootstrap interval; $p_{0.25}$: non-inferiority at 0.25 points. Sample@ b decides with twice the sample’s standard error; at $b \geq 60$ on airline it uses every pair.

	Seq. full		Sample@40		Net@40		Replay—null@40	
	—full	$p_{0.25}$	—full	$p_{0.75}$	—full	$p_{0.75}$	—full	$p_{0.75}$
Left out								
r2+r3 without r2 retail	−0.02	0.001	+0.08	<0.001	+0.03	0.002	−0.42	0.171
r2+r3 without r2 airline	−0.01	<0.001	−0.13	0.035	−0.09	0.030	−0.27	0.051
r2+r3 without r3 retail	−0.04	0.005	−0.17	0.043	−0.25	0.049	−0.46	0.149
r2+r3 without r3 airline	−0.02	0.004	−0.18	0.060	−0.24	0.063	−0.58	0.289
r1−r3 without r1 retail	−0.05	0.004	−0.09	0.004	−0.16	0.009	−0.44	0.119
r1−r3 without r1 airline	−0.01	<0.001	−0.13	0.011	−0.16	0.010	−0.37	0.069
r1−r3 without r2 retail	−0.04	0.004	−0.03	<0.001	−0.08	<0.001	−0.37	0.092
r1−r3 without r2 airline	−0.04	0.002	−0.15	0.015	−0.16	0.016	−0.28	0.029
r1−r3 without r3 retail	−0.05	0.006	−0.17	0.013	−0.25	0.025	−0.39	0.081
r1−r3 without r3 airline	−0.04	0.004	−0.18	0.019	−0.25	0.028	−0.47	0.138

Table 34: Leave one trajectory and domain out (post hoc): difference from full evaluation in points per round and one-sided bootstrap p of non-inferiority (block bootstrap, not adjusted).

Trajectory	Rounds	Cand.	Full acc.	Blocks	Evolve	Held out	Cand. ep.
r1 retail	20	24	2	3	74 × 2	40 × 6	3,552
r1 airline	20	23	3	4	30 × 2	20 × 12	1,380
r2 retail	20	27	3	4	74 × 2	40 × 6	3,996
r2 airline	20	29	6	6	30 × 2	20 × 12	1,740
r3 retail	20	28	6	6	74 × 2	40 × 6	4,144
r3 airline	20	23	6	7	30 × 2	20 × 12	1,380
r4 airline	10	6	2	3	30 × 2	20 × 12	360
E3 AppWorld	20	29	3	4	60 × 2	168 × 2	3,480

Table 35: Size of the independent evidence per trajectory and domain: rounds, measured candidates, full evaluation’s acceptances, incumbent blocks (the units of the block bootstrap), evolve and held-out tasks × trials per candidate, and candidate-evaluation episodes.

of r2+r3, which hold 21 acceptances, and held-out sets of 40 and 20 tasks; the 1,400 to 4,100 candidate-evaluation episodes per trajectory and domain are repetitions within these units. Narrower intervals need more trajectories and more held-out tasks rather than more trials per task.

S Loops Driven by Sequential Evaluation

Two registered experiments ran RRSI with sequential full evaluation as its selection step: CL (two loops, descriptive) and IL (eight concurrent loops, four with each selection step, and the registered comparison of the main text).

CL design. Registered after every analysis above and before any of its episodes (verify/PREREGISTRATION_CL.md), two new airline trajectories, cl1 and cl2, ran rounds 0–9 of RRSI with its selection step replaced by sequential full evaluation ($\gamma = 0.05$, batches of

10; Appendix N); everything else is as for r2 and r3, with $T = 20$ so that the edit-budget schedule matches r1–r3’s first ten rounds. A dropped candidate enters RRSI’s history as rejected with its early-stopped estimate, which the proposer sees in later rounds. The base, every accepted incumbent and the final incumbent were deployed on the 20 test tasks, 12 trials each. After round 9, every dropped candidate’s evaluation was completed to all 60 episodes in a separate job (the shadow evaluation) and decided by RRSI’s rule with that round’s S^* , noise band and novelty. The reference is the first ten airline rounds of r1–r3. All outcomes are descriptive; the registered consistency

Traj.	Selection	Accepted	Cand. episodes	Loop \$	Transfer	90% CI
r1	full	3 (3)	780	\$3.42	+8.75	[+2.50, +15.42]
r2	full	3 (2)	900	\$3.46	+8.75	[+3.75, +14.17]
r3	full	4 (4)	780	\$3.49	+2.92	[+0.42, +5.42]
cl1	seq.	1 (1)	460 (48%)	\$3.20	+0.42	[-4.58, +4.58]
cl2	seq.	5 (4)	760 (79%)	\$3.68	+12.08	[+5.81, +19.17]

Table 36: Loops on airline, rounds 0–9: r1–r3 driven by full evaluation, cl1 and cl2 by sequential full evaluation. Accepted changes (in parentheses, those inside the noise band), candidate-evaluation episodes (share of the planned episodes), the loop’s dollar cost at list prices (Table 28), and transfer at round 10: held-out success of the incumbent after round 9 minus the base, in points, with a 90% task-bootstrap interval.

criterion asks both trajectories’ transfer to lie in r1–r3’s range widened by their 90% intervals, $[+0.42, +15.42]$ points.

CL results. Both trajectories meet the criterion, cl1 only at its lower edge (Table 36). Their noise bands, calibrated from three base evaluations, were 12.5 and 7.2 points (r1–r3: 17.8, 5.4 and 9.4). cl1 accepted one change in ten rounds: sequential evaluation dropped 14 of its 16 candidates, and the shadow evaluation agrees with 12 of those drops. The accepted change (round 6) lowered evolve success from 71.7% to 66.7% and was admitted inside the band for a new component; held out it gained +0.42 points. Full evaluation would also have accepted round 3’s first candidate (evolve –6.7 points, admitted inside the band for a new component) and round 9’s (+1.7); deployed post hoc, they gained +3.75 $[-0.8, +9.2]$ and +4.17 $[-0.8, +9.6]$ points over their incumbents (90% intervals), which suggests that cl1’s low transfer came partly from what sequential evaluation dropped. Both would have been accepted inside the band, for a new component and for a lower token cost, and both had started worse or barely better than the incumbent: sequential evaluation dropped them after 10 episodes at –10 points and after 40 at +2.5. cl2 accepted five changes, four inside its band, each of which gained held out (+0.4 to +3.75 points), and its final harness gained +12.1, the most of the five trajectories. Sequential evaluation dropped 8 of its 16 candidates; one of them (round 2) full evaluation would also have admitted, but not chosen. Over both trajectories, full evaluation as the shadow evaluation scores it would have made the sequential loop’s decision in 18 of 20 rounds, and the sequential loop made 6 of its 8 acceptances, close to the 93% and 74% measured offline on r2 and r3.

CL cost. Over rounds 0–9 the loops cost \$3.20 and \$3.68 at list prices, against \$3.42–\$3.49 for r1–r3. Candidate evaluation fell to \$0.59 and \$1.00 from \$0.98–\$1.16, but cl1 and cl2 measured 16 candidates each, against 13–15, and their proposal-side model calls cost \$2.38 and \$2.43, against \$2.06–\$2.27. The experiment, deployments and shadow evaluations included, cost about \$11.

CL changes after registration. The container restarted during round 7 of both trajectories; both resumed from their checkpoints two hours later, reusing every episode on disk and replaying the stopping order from the same seed. To make up the time, deployments and shadow evaluations ran three at a time. Both changes were made before any held-out result was seen. The deployment of cl1’s two forgone acceptances was decided after the registered analysis and is post hoc. The operations log of the registration records each change.

IL design. Registered after CL’s results and before any of its episodes (verify/PREREGISTRATION_IL.md), eight new airline loops ran rounds 0–9 concurrently, each in its own checkout of one commit: four with sequential full evaluation (s1–s4, as cl1 and cl2) and four with RRSI’s own full evaluation (f1–f4); everything else is as for CL. The base harness and the incumbent after round 9 of every loop were deployed on the 20 test tasks, 12 trials each, and the sequential loops’ stopped candidates were completed in shadow evaluations. The registered comparison pools them with cl1 and cl2 (sequential) and r2 and r3 (full; their incumbent after round 9 had been deployed for CL), six loops per arm. Its outcomes are transfer at round 10, the loop’s dollar cost over rounds 0–9 and its candidate-evaluation episodes, each as sequential minus full with a 90% interval from a bootstrap over loops within arm and, for transfer, over held-out tasks ($B = 2000$), and the one-sided bootstrap probability that the transfer difference is below –5 points. Before any episode we noted that six loops per arm give about 55% power at that margin, so failing it would be uninformative. Sensitivity analyses use the eight concurrent loops only and add r1 to the full arm. No held-out success rate of IL was examined before every deployment had finished.

IL results. Table 37 and Figure 2. Transfer after ten rounds was +0.4 to +12.1 points in the

Loop	Selection	Accepted	Cand. episodes	Loop \$	Transfer
r2	full	3	900	\$3.46	+8.75
r3	full	4	780	\$3.49	+2.92
f1	full	2	1080	\$3.72	+9.58
f2	full	4	600	\$3.04	+9.58
f3	full	5	960	\$3.71	+12.08
f4	full	5	960	\$3.78	+9.17
cl1	seq.	1	460	\$3.20	+0.42
cl2	seq.	5	760	\$3.68	+12.08
s1	seq.	4	710	\$3.17	+5.00
s2	seq.	0	330	\$2.83	+0.00
s3	seq.	1	390	\$3.07	+6.25
s4	seq.	3	580	\$3.17	+1.67
Seq.—full (6 vs 6)			−342	−0.35	−4.44
90% interval			[−483, −203]	[−0.57, −0.10]	[−9.44, −0.14]

Table 37: The registered comparison of independent airline loops, rounds 0–9: accepted changes, candidate-evaluation episodes, the loop’s dollar cost at list prices, and transfer at round 10 (held-out success of the incumbent after round 9 minus the base, points). Intervals: bootstrap over loops within arm and, for transfer, held-out tasks. f1–f4 and s1–s4 ran concurrently from one commit.

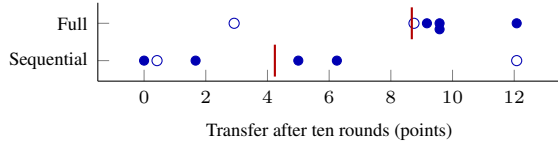


Figure 2: Transfer at round 10 of the twelve airline loops of the registered comparison (held-out success of the incumbent after round 9 minus the base, 240 episodes each). Filled: the eight loops run concurrently from one commit; open: cl1, cl2, r2, r3. Bars: arm means, +4.2 and +8.7; difference −4.4 [−9.4, −0.1].

sequential loops (mean +4.2, standard deviation 4.6) and +2.9 to +12.1 in the full-evaluation loops (mean +8.7, standard deviation 3.1): a difference of −4.4 [−9.4, −0.1] points (Welch standard error 2.25), with probability 0.38 of lying below −5, so non-inferiority at 5 points is not shown. Among the eight concurrent loops the difference was −6.9 [−11.7, −2.6]; every full-evaluation loop ended above every sequential one (descriptive). With r1 in the full arm it was −4.5 [−9.5, −0.3]. The sequential loops ran 538 candidate-evaluation episodes on average against 880 (−342 [−483, −203]) and cost \$3.19 against \$3.53 (−\$0.35 [−0.57, −0.10]). They accepted 0 to 5 changes, 14 in all, and the full-evaluation loops 2 to 5, 23 in all. In the shadow evaluation of the six sequential loops, full evaluation would have made the same decision in 54 of 60 rounds and 20 acceptances, 14 of which the loops made; of their 62 stopped candidates it would have admitted 8 and selected 6. The concurrent loops ran without restart, retry or change after registration, and IL cost \$34.12.

Matched offline comparison (post hoc). The offline analyses of §5.1 pool retail and airline over twenty rounds, whereas the loops measure airline transfer after ten. Restricted to airline rounds 0–9 of r1–r3, the trajectories full evaluation drove, full evaluation’s decision value was +0.68 points per round [+0.08, +1.59] and sequential full evaluation’s +0.58 [+0.07, +1.35], a difference of −0.10 [−0.35, +0.07]; sequential evaluation kept 63% of full evaluation’s ten acceptances at 69% of its episodes. In this restricted set the reference gains, unlike in the primary analysis. Because an accepted candidate becomes the next incumbent and every incumbent is deployed, the sum of full evaluation’s round values over rounds 0–9 equals the trajectory’s transfer at round 10 (+8.75, +8.75 and +2.92 on r1–r3). The same sum for sequential evaluation, whose decisions are scored in full evaluation’s states, is +5.76, +9.30 and +2.26: offline it would end 1.0 point lower [−3.7, +0.8] (bootstrap of incumbent blocks and tasks). Adding r4 gives −0.9; r2 and r3 alone give −0.1. The live difference, −4.4, exceeds the offline prediction by 3.4 points [−8.7, +1.3] (independent bootstraps; the live loss is larger in 88% of draws). In the live loops’ own states the rule kept 14 of full evaluation’s 20 acceptances (70%), as offline, so the per-decision behaviour transferred and the cumulative loss did not match.

Noise bands (post hoc). Each loop calibrates its noise band δ from three evaluations of the base harness before round 0, which its selection step cannot influence; δ ranged from 2.7 to 12.5 points over the twelve loops (17.8 on r1). By chance the sequential loops’ bands were wider, 9.8 points on average against 6.7 (concurrent loops 9.7 against 6.4). A wider band leaves more candidates to RRSI’s in-band branch, where admission turns on cost or novelty and the sequential rule stops about a third of admissible candidates (Table 31), and δ correlated with transfer in both arms (−0.57 sequential, −0.63 full). Regressing transfer on arm and centred δ gives an arm difference of −2.1 [−6.2, +1.9] for the six loops per arm ($p = 0.35$; slope −0.76 points of transfer per point of δ), −5.8 [−9.4, −2.2] for the eight concurrent loops ($p = 0.02$) and −4.0 [−7.6, −0.3] with r1 (t intervals; a bootstrap over loops and tasks gives [−7.1, +3.4], [−10.9, −1.2] and [−8.5, +1.2]). The adjustment was not registered, and the band may also moderate the rule’s loss rather than only shift transfer;

the concurrent comparison, in which both arms ran together, keeps its difference.

What the shadow evaluations separate (post hoc). A sequential loop differs from a full-evaluation loop in admission, since a stopped candidate cannot be accepted, and in history, since a stopped candidate enters the proposer’s history as rejected with its early-stopped estimate. In the sequential loops’ states, full evaluation would have accepted in 20 of 60 rounds, against 23 acceptances in the full-evaluation loops’ own 60 rounds; the sequential loops made 14 of the 20. Of the 9 fewer accepted changes, 6 were acceptances the rule stopped and 3 reflect fewer rounds with an admissible candidate. The candidates themselves, scored by complete evaluation against their own incumbents, were similar in the two arms: 86 and 88 evaluated, 27% and 32% admissible, mean change in evolve success -1.5 and -1.1 points, 36% and 35% positive. The 62 stopped candidates reached the proposer after 29 of 60 episodes on average, with estimates of -4.7 points against -2.5 under complete evaluation (correlation 0.53, opposite signs in 18%); 51 of them were stopped in rounds where the decision agreed with full evaluation’s, so the history differed even where admission did not. Of the lost acceptances, only `cl1`’s two were deployed ($+3.75$ and $+4.17$ points over their incumbents). These descriptives do not separate the mechanisms: weaker incumbents make improvements easier to find, which could hide worse proposals, and no loop restored a lost acceptance or fed complete estimates to a sequential loop’s proposer.

T Reproducibility

The code, environments, banks and Study 2’s registered pipeline are submitted as anonymised supplementary material. *Study 1:* the environments and fault suites (`agent_exp/env.py`, `agent_exp/tau2_env.py`), every prompt used for attribution, taxonomy induction, labelling, patch generation and judging (`agent_exp/analyst.py`), the agent and analyst settings (DeepSeek-V4-Flash and DeepSeek-V4-Pro through the provider’s Anthropic-compatible endpoint, at most 4,000 output tokens per agent call and 8,000 per structured analyst call, provider-default sampling), and the complete intervention banks (`runs/tau2_*/bank.json`: every replay, judge check, null replay and regression run with its token count and source trace). Bank con-

struction is deterministic given the seeds in `agent_exp/bank.py`. Every table and figure of Study 1 is produced offline from the banks and held-out runs by `agent_exp.report_tau2`, from the outputs of `agent_exp.reanalysis`, `reanalysis3`, `reanalysis5` (binomial harm model), `deployment` (deployment gains, $\text{Net}(N)$, transfer) and `judge_configs`. *Study 2:* the vendored RRSI loop with a list of our changes (`rrsi/`, `rrsi/NOTICE.md`), the τ^2 domains (`domains/tau2*`), the harness check against τ^2 ’s orchestrator (`agent_exp/harness_check.py`), and the evidence pipeline, pre-registration and registered analysis (`verify/`); Table 21 is produced by `verify.report_e1` from the output of `verify.analyze`, the analyses with the common error check by `verify.posthoc_e1`, `r2` and `r3`’s registered analysis and Table 19 by `verify.e1r` and `verify.report_e1r`, the closed loops by `verify/run_cl.sh`, `verify.cl`, `verify/run_il.sh` and `verify.il` (Tables 36 and 37), and the post hoc analyses of Appendices R and S by `verify.posthoc_r9` and `verify.posthoc_r10`. Re-collecting any run needs API access and is not bit-reproducible, because the provider does not offer seeded sampling.

U The CARVE Design Study

Why the simulator was optimistic. Our first design study, CARVE, used a simulator in which it needed half the budget of the strongest baseline (Appendix U). Calibrated to the measured judge accuracy, spurious recovery, analyst accuracy and cost ratio, the simulator still predicts that CARVE leads uncertainty sampling by 14 points on retail at budget 50; on the bank it trails by 15 points at budget 40 (significant harm, $c_w = 0.02$). The simulator shared the method’s structure: one b for all categories, value spread over many edges, and a fixed cost c that made every wrong change harmful, an assumption that was never measured and on which the value of verification largely depends.

Our first approach treated judge checks as cheap and replays as expensive, and allocated a budget over both by Bayesian experimental design. We keep it here as the multi-fidelity method compared in §4, together with the simulator in which it was developed. The simulator shares the method’s structure. Its results show the method working under its own assumptions, and they are not evidence about

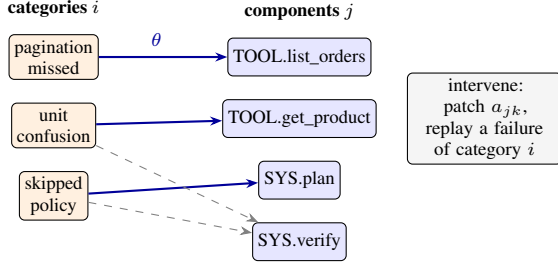


Figure 3: The category–component graph. Solid edges are true causal edges; dashed ones are plausible but wrong attributions (e.g. blaming verification for a unit error). Each intervention patches one component and replays one failure.

real agents.

Calibrated simulation. With the measured judge accuracy, spurious recovery, analyst accuracy and cost ratio, the simulator predicts that CARVE runs no judge checks on ShopDesk and retail and only a few on airline, which matches the break-even analysis in Table 4. For τ^2 retail it predicts net gains at budget 50 of 0.51 for CARVE and 0.38 for uncertainty sampling; for airline, 0.33 and 0.31; for ShopDesk, 0.81 and 0.79.

U.1 Problem Formulation

An agent is built from *components* $j \in \{1, \dots, J\}$: sections of its system prompt, tool descriptions, tool implementations, orchestration settings. Its failures have been grouped into *categories* $i \in \{1, \dots, I\}$ by an LLM, with f_i the share of failures in category i . For every cell (i, j) an analyst LLM can propose K candidate patches a_{ijk} that rewrite component j to address category i .

Interventions. An intervention (i, j, k, ϕ) samples a failed trajectory labelled i , applies patch a_{ijk} , and observes a binary outcome at fidelity ϕ . A *full replay* ($\phi = F$) resumes the episode from the attributed failing step and records whether the task now succeeds. A *single-step check* ($\phi = S$) regenerates only the failing step and asks a judge whether the diagnosed error is gone. They cost c_F and $c_S \ll c_F$.

Edges and patch quality. We write $e_{ij} \in \{0, 1\}$ for whether component j is a cause of category i , meaning some change to j repairs a non-trivial fraction of its failures, and $q_{ijk} \in [0, 1]$ for the repair probability of patch k when $e_{ij} = 1$. This separation matters because a negative replay outcome is ambiguous: either j is the wrong component or

a_{ijk} is a bad patch. Testing several patches per cell resolves the ambiguity.

Objective. After spending at most budget B , the method accepts a set of cells $\mathcal{A}$ and, for each, a patch k_{ij}^* . Accepting a true edge repairs a fraction q_{ijk^*} of category i ’s failures; accepting a false edge costs a regression c_{fp} , reflecting that unnecessary changes perturb behaviour on tasks that used to succeed (Vasudev et al., 2026). We measure

$$G(\mathcal{A}) = \sum_i f_i \left(1 - \prod_{j \in \mathcal{A}_i^+} (1 - q_{ijk_{ij}^*}) \right) - c_{fp} |\{(i, j) \in \mathcal{A} : e_{ij} = 0\}|, \quad (5)$$

where $\mathcal{A}_i^+ = \{j : (i, j) \in \mathcal{A}, e_{ij} = 1\}$. This is the net failure mass repaired; we report it as a fraction of the oracle value obtained by accepting exactly the true edges with their best patches.

U.2 CARVE

Observation model For a trajectory labelled i and patch a_{ijk} , the probability that a full replay succeeds is

$$p_{ijk} = \begin{cases} \lambda q_{ijk} + (1 - \lambda) b & e_{ij} = 1, \\ b & e_{ij} = 0, \end{cases} \quad (6)$$

where b is the spurious recovery rate (stochastic re-generation sometimes succeeds regardless of the patch) and λ is the accuracy of the category labels used to sample trajectories; a mislabelled trajectory is treated as unaffected by the patch. A single-step check returns $z = 1$ with probability $sp + \epsilon(1 - p)$, where s and ϵ are the sensitivity and false-positive rate of the check relative to the full outcome. All four nuisance parameters can be estimated from a small calibration set on which both fidelities and a human audit are run.

Attribution-informed prior Let n_{ij} be the number of failures in category i that the analyst attributed to component j , and $\hat{r}$ the analyst’s attribution accuracy estimated on a small audited sample. With d the expected number of causal components per category and $\rho = d/J$ the base edge density, we set

$$\pi_{ij} = (1 - \hat{r}) \rho + \hat{r} \min(1, dn_{ij}/n_i), \quad (7)$$

so that an unreliable analyst ($\hat{r} \rightarrow 0$) yields the flat prior ρ and a reliable one concentrates mass on the components it names. Eq. 7 is a heuristic interpolation, not a calibrated model; with a larger

audit, π could instead be fitted by logistic regression on audited attributions. Patch quality has prior $q_{ijk} \sim \text{Beta}(\alpha_q, \beta_q)$.

Posterior Cells are independent given the data. For each cell we keep the posterior log-odds of e_{ij} and, for each patch, a posterior over q_{ijk} on a fixed grid of $G = 101$ points. With $\mathcal{D}_{ijk}$ the outcomes observed for patch k ,

$$\text{logit } P(e_{ij}=1 \mid \mathcal{D}) = \text{logit } \pi_{ij} + \sum_k \log(\bar{L}_{ijk}^1 / L_{ijk}^0), \quad (8)$$

where $\bar{L}_{ijk}^1 = \int P(\mathcal{D}_{ijk} \mid q, e_{ij}=1) dP(q)$ is the marginal likelihood of patch k 's outcomes under a true edge and $L_{ijk}^0 = P(\mathcal{D}_{ijk} \mid e_{ij}=0)$. Every update multiplies one grid vector by a likelihood vector, so updates and look-aheads are vectorised dot products.

Selecting interventions The value of the current decision for cell (i, j) , with P_{ij} its edge posterior and $\hat{Q}_{ij} = \max_k \mathbb{E}[q_{ijk} \mid \mathcal{D}]$, is

$$V_{ij} = \max(P_{ij} f_i \hat{Q}_{ij} - (1 - P_{ij}) c_{fp}, 0), \quad (9)$$

the expected contribution of the cell to Eq. 5 under the Bayes decision (accept iff the first term is positive), treating cells as additive. Eq. 5 combines components of one category multiplicatively, so V_{ij} slightly overstates the value of a second cause for an already repaired category; we accept this approximation because it keeps cells independent. For an action $a = (i, j, k, \phi)$ repeated n times, let $V'_{ij}(s)$ be the value after observing s successes. The n -step knowledge gradient is

$$\text{KG}_n(a) = \sum_{s=0}^n P(s \mid \mathcal{D}, a) V'_{ij}(s) - V_{ij}, \quad (10)$$

and CARVE runs the action maximising

$$\text{score}(a) = \max_{n \in \{1, 2, 4, 8\}} \frac{\text{KG}_n(a)}{n c_\phi}. \quad (11)$$

Looking ahead several observations matters because a single binary outcome often cannot flip a decision, in which case $\text{KG}_1 = 0$ even for a valuable cell; maximising over n is the standard remedy (Frazier et al., 2008). Because cells are independent, only the observed cell's scores change after each outcome, so each step rescores one cell in $O(KG)$ time after an initial pass over all cells.

Decisions and stopping. At any point the method accepts the cells whose current value is positive and applies, for each, the patch with the highest posterior mean quality. It stops early when no action has a score above a small threshold, i.e. when no remaining experiment is expected to be worth its cost.

U.3 When Do Priors and Cheap Checks Help?

These results analyse an idealised per-cell sequential test, not CARVE's full policy; they explain which quantities govern the method's behaviour. We consider a single cell and a single patch with known success probabilities p_1 (true edge) and $p_0 = b$, under a rule that stops when the posterior log-odds leaves $[-A, A]$, $A = \log \frac{1-\delta}{\delta}$. Let $D_1 = \text{KL}(p_1 \parallel p_0)$ and $D_0 = \text{KL}(p_0 \parallel p_1)$ be the expected log-likelihood-ratio drift per observation under each hypothesis. Proofs, which follow Wald's approximations (Wald, 1945), are in Appendix U.5.

Proposition 3 (Value of the LLM prior). *Let $|\ell| < A$ and neglect overshoot and stopping at the wrong boundary (whose probability is approximately $\delta e^{-\ell}$ for a true edge and δe^ℓ for a false one). A cell with initial log-odds ℓ then needs $N_1(\ell) = (A - \ell)/D_1$ observations in expectation if it is a true edge and $N_0(\ell) = (A + \ell)/D_0$ if not. Replacing a flat prior ℓ_ρ by LLM-derived log-odds ℓ_{ij} saves, in expectation over cells,*

$$\Delta = \frac{\rho \mathbb{E}[\delta \ell \mid e=1]}{D_1} - \frac{(1 - \rho) \mathbb{E}[\delta \ell \mid e=0]}{D_0}, \quad (12)$$

with $\delta \ell = \ell_{ij} - \ell_\rho$, observations per cell, which is positive exactly when the prior raises true edges more, relative to their base rate and drift, than it raises false ones.

Under Eq. 7, $\ell_{ij} \rightarrow \ell_\rho$ as $\hat{r} \rightarrow 0$, so $\Delta \rightarrow 0$: a useless analyst costs nothing beyond the flat prior, whereas trusting its labels outright (LLM-only) makes permanent errors. A miscalibrated analyst can make Δ negative, but only by an amount linear in the log-odds shift, after which the evidence takes over. Appendix U.7 compares these predictions with simulated sequential tests.

Proposition 4 (Cheap checks). *Let D_e^S denote the drifts of single-step outcomes, $z \sim \text{Bern}(s p_e + \epsilon(1 - p_e))$. Under the same approximations, a test using only single-step checks reaches the stopping boundary at lower expected cost than one using only full replays if and only if*

$$\frac{c_S}{c_F} < \frac{D_e^S}{D_e}, \quad (13)$$

where e is the true state of the cell.

Since e is unknown, the proposition gives a clear rule only when the cost ratio lies outside the interval spanned by the two critical ratios; inside it, which fidelity is better depends on the posterior, which is what CARVE’s look-ahead weighs.

With our default parameters ($p_1 = 0.54$, $b = 0.05$, $s = 0.85$, $\epsilon = 0.15$) the critical ratio is 0.31 for true edges and 0.44 for false ones; with a weak check ($s = 0.65$, $\epsilon = 0.35$) it falls to 0.05–0.08. Section U.4 tests both predictions.

U.4 Controlled Experiments

Real agents do not reveal which component truly caused a failure, so we first evaluate in a testbed where the causal graph is known and the method’s model is deliberately misspecified.

Setup

Testbed. Each world has $I = 12$ categories with Zipfian frequencies, $J = 20$ components and $K = 3$ patches per cell. Every category has one or two true components and one *decoy* component that wrong attributions prefer, mimicking symptom-level mislabelling. Patch quality is bimodal: with probability 0.3 a patch is bad ($q \sim \text{Beta}(1, 9)$), otherwise $q \sim \text{Beta}(6, 3)$. A trajectory sampled for category i is mislabelled with probability $1 - \lambda = 0.2$, in which case it comes from another category (weighted by frequency) and is repaired only if *its own* edge to j exists, unlike the model’s flat treatment of mislabelled traces. The analyst attributes 20 failures per category with accuracy $r = 0.5$; wrong attributions go to the decoy with probability 0.6. The analyst’s accuracy is estimated from an audit of 20 attributions. Costs are $c_F = 1$, $c_S = 0.1$, $c_{fp} = 0.02$, and $s = 0.85$, $\epsilon = 0.15$, $b = 0.05$. Section U.4 relaxes the assumptions the generator shares with the model. Full details are in Appendix U.6.

Baselines. *LLM-only* accepts, for every category, the most-attributed component and any component holding at least 40% of the attributions, with the first patch, i.e. it acts on an induced taxonomy without verification. *Replay-each* tests attributed hypotheses in order of attribution count with three full replays of the first patch and accepts a hypothesis if at least two succeed, mirroring do-then-verify methods such as DoVer; *Replay-each, 3 patches* gives each hypothesis two replays of

Method	Budget (full-replay units)			
	20	50	100	200
LLM attribution only	0.297 $\pm$ 0.021	0.297 $\pm$ 0.021	0.297 $\pm$ 0.021	0.297 $\pm$ 0.021
Replay-each (do-then-verify)	0.255 $\pm$ 0.021	0.270 $\pm$ 0.021	0.466 $\pm$ 0.027	0.468 $\pm$ 0.026
Replay-each, 3 patches	0.301 $\pm$ 0.025	0.317 $\pm$ 0.026	0.401 $\pm$ 0.026	0.612 $\pm$ 0.026
Sequential-each (adaptive stopping)	0.426 $\pm$ 0.016	0.522 $\pm$ 0.018	0.643 $\pm$ 0.016	0.813 $\pm$ 0.014
Thompson sampling	0.407 $\pm$ 0.020	0.419 $\pm$ 0.020	0.500 $\pm$ 0.017	0.625 $\pm$ 0.014
Uncertainty sampling	0.455 $\pm$ 0.016	0.581 $\pm$ 0.019	0.690 $\pm$ 0.017	0.804 $\pm$ 0.016
Uncertainty + cheap checks	0.549 $\pm$ 0.018	0.662 $\pm$ 0.016	0.784 $\pm$ 0.015	0.876 $\pm$ 0.009
CARVE (all components)	0.666 $\pm$ 0.019	0.787 $\pm$ 0.015	0.853 $\pm$ 0.011	0.902 $\pm$ 0.009
full replay only	0.514 $\pm$ 0.016	0.609 $\pm$ 0.020	0.697 $\pm$ 0.017	0.825 $\pm$ 0.012
myopic ($n=1$ look-ahead)	0.591 $\pm$ 0.016	0.652 $\pm$ 0.014	0.706 $\pm$ 0.016	0.729 $\pm$ 0.014
uninformed prior	0.347 $\pm$ 0.019	0.474 $\pm$ 0.017	0.631 $\pm$ 0.015	0.800 $\pm$ 0.012
one patch per cell	0.571 $\pm$ 0.019	0.636 $\pm$ 0.020	0.673 $\pm$ 0.019	0.688 $\pm$ 0.020
no label-noise term	0.663 $\pm$ 0.020	0.772 $\pm$ 0.018	0.864 $\pm$ 0.012	0.922 $\pm$ 0.006

Table 38: Net gain as a fraction of the oracle (mean $\pm$ s.e., 50 worlds). The lower block ablates CARVE.

each of three patches and accepts if any patch succeeds twice; *Sequential-each* replays each hypothesis, cycling through patches, until CARVE’s own posterior leaves $[0.05, 0.95]$. Untested hypotheses fall back to LLM-only, except for *Sequential-each*, which uses the Bayes decision. *Thompson sampling* samples (e, q) from the posterior and replays the cell and patch with the largest sampled repair. *Uncertainty sampling* replays the cell with the largest frequency-weighted uncertainty $f_i \min(P_{ij}, 1 - P_{ij})$, cycling through patches, at full fidelity; *Uncertainty + cheap checks* first runs three single-step checks per (cell, patch) and then confirms with full replays. All baselines except LLM-only and the *Replay-each* variants share CARVE’s posterior and decision rule, so comparisons among them isolate the selection rule.

Metrics. Net gain (Eq. 5) as a fraction of the oracle, and edge F1. We report means and standard errors over 50 worlds for the main experiment and 30 for the others; all methods see the same worlds, so differences are paired.

Main results Figure 4 and Table 38 show that CARVE is best at every budget. With 50 units it recovers 79% of the oracle gain; the strongest baseline, uncertainty sampling with the same cheap checks, recovers 66% and needs about twice the budget (100 units, 78%) to match it. The gap narrows at large budgets (90% vs. 88% at 200), as every reasonable method eventually resolves the frequent categories.

Where the gain comes from. Two comparisons separate the selection rule from the cheap checks. With full replays only, CARVE beats uncertainty sampling by 1–6 points (e.g. 0.609 vs. 0.581 at budget 50): the knowledge-gradient rule alone helps only modestly. Given cheap checks, its advantage grows to 7–13 points (0.787 vs. 0.662 at 50), be-

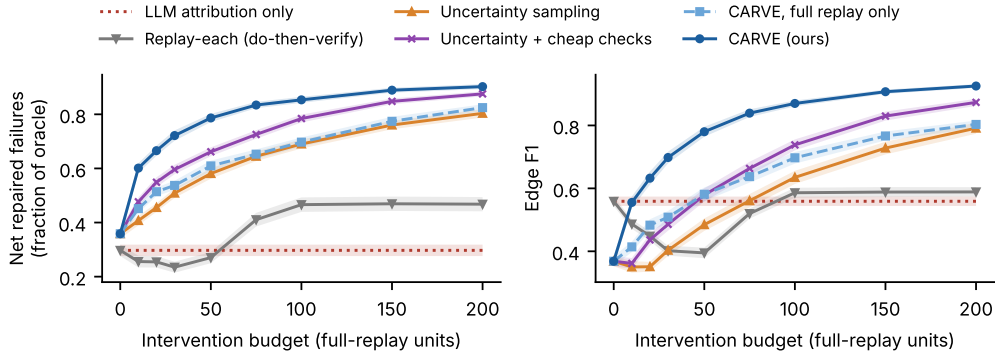


Figure 4: Net repaired failure mass (left) and edge F1 (right) against intervention budget, 50 worlds, shaded ± 1 s.e.

cause it decides per action whether a check is informative enough for its price and stops checking a cell once checks can no longer change the decision, whereas a fixed check-then-confirm schedule wastes checks on resolved cells. Most of CARVE’s advantage is therefore in *how it uses* cheap checks, which also makes their calibration critical (§U.4).

Verification baselines. LLM-only is stuck at 30% because it accepts decoys and bad first patches. Replay-each is initially *worse* than not verifying: with $p_1 \approx 0.54$, requiring two successes in three replays rejects about 44% of true edges even when the patch is good, and its fixed per-hypothesis cost exhausts small budgets on frequent but already-certain hypotheses. Testing three patches (61% at 200) or stopping adaptively (81% at 200) helps, but both remain well below CARVE at small budgets (32% and 52% at 50). Thompson sampling spends most replays on cells it already believes in and reaches only 63% at 200.

Ablations. Removing single-step checks costs 15–18 points at small budgets. Restricting the look-ahead to one observation is below the full method at every budget and plateaus at 73%, because many cells cannot be flipped by a single outcome and are never revisited. A flat prior costs 32 points at budget 20 and still 10 points at 200. With one patch per cell the method plateaus at 69%: it cannot tell a wrong component from a bad patch and rejects true edges for good. Dropping the label-noise term makes no difference at the default label accuracy $\lambda = 0.8$ (it is even slightly better at large budgets, 0.922 vs. 0.902 at 200), but it matters when labels are worse (§U.4).

When does the LLM prior help? Figure 5 varies the analyst’s accuracy r . As Proposition 3 suggests, CARVE improves monotonically with r

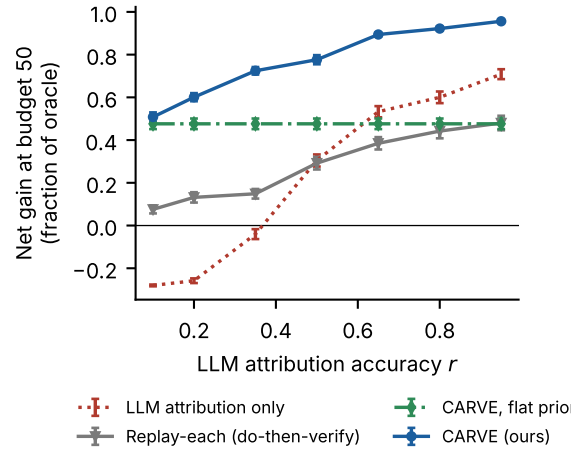


Figure 5: Net gain at budget 50 as the analyst’s attribution accuracy r varies (30 worlds, ± 1 s.e.).

and is never below the flat-prior variant: at $r = 0.1$ it scores 0.51 ± 0.02 against 0.48 ± 0.03 . The prior is not exactly flat there (the decoy still receives about twice the base rate), but its shifts are small, so evidence quickly dominates. Acting on attributions directly is harmful below $r \approx 0.35$, where LLM-only has a *negative* net gain (regressions outweigh repairs), and even at $r = 0.95$ it reaches only 71% of the oracle against 96% for CARVE, since it cannot detect bad patches. Replay-each improves with r and only matches the flat-prior variant of CARVE at $r = 0.95$ (0.48 vs. 0.48).

When do cheap checks help? Figure 7 (Appendix U.8) is qualitatively consistent with Proposition 4. With the default check, the benefit of cheap checks shrinks as their cost grows and vanishes between 0.35 and 0.5 (0.64 ± 0.02 vs. 0.61 ± 0.03 at 0.35, overlapping; 0.59 vs. 0.61 at 0.5), in line with the predicted critical ratios of 0.31 and 0.44.

Condition (budget 50)	Unc. + checks	CARVE
Default testbed	0.656 ± 0.022	0.776 ± 0.021
Label accuracy $\lambda = 0.5$	0.513 ± 0.023	0.688 ± 0.023
Analyst overconfident ($\hat{r}+0.3$)	0.752 ± 0.018	0.857 ± 0.014
Analyst underconfident ($\hat{r}-0.3$)	0.495 ± 0.023	0.653 ± 0.032
Judge fooled by bad patches	0.579 ± 0.022	0.377 ± 0.027
Regression cost $c_{fp} = 0.005$	0.711 ± 0.022	0.798 ± 0.016
Regression cost $c_{fp} = 0.05$	0.643 ± 0.024	0.793 ± 0.019
Regression cost $c_{fp} = 0.1$	0.624 ± 0.023	0.823 ± 0.018

Table 39: Violations of the model’s assumptions (30 worlds). The judge-fooled row is the one setting where CARVE loses.

At cost 0.1, a check of quality $s - \epsilon = 0.5$ (critical ratio 0.14–0.22) still helps, while one of quality 0.3 (critical ratio 0.05–0.08) brings no gain. At cost 0.5 CARVE is slightly below its full-replay-only variant although its action set contains every full-replay action, which shows that the greedy rule is not exactly cost-optimal when the two fidelities are nearly equally efficient.

Robustness The generator shares several assumptions with the model: check errors depend only on the full outcome, cells are independent, the analyst’s accuracy is estimated without bias, and the regression cost is small. Table 39 and Appendix U.8 relax each in turn.

Worse labels. With label accuracy $\lambda = 0.5$ both methods lose ground, and the label-noise term now matters: CARVE scores 0.688, against 0.588 without the term and 0.513 for the baseline.

Biased analyst audit. If the audit overstates the analyst’s accuracy by 0.3, CARVE actually improves (0.857), because a stronger prior concentrates the budget and the evidence corrects the decoys. Understating it by 0.3 costs 12 points (0.653), still above the baseline. Under-trusting a decent analyst is thus the more costly error in this testbed.

Regression cost. Raising c_{fp} from 0.005 to 0.1 leaves CARVE between 0.78 and 0.82, while LLM-only falls from 0.39 to -0.13 : the more costly a wrong change, the more verification is worth.

Joint causes. When 30% of categories need two components changed together (a single patch then repairs only a fifth as often), violating cell independence, CARVE drops from 0.787 to 0.661 at budget 50 but stays ahead of uncertainty sampling with checks (0.558; Table 42).

Correlated check errors. The one setting where CARVE loses is when the single-step judge is fooled

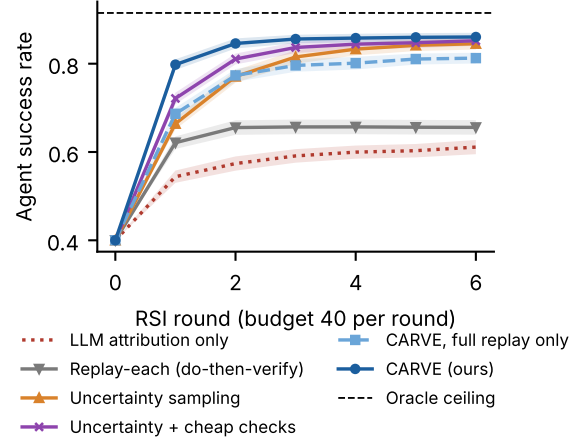


Figure 6: Agent success over six rounds of attribute-verify-patch, with a budget of 40 units per round (30 worlds, ± 1 s.e.).

by plausible-looking bad patches (false-positive rate 0.6 for patches with $q < 0.3$). CARVE then trusts checks that systematically pass bad patches and falls to 0.377, below every method that relies on full replays (0.58–0.61). Modelling the dependence (a quality-dependent false-positive rate in the likelihood) restores it to 0.584, on par with full replays, at a cost of 9 points when the judge is not fooled (0.684 vs. 0.776). Whether real judges behave this way is therefore the most important quantity to measure before relying on cheap checks.

Nuisance parameters and scale. Misspecifying b , λ or the patch-quality prior changes the result by at most 5 points (Table 43, Appendix U.8); misstating the check’s sensitivity and false-positive rate costs 6–14 points. On a larger graph ($I = 20$, $J = 50$; Table 41) the gap widens: at budget 50 CARVE reaches 78% of the oracle against 60% for uncertainty sampling with checks.

Inside a self-improvement loop Finally we embed each method in a loop. The agent starts at 40% success. In each round the analyst attributes 240 new failures, sampled in proportion to the current failure distribution; the method spends 40 units; accepted patches are applied, which repairs failures (true edges) or adds a regression of c_{fp} times the current failure mass (false edges); and the failure distribution is recomputed, so priorities shift towards categories that remain. Posteriors carry over between rounds.

Figure 6 shows that CARVE reaches 80% suc-

cess after one round and 85% after two, close to its plateau of 86% (the oracle ceiling is 91.5%). Uncertainty sampling with checks needs four rounds to reach 84% and ends at 85%; without checks it needs six. LLM-only and Replay-each plateau at 61% and 66%: once their accepted false edges and rejected true edges are fixed in place, new attributions do not correct them.

U.5 Proofs for CARVE’s propositions

Proposition 3. Let $L_t = \ell + \sum_{u \leq t} \log \frac{P(o_u|e=1)}{P(o_u|e=0)}$ be the posterior log-odds after t observations. Under $e = 1$, the increments are i.i.d. with mean $D_1 = \text{KL}(p_1||p_0) > 0$. The rule stops at $N = \inf\{t : L_t \notin (-A, A)\}$. By Wald’s identity, $\mathbb{E}_1[L_N - \ell] = D_1 \mathbb{E}_1[N]$. Wald’s approximations give $P_1(L_N \leq -A) \approx e^{-(A+\ell)} \approx \delta e^{-\ell}$ and $P_0(L_N \geq A) \approx \delta e^{\ell}$. Neglecting overshoot and this probability (which requires $\delta e^{|\ell|} \ll 1$), $L_N = A$, hence $\mathbb{E}_1[N] = (A - \ell)/D_1$; if $\ell \geq A$ no observation is taken, which is why we assume $|\ell| < A$. Symmetrically, under $e = 0$ the drift is $-D_0$ and $\mathbb{E}_0[N] = (A + \ell)/D_0$. Taking the difference between ℓ_ρ and ℓ_{ij} and averaging over cells with $P(e=1) = \rho$ gives Δ . When the patch quality q is unknown, outcomes are exchangeable rather than i.i.d. under the marginal likelihood, and Wald’s identity does not apply directly; conditioning on q recovers the result with $D_1 = D_1(q)$, which approaches zero for a bad patch. This is the formal reason a single bad patch can stall verification and why CARVE keeps several patches per cell.

Proposition 4. By the same argument a single-step test needs $(A \mp \ell)/D_e^S$ observations at cost c_S each, and a full-replay test needs $(A \mp \ell)/D_e$ at cost c_F . The single-step test is cheaper iff $c_S/D_e^S < c_F/D_e$. Since the single-step outcome is a garbling of the full outcome, the data-processing inequality gives $D_e^S \leq D_e$, so the critical ratio is at most one.

U.6 Simulation Details

Category frequencies are $f_i \propto 1/i$. For each category the number of true components is uniform on $\{1, 2\}$ and the decoy is a further distinct component. The audit estimate $\hat{r}$ is a binomial proportion from 20 audited attributions. The prior uses $d = 1.5$. The model’s grid has 101 points; look-ahead horizons are $\{1, 2, 4, 8\}$; the stopping threshold is 10^{-6} . Replay-each uses three replays and accepts with at least two successes. Uncer-

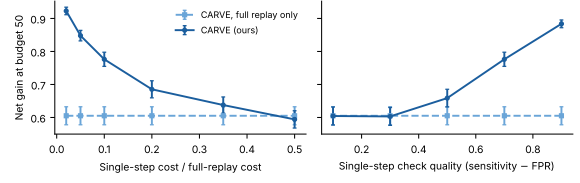


Figure 7: Effect of the single-step check’s cost (left, quality fixed at $s=0.85$, $\epsilon=0.15$) and quality (right, cost fixed at 0.1) on net gain at budget 50.

tainty sampling switches to an untried patch once the current patch has three full observations. In the loop experiment, 240 attributions per round are allocated to categories by a multinomial draw with the current failure distribution. All randomness is seeded; the code reproduces every number in this paper with `python -m sim.experiments all`.

U.7 Checking Proposition 3

Table 40 compares the predicted expected number of observations with simulated sequential tests ($p_1 = 0.543$, $p_0 = 0.05$, $\delta = 0.02$, 4000 runs per row). The predictions capture the direction and approximate size of the effect of the initial log-odds; they are least accurate when $|\ell|$ is large, where overshoot and wrong-boundary exits (last column) are no longer negligible.

ℓ	e	predicted N	simulated N	wrong exit
-3	1	7.17	5.94	0.242
-3	0	1.55	2.54	0.000
-2	1	6.13	6.28	0.117
-2	0	3.28	3.73	0.001
-1	1	5.09	5.79	0.047
-1	0	5.02	5.21	0.003
+0	1	4.05	4.76	0.012
+0	0	6.75	7.28	0.009
+1	1	3.01	4.27	0.006
+1	0	8.48	8.58	0.015
+2	1	1.97	2.97	0.001
+2	0	10.22	10.23	0.068

Table 40: Proposition 3: predicted vs. simulated observations to a decision.

U.8 Additional Results

Method	50	100	300
LLM attribution only	0.231 $\pm$ 0.033	0.231 $\pm$ 0.033	0.231 $\pm$ 0.033
Replay-each (do-then-verify)	0.224 $\pm$ 0.034	0.400 $\pm$ 0.037	0.531 $\pm$ 0.032
Uncertainty sampling	0.502 $\pm$ 0.022	0.637 $\pm$ 0.020	0.822 $\pm$ 0.015
Uncertainty + cheap checks	0.603 $\pm$ 0.019	0.746 $\pm$ 0.018	0.873 $\pm$ 0.013
CARVE, full replay only	0.613 $\pm$ 0.013	0.710 $\pm$ 0.012	0.834 $\pm$ 0.010
CARVE (ours)	0.780 $\pm$ 0.016	0.860 $\pm$ 0.011	0.933 $\pm$ 0.008

Table 41: Net gain on a larger graph ($I=20$, $J=50$), 20 worlds.

Method	20	50	100	200
LLM attribution only	0.228 $\pm$ 0.028	0.228 $\pm$ 0.028	0.228 $\pm$ 0.028	0.228 $\pm$ 0.028
Replay-each	0.202 $\pm$ 0.027	0.207 $\pm$ 0.023	0.348 $\pm$ 0.033	0.362 $\pm$ 0.036
Thompson sampling	0.325 $\pm$ 0.031	0.343 $\pm$ 0.032	0.398 $\pm$ 0.032	0.484 $\pm$ 0.028
Uncertainty sampling	0.345 $\pm$ 0.036	0.465 $\pm$ 0.035	0.544 $\pm$ 0.035	0.662 $\pm$ 0.029
Uncertainty + cheap checks	0.423 $\pm$ 0.037	0.558 $\pm$ 0.035	0.615 $\pm$ 0.034	0.685 $\pm$ 0.030
CARVE (ours)	0.573 $\pm$ 0.029	0.661 $\pm$ 0.029	0.701 $\pm$ 0.027	0.745 $\pm$ 0.026

Table 42: Net gain when 30% of categories need two components changed together (30 worlds).

Model assumption (true value)	Net gain
All nuisance parameters as in the generator	0.776 $\pm$ 0.021
Spurious rate b : 0.01 (0.05)	0.811 $\pm$ 0.012
Spurious rate b : 0.15 (0.05)	0.826 $\pm$ 0.016
Label accuracy λ : 1.0 (0.8)	0.768 $\pm$ 0.022
Label accuracy λ : 0.6 (0.8)	0.794 $\pm$ 0.017
Check s/ϵ : .95/.05 (.85/.15)	0.717 $\pm$ 0.016
Check s/ϵ : .70/.30 (.85/.15)	0.636 $\pm$ 0.021
Patch prior Beta(1, 1) (Beta(2, 2))	0.811 $\pm$ 0.015
Patch prior Beta(5, 2) (Beta(2, 2))	0.777 $\pm$ 0.020

Table 43: CARVE with misspecified nuisance parameters, budget 50, 30 worlds. Several “wrong” values do slightly better than the generator’s, a reminder that the model is misspecified in other ways too.